

blastFoam Theory and User Guide

Version 7.0

Jeff Heylmun, Peter Vonk,
Benjamin Shields, and Timothy Brewer
Synthetik Applied Technologies

info@synthetik-technologies.com

<https://github.com/synthetik-technologies/blastFoam>

July 28, 2025

Abstract

blastFoam is an open-source toolbox for simulating detonations based on the OpenFOAM[®]¹ framework (OpenCFD Ltd., 2018a). blastFoam provides solutions to highly compressible systems including single- and multi-phase compressible flow, including single- and multi-velocity systems, based on the work of Zheng et al. (2011), Shyue (2001), and Houim and Oran (2016). blastFoam provides implementations of the essential numerical methods (e.g. 2nd and 3rd order schemes), equations of state (e.g. ideal gas, stiffened gas, Jones-Wilkins-Lee, etc.), run-time selectable flux schemes (e.g. HLL, HLLC, AUSM+, Kurganov/Tadmor), and high-order explicit time integration (e.g. 2nd, 3rd, and 4th order). blastFoam provides activation and explosive burn models to simulate the initiation and expansion of energetic materials, as well as afterburn models to simulate under-oxygenated explosives that exhibit delayed energy release.

¹*DISCLAIMER: This offering is not approved or endorsed by OpenCFD Limited, producer and distributor of the OpenFOAM software via www.openfoam.org, and owner of the OpenFOAM and OpenCFD trade marks.*

Contents

1	Introduction	10
1.1	System Requirements	10
1.2	Downloading	11
1.3	Compilation	11
1.4	Libraries	12
1.5	Executables	12
1.6	Getting Help	12
2	Governing equations	13
2.1	Single-fluid model	13
2.2	Multi-fluid	15
2.2.1	Granular phases	17
3	Interfacial models	22
3.1	Drag models	22
3.1.1	SchillerNaumann	23
3.1.2	WenYu	23
3.1.3	Ergun	23
3.1.4	Gibilaro	24
3.1.5	SyamlalOBrien	24
3.1.6	GidaspowErgunWenYu	24
3.1.7	GidaspowSchillerNaumann	25
3.2	Lift models	25
3.2.1	constant	25
3.3	Heat transfer models	26
3.3.1	RanzMarshall	26
3.3.2	noneNu	26
3.4	Nusselt number models	26
3.4.1	constant	27
3.4.2	duct	27
3.4.3	GelperinEinstein	27
3.4.4	Gunn	27
3.4.5	RanzMarschall	27
3.4.6	sphere	27
3.5	Mass transfer models	28

3.5.1	none	28
3.5.2	reactingParticle	28
3.6	Interfacial pressure models	28
3.6.1	single	28
3.6.2	volumeAveraged	28
3.6.3	totalPressure	28
3.7	Interfacial velocity models	29
3.7.1	single	29
3.7.2	massAveraged	29
4	Granular models	30
4.1	Radial distribution models	30
4.1.1	CarnahanStarling	30
4.1.2	SinclairJackson	31
4.1.3	LunSavage	31
4.1.4	Gao	31
4.1.5	Lebowitz	31
4.2	Granular viscosity models	32
4.2.1	Gidaspow	32
4.2.2	HrenyaSinclair	32
4.2.3	Princeton	33
4.2.4	Syamlal	33
4.2.5	none	33
4.3	Granular conductivity models	34
4.3.1	Chao	34
4.3.2	Gidaspow	34
4.3.3	HrenyaSinclair	34
4.3.4	Princeton	35
4.3.5	Syamlal	35
4.4	Granular pressure models	36
4.4.1	Huilin	36
4.4.2	Lun	36
4.5	Frictional stress models	36
4.5.1	JohnsonJackson	37
4.5.2	Schaeffer	37
4.5.3	JohnsonJacksonSchaeffer	38
4.5.4	Princeton	38
4.6	Packing limit models	39

4.6.1	constant	40
4.6.2	YuStandish	40
4.6.3	FedorsLandel	41
4.7	Granular drag models	41
4.7.1	Chao	41
4.7.2	Syamlal	42
5	Fluid and solid thermodynamic models	43
5.1	Thermodynamic models	44
5.1.1	eConst	45
5.1.2	ePolynomial	46
5.1.3	ePower	46
5.1.4	eIcoTabulated	47
5.1.5	eTabulated	47
5.1.6	hConst	47
5.1.7	hPolynomial	48
5.1.8	hPower	48
5.1.9	hIcoTabulated	48
5.1.10	hTabulated	49
5.1.11	JANAF	49
5.2	Equation of states	49
5.2.1	rhoConst	49
5.2.2	Internal energy based (Mie-Grüneisen form)	50
5.2.2.1	idealGas	50
5.2.2.2	stiffenedGas	51
5.2.2.3	linearTillotson	51
5.2.2.4	Tillotson	52
5.2.2.5	Tait	53
5.2.2.6	vanderWaals	54
5.2.2.7	LSZK	55
5.2.2.8	JWL	55
5.2.2.9	CochranChan	56
5.2.2.10	DoanNickel	57
5.2.3	Temperature-based	58
5.2.3.1	perfectGas	59
5.2.3.2	JWLC	59
5.2.3.3	AbelNobel	60
5.2.3.4	VanderWaals	60

	5.2.3.5	BKW	60
	5.2.3.6	BWR	61
	5.2.3.7	Murnaghan	61
5.2.4		tabulatedEOS	62
	5.2.4.1	BirchMurnaghan2	62
	5.2.4.2	BirchMurnaghan3	62
5.2.5		eTabulated/tabulatedMG	63
5.3		Fluid Transport	65
	5.3.1	const	65
	5.3.2	sutherland	65
	5.3.3	logPolynomial	66
	5.3.4	polynomial	66
	5.3.5	WLF	66
	5.3.6	icoTabulated	67
	5.3.7	tabulated	67
5.4		Solid Transport	67
	5.4.1	constIso	67
	5.4.2	constAnIso	68
	5.4.3	exponential	68
	5.4.4	polynomial	68
	5.4.5	tabulatedIso	68
	5.4.6	tabulatedAnIso	69
5.5		Basic thermodynamic model	69
5.6		Detonating thermodynamic model	69
	5.6.1	Activation Models	71
		5.6.1.1 none	72
		5.6.1.2 linear	72
		5.6.1.3 programmedIgnition	72
		5.6.1.4 pressureBased	73
		5.6.1.5 ArrheniusRate	74
	5.6.2	Afterburn Models	75
		5.6.2.1 none [default]	75
		5.6.2.2 constant	75
		5.6.2.3 linear	75
		5.6.2.4 Miller	76
5.7		Multicomponent thermodynamic model	76
5.8		Two/multiphase models	78
5.9		Initialization	79

5.10	Example	79
6	Turbulence	82
7	Region models	84
7.1	Boundary conditions	85
7.1.1	globalMapped	85
7.1.2	globalInterpolated	85
7.1.3	globalTemperatureCoupled	86
8	Diameter models	87
8.1	constant	87
8.2	constantMass	87
8.3	reacting	87
8.4	Reaction Rates	88
8.4.1	pressureBased	88
8.4.2	Arrhenius	88
9	Flux Evaluation	90
9.1	Riemann Solvers	92
9.1.1	HLL	92
9.1.2	HLLC	92
9.1.3	HLLCP	93
9.1.4	AUSM+	94
9.1.5	AUSM+up	95
9.1.6	AUSM+up (granular)	96
9.1.7	Tadmor/Kurganov	97
9.2	Riemann advection scheme	98
9.3	Time integration	99
9.3.1	Euler	100
9.3.2	RK1SSP	100
9.3.3	RK2	101
9.3.4	RK2SSP	101
9.3.5	RK3SSP	102
9.3.6	RK4	103
9.3.7	RK4SSP	103
10	fvMesh Motion Solvers	104

11 fvMesh Topo-Changers	105
11.1 fvMeshRefiner	105
11.1.1 Error estimators	106
11.1.1.1 cellSize	107
11.1.1.2 fieldValue	107
11.1.1.3 volumeFraction	107
11.1.1.4 delta	108
11.1.1.5 scaledDelta	108
11.1.1.6 gradient	108
11.1.1.7 densityGradient	109
11.1.1.8 Lohner	109
11.1.1.9 multiComponent	110
11.1.2 Example	111
11.2 burst	112
11.2.1 Burst models	112
11.2.1.1 pressure	113
11.2.1.2 impulse	113
11.2.1.3 pressureAndImpulse	113
11.2.1.4 field	113
12 fvMesh Distributors	115
12.1 redistributor	115
12.2 Example	115
13 Numerics	117
13.1 Lookup tables	117
13.1.1 Modifiers	117
13.1.2 Interpolation schemes	118
13.1.3 1D lookup tables	118
13.1.4 2D lookup table	118
13.1.5 3D lookup tables	119
13.2 Equations	119
13.3 Root solvers	119
13.4 Minimization	120
13.5 Integrators	121
14 Compressible systems	123

15 Solid models	124
16 Function objects	125
16.1 blastProbes	125
16.2 fieldMinMax (fieldMax)	126
16.3 writeTimeList	127
16.4 conservedQuantities	128
16.5 vtkTimeSeries surface writer	128
16.6 Schlieren (grad)	128
16.7 shadowgrap (laplacian)	128
16.8 blastMachNo	129
16.9 dynamicPressure	129
16.10overpressure	129
16.11impulse	130
16.12timeOfArrival	130
16.13blastSurfaces	130
16.14stopProbes	130
16.15TExposure	131
16.16thermalImpulse	131
17 Utilities	133
17.1 setRefinedFields	133
17.2 blastToVTK	138
17.3 createTimeSeries	139
17.4 mergeProbes	139
17.5 calculateImpulse	139
17.6 initializeAtmosphere	140
17.7 rotateFields	141
17.8 rotateConservativeFields	142
17.9 convertLagrangianPositions	142
17.10blastPruneTimes	143
17.11addInternalPatch	143
17.12addEmptyPatch	144
17.13blastConvertCaseFiles	144
18 fvModels and fvConstraints	146
19 Optional libraries	147

20 Solvers	148
20.1 blastFoam	148
20.2 blastEulerFoam	148
20.3 blastFSIFoam	149
20.4 blastXiFoam (Experimental)	149
20.5 blastReactingFoam (Experimental)	149
20.6 blastMultiRegionFoam (Experimental)	150
20.7 blastParcelFoam (Experimental)	150
20.8 blastEquation	150
21 Examples	152
21.1 Double Mach reflection	152
21.2 Shock Tube - Two Fluid	157
21.3 Two charge detonation	161
21.4 Mine (buried charge)	174
21.5 Bursting windows	185
References	193

1 Introduction

blastFoam is an opensource computational fluid dynamics (CFD) code based on the OpenFOAM C++ library (OpenCFD Ltd., 2018a) to simulate strongly compressible systems of equations with detonations. Along with the main solvers for simulating fluid flow, additional utilities have been added to simplify case setup, allow adaptive mesh refinement with load balancing based on the work of Rettenmaier et al. (2019), and facilitate blastFoam specific case post-processing.

The purpose of this guide is to serve as a reference for the governing equations and the models that have been implemented in blastFoam. The guide also provides a brief description of the capabilities of the solver and the new utilities that have been introduced on top of those available in the standard OpenFOAM. Many of the available models include options that the user can tune to best suit their simulations, and are given in a table at the end of each model description.

1.1 System Requirements

blastFoam currently builds against OpenFOAM-12². The following dependencies are required to successfully compile and run blastFoam:

1. A working installation of OpenFOAM-12 complete with libraries and headers produced by OpenFOAM-12, no other packages are required to run blastFoam applications. If OpenFOAM-12 is compiled from source, the [ThirdParty-12](#) repository source is also required. If a pre-compiled version of OpenFOAM-12 is used, this is automatically installed.
2. In order to create and view the postscript plots (e.g. **.eps* files), **gnu-plot** is available on all platforms. **EPS Viewer** can be used on Windows and **ghostview** can be used on linux. On macOS, **Preview** opens eps files natively.

²<https://github.com/OpenFOAM/OpenFOAM-12\protect\let\futurelet\@let@token\let\let\@xspace@maybespace\relax>

1.2 Downloading

The un-compiled source code can be obtained at: <https://github.com/synthetik-technologies/blastfoam>. Download or clone the source in the following location: *\$HOME/OpenFOAM/blastfoam*

```
mkdir -p $HOME/OpenFOAM # create the OpenFOAM directory
cd $HOME/OpenFOAM # go to the directory
git clone https://github.com/synthetik-technologies/blastfoam
```

1.3 Compilation

After cloning the the blastFoam source code from <https://github.com/synthetik-technologies/blastfoam> (see previous section) and installing any optional libraries, similar to OpenFOAM, add the following line to *\$HOME/.bashrc* file:

```
source ~/OpenFOAM/blastfoam/etc/bashrc
```

and run:

```
source ~/.bashrc
```

in any open terminals to update the BASH environment. This is used to specify the local directories included during blastFoam compilation, as well as make future developments easier and less intrusive.

Next run the **./Allwmake** command from the top blastFoam directory to compile the libraries and applications. Finally, ensure that OpenFOAM-12 and **blastFoam** have been installed and that the environment has been correctly setup by running one of the tutorial or validation cases.

Summary of steps to download, configure and compile blastFoam:

```
# 1. create the OpenFOAM directory if it does not exist
mkdir -p $HOME/OpenFOAM

# 2. go to the $HOME/OpenFOAM directory
cd $HOME/OpenFOAM

# 3. # clone the blastFoam repository
```

```

git clone https://github.com/synthetik-technologies/blastfoam

# go to the blastfoam directory
cd $HOME/OpenFOAM/blastfoam

# Source the etc/bashrc to your .bashrc file
# by runngin the following command in terminal
echo "source $HOME/OpenFOAM/blastfoam/etc/bashrc" >> $HOME/.bashrc

# Re-load and set the bash environment to compile blastFoam
source $HOME/.bashrc

# Compile blastFoam (for parallel use "-j")
./Allwmake

```

1.4 Libraries

The backend libraries used by the executables are stored within the `$BLAST_APPBIN` (`$FOAM_USER_APPBIN` by default but this can be overridden) directory should be on the `LD_LIBRARY_PATH` (default locations are handled by OpenFOAM).

1.5 Executables

The applications to solve these equations are executed by running the executable name (i.e. **blastFoam**). The executables are stored within the `$BLAST_APPBIN` (`$FOAM_USER_APPBIN` by default but this can be overridden) directory and can be run from any directory.

1.6 Getting Help

This guide is intended to cover the major points of the library. For more specific questions regarding the equations or models, the references listed in the class header files (*.H) should be consulted.

Please report bugs using the issues tab on the GitHub page:

<https://github.com/synthetik-technologies/blastfoam/issues>

2 Governing equations

Here the governing conservation equations used within blastFoam will be described. We will describe two cases: (1) the single-fluid case where a single shared velocity and energy is used and (2) the multi-fluid case where each fluid has a unique velocity and energy. The phrase "multi-phase" will be used to denote a fluid that uses multiple equations of states to describe the density-energy-pressure relation. For both variations $\mathbf{U}$ is the vector of conservative variables, $\mathbf{F}$ are the fluxes corresponding to the respective conservative variables, and $\mathbf{S}$ is a vector of source terms,

$$\partial_t \mathbf{U} + \nabla \cdot \mathbf{F} = \mathbf{S} \quad (2.1)$$

The exact form each of these take will depend on whether the single-fluid or multi-fluid is used. Each of these are presented in the following sections.

2.1 Single-fluid model

For a single phase with multiple species, the conservative variables and fluxes are defined as

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho Y_s \\ \rho \mathbf{u} \\ \rho E \end{pmatrix} \quad (2.2)$$

$$\mathbf{F} = \begin{pmatrix} \rho \mathbf{u} \\ \rho Y_s \mathbf{u} \\ \rho \mathbf{u} \otimes \mathbf{u} + p \mathbf{I} \\ (\rho E + p) \mathbf{u} \end{pmatrix} \quad (2.3)$$

$$\mathbf{S} = \begin{pmatrix} 0 \\ \dot{R}_s + \nabla \cdot \mathbf{J}_s \\ \dot{M} + \dot{M}_v \\ \dot{E} + \dot{E}_v \end{pmatrix} \quad (2.4)$$

where Y_s is the mass fraction of specie s , and $\mathbf{J}_s$ is the diffusion flux of species s . For multiple phases

$$\mathbf{U} = \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_n \\ \alpha_1 \rho_1 \\ \vdots \\ \alpha_n \rho_n \\ \rho \mathbf{u} \\ \rho E \end{pmatrix} \quad (2.5)$$

$$\mathbf{F} = \begin{pmatrix} \alpha_1 \mathbf{u} \\ \vdots \\ \alpha_n \mathbf{u} \\ \alpha_1 \rho_1 \mathbf{u} \\ \vdots \\ \alpha_n \rho_n \mathbf{u} \\ \rho \mathbf{u} \otimes \mathbf{u} + p \mathbf{I} \\ (\rho E + p) \mathbf{u} \end{pmatrix} \quad (2.6)$$

$$\mathbf{S} = \begin{pmatrix} \alpha_1 \nabla \cdot \mathbf{u} \\ \vdots \\ \alpha_n \nabla \cdot \mathbf{u} \\ 0 \\ \vdots \\ 0 \\ \dot{\mathbf{M}} + \dot{\mathbf{M}}_v \\ \dot{E} + \dot{E}_v \end{pmatrix} \quad (2.7)$$

where ρ is the mixture density, $\mathbf{u}$ is the mixture velocity, E is the total energy (thermal and kinetic), p is the pressure, and ρ_i and α_i are the density and volume fraction of each phase. The source terms, $\dot{R}_s$, $\dot{\mathbf{M}}$, and $\dot{E}$ are the specie production from reactions, momentum, and energy sources (including gravitational acceleration), and $\dot{\mathbf{M}}_v$ and $\dot{E}_v$ are viscous terms.

The sum of all volume fractions is defined to be one, i.e.

$$\sum_i \alpha_i = 1 \quad (2.8)$$

When only two phases are used, the second volume fraction is defined

$$\alpha_2 = 1 - \alpha_1 \quad (2.9)$$

Additionally, the mixture density is defined by

$$\rho = \sum_i \alpha_i \rho_i \quad (2.10)$$

When all phases are considered inviscid, $\dot{\mathbf{M}}_v = \dot{\mathbf{E}}_v = 0$. However, when viscosity is included,

$$\dot{\mathbf{M}}_v = \nabla \cdot \left(\rho \nu \left[\nabla \mathbf{u} + (\nabla \mathbf{u})^\top - \frac{2}{3} (\nabla \cdot \mathbf{u}) \mathbf{I} \right] \right) \quad (2.11)$$

and

$$\dot{\mathbf{E}}_v = \nabla \cdot \left(\rho \nu \left[\nabla \mathbf{u} + (\nabla \mathbf{u})^\top - \frac{2}{3} (\nabla \cdot \mathbf{u}) \mathbf{I} \right] \cdot \mathbf{u} \right) + \nabla \cdot (\alpha_T \nabla e) \quad (2.12)$$

where ν is the viscosity, α_T is the thermal diffusivity, and $\mathbf{I}$ is the identity matrix.

The five equation model is used for multiphase systems in which a single shared velocity, a shared internal energy and temperature is used, and individual phase masses are conserved. Pressure is defined using a specified EOS, where the mixture's internal energy, densities, and volume fraction are used to calculate the total pressure.

2.2 Multi-fluid

Similar to flows that use a single shared velocity and energy, the mass, momentum, and energy are all conserved. In contrast, however, each phase has its own set of mass, momentum, and energy, which is transferred between phases using interfacial transfer models (such as drag, heat transfer, etc.).

The conservation equations for the i^{th} fluid phase are

$$\mathbf{U}_i = \begin{pmatrix} \alpha_i \\ \alpha_i \rho_i \\ \alpha_i \rho_i \mathbf{u}_i \\ \alpha_i \rho_i E_i \end{pmatrix} \quad (2.13)$$

$$\mathbf{F} = \begin{pmatrix} \alpha_i \mathbf{u}_i \\ \alpha_i \rho_i \mathbf{u}_i \\ \alpha_i \rho_i \mathbf{u}_i \otimes \mathbf{u}_i + \alpha_i p_i \mathbf{I} \\ \alpha_i (\rho_i E_i + p_i) \mathbf{u}_i \end{pmatrix} \quad (2.14)$$

$$\mathbf{S} = \begin{pmatrix} \alpha_i \nabla \cdot \mathbf{u}_i + \sum_{i \neq j} \frac{\dot{\omega}_{ij}}{I(\rho_i, \rho_j)} \\ \sum_{i \neq j} \dot{\omega}_{ij} \\ \alpha_i \rho_i \mathbf{g} + \sum_{i \neq j} \dot{\mathbf{M}}_{ij} + \dot{\mathbf{M}}_{v,i} \\ \alpha_i \rho_i \mathbf{g} \cdot \mathbf{u}_i + \sum_{i \neq j} (\dot{E}_{ij} + \dot{\mathbf{M}}_{ij} \cdot \mathbf{u}_{\text{int},ij}) + \dot{E}_{\text{det},i} + \dot{E}_{v,i} \end{pmatrix} \quad (2.15)$$

where α_i is the phase volume fraction, ρ_i is the phase density, $\mathbf{u}_i$ is the phase velocity, E_i is the total phase energy, and p_i is the phase pressure. The source terms, $\dot{\mathbf{M}}_{ij}$ and $\dot{E}_{ij}$, are the momentum and energy sources that account for interactions between other phases, $\dot{\mathbf{M}}_{v,i}$ and $\dot{E}_{v,i}$ are viscous terms, $\mathbf{g}$ is the gravitational acceleration vector, $\dot{E}_{\text{det},i}$ is the energy added due to reactions, and $\mathbf{u}_{\text{int},ij}$ is the interfacial velocity between two phases. When mass transfer is used $\dot{\omega}_{i,j}$ is the mass transfer rate from the i^{th} phase to the j^{th} phase, and I is a switching function used to select the transferring quantity (Xue et al., 2011) and is defined as

$$I(x_i, x_j) = \xi_{i,j} x_j + (1 - \xi_{i,j}) x_i \quad (2.16)$$

and

$$\xi_{i,j} = \begin{cases} 1 & \text{if } \dot{\omega}_{i,j} > 0 \\ 0 & \text{else} \end{cases} \quad (2.17)$$

The viscous stress uses a Newtonian stress tensor defined as

$$\dot{\mathbf{M}}_{v,i} = \nabla \cdot \left(\alpha_i \rho_i \nu_i \left[\nabla \mathbf{u}_i + (\nabla \mathbf{u}_i)^\top - \frac{2}{3} (\nabla \cdot \mathbf{u}_i) \mathbf{I} \right] \right) \quad (2.18)$$

and

$$\begin{aligned}\dot{\mathbf{E}}_{v,i} = & \nabla \cdot \left(\alpha_i \rho_i \nu_i \left[\nabla \mathbf{u}_i + (\nabla \mathbf{u}_i)^\top - \frac{2}{3} (\nabla \cdot \mathbf{u}_i) \mathbf{I} \right] \cdot \mathbf{u}_i \right) \\ & + \nabla \cdot (\alpha_i \alpha_{T,i} \nabla e_i)\end{aligned}\quad (2.19)$$

where ν_i is the viscosity, $\alpha_{T,i}$ is the thermal diffusivity, and $\mathbf{I}$ is the identity matrix.

The momentum transfer term, $\dot{\mathbf{M}}_{ij}$, is defined as

$$\dot{\mathbf{M}}_{ij} = \dot{\mathbf{M}}_{D,ij} + \dot{\mathbf{M}}_{L,ij} + \dot{\mathbf{M}}_{VM,ij} + \dot{\mathbf{M}}_{WL,ij} + \dot{\mathbf{M}}_{TD,ij} + \dot{\omega}_{i,j} I(\mathbf{u}_i, \mathbf{u}_j) \quad (2.20)$$

where the summed terms, in order, represent drag, lift, virtual mass, wall lubrication, and turbulent dispersion, respectively.

The energy transfer term, $\dot{E}_{ij}$, is defined as

$$\dot{E}_{ij} = \dot{E}_{HT,ij} + \dot{\omega}_{i,j} I(E_i, E_j + h_{f_i} - h_{f_j}) \quad (2.21)$$

where $\dot{E}_{HT,ij}$ is the energy transfer rate due to convective heat transfer, and h_f is the heat of formation.

2.2.1 Granular phases

In contrast to a standard fluid (gas or liquid), a granular fluid tracks the phase mass, momentum, thermal energy, and granular energy. The last quantity, granular energy, is the energy that is present due to fluctuation in the phase velocity within a cell. This is the kinetic energy due to the local velocity variance. The more common quantity is the granular temperature, Θ_i , which is related to the granular energy by $\langle E \rangle_i = \frac{3}{2} \Theta_i$. The governing equations

are

$$\mathbf{U}_i = \begin{pmatrix} \alpha_i \rho_i \\ \alpha_i \rho_i \mathbf{u}_i \\ \alpha_i \rho_i e_i \\ \alpha_i \rho_i \langle E \rangle_i \end{pmatrix} \quad (2.22)$$

$$\mathbf{F} = \begin{pmatrix} \alpha_i \rho_i \mathbf{u}_i \\ \alpha_i \rho_i \mathbf{u}_i \otimes \mathbf{u}_i + (P_{s,i} + P_{\text{fric},i}) \mathbf{I} \\ \alpha_i \rho_i e_i \mathbf{u}_i \\ \alpha_i \rho_i \mathbf{u}_i \langle E \rangle_i \end{pmatrix} \quad (2.23)$$

$$\mathbf{S} = \begin{pmatrix} \sum_{i \neq j} \dot{\omega}_{ij} \\ \alpha_i \rho_i \mathbf{g} - \alpha_i \sum_{\ell} \nabla p_{\text{int},i\ell} + \sum_j \dot{\mathbf{M}}_{ij} + \dot{\mathbf{M}}_{v,i} \\ \sum_j \dot{e}_{ij} + \sum_k \dot{\gamma}_{ik} + \dot{e}_{\text{det},i} \\ - \sum_k \dot{\gamma}_{ik} - \sum_{\ell} \phi_{\text{prod},i\ell} \\ - P_{s,i} \nabla \cdot \mathbf{u}_i + \alpha_i \boldsymbol{\tau}_i : \nabla \mathbf{u}_i \\ + \nabla \cdot (\kappa_{s,i} \nabla \Theta_i) + \sum_{i \neq j} \dot{\omega}_{ij} I(\langle E \rangle_i, 0) \end{pmatrix} \quad (2.24)$$

where, again, α_i is the phase volume fraction, ρ_i is the phase density, $\mathbf{u}_i$ is the phase velocity, and e_i is the phase thermal energy. The summations $\sum_j$, $\sum_k$, and $\sum_{\ell}$ denote summations over all phases, all solid phases, and all fluid phases, respectively. The new quantities, $P_{s,i}$, $P_{\text{fric},i}$, $\langle E \rangle$, and p_{int} are the granular pressure, frictional pressure, granular energy, and interfacial pressure (generally taken as the gas phase pressure), respectively. The density is assumed to be constant, so the volume fraction is found by simply dividing the total phase mass by the known density. The source terms, $\dot{\mathbf{M}}_{ij}$, $\dot{e}_{ij}$, $\dot{\gamma}_{ij}$, and $\phi_{\text{prod},i\ell}$ are the momentum transfer, energy transfer, granular energy dissipation, and production terms. The momentum and energy transfer terms are the same as for a fluid phase ($\dot{e}_{ij} = \dot{E}_{ij}$ and $\dot{e}_{\text{det},i} = \dot{E}_{\text{det},i}$). The production and dissipation terms are described below.

The production and dissipation of granular energy introduce a new term into the continuous phase total energy equation so that the continuous phase

source terms become

$$\mathbf{S} = \begin{pmatrix} \alpha_i \nabla \cdot \mathbf{u}_i + \sum_{i \neq j} \frac{\dot{\omega}_{ij}}{I(\rho_i, \rho_j)} \\ \sum_{i \neq j} \dot{\omega}_{ij} \\ \alpha_i \rho_i \mathbf{g} + \sum_{i \neq j} \dot{\mathbf{M}}_{ij} + \dot{\mathbf{M}}_{v,i} \\ \alpha_i \rho_i \mathbf{g} \cdot \mathbf{u}_i + \sum_{i \neq j} \dot{E}_{ij} + \dot{E}_{\text{det},i} + \dot{E}_{v,i} + \sum_k \phi_{\text{prod},k} \\ \sum_k \dot{\omega}_{ik} I(0, \langle E \rangle_k) \end{pmatrix} \quad (2.25)$$

You can also see in Eq. (2.22) that the dissipation of granular energy is converted into thermal energy within the solid phase. For those familiar with incompressible flows, this term is typically neglected due to the fact that the amount of heat produced from collisions is typically very small. In highly compressible flows, this is not the case, and significant heat can be generated due to the loss of energy from inelastic collisions.

We define the change in momentum due to granular phase stress as

$$\dot{\mathbf{M}}_{ij} = \nabla \cdot \boldsymbol{\tau}_{s,i} + \dot{\omega}_{i,j} I(\mathbf{u}_i, \mathbf{u}_j) \quad (2.26)$$

and the granular stress is defined as

$$\boldsymbol{\tau}_{s,i} = \rho_i \nu_{s,i} [\nabla \mathbf{u}_i + (\nabla \mathbf{u}_i)^\top] + \left(\lambda_{s,i} - \frac{2}{3} \rho_i \nu_{s,i} \right) (\nabla \cdot \mathbf{u}_i) \mathbf{I} \quad (2.27)$$

with $\lambda_{s,i}$ being the bulk viscosity of the granular phase defined by (Huilin and Gidaspow, 2003)

$$\lambda_{s,i} = \sum_k P_{s,ik} \frac{d_{ik}}{3} \sqrt{\frac{2(m_i \Theta_i + m_k \Theta_k)^2}{\pi \Theta_i \Theta_k (m_i^2 \Theta_i + m_k^2 \Theta_k)}} \quad (2.28)$$

where $P_{s,ij}$ is the granular pressure between solid phases i and k , and is related to $P_{s,i}$ by

$$P_{s,i} = \alpha_i \rho_i \Theta_i + \sum_k P_{s,ik} \quad (2.29)$$

The production of granular energy is given by

$$\phi_{\text{prod},i\ell} = \frac{81 \alpha_i \mu_\ell^2}{g_{0,ii} d_i^3 \rho_i \sqrt{\pi}} \frac{|\mathbf{u}_\ell - \mathbf{u}_i|^2}{\sqrt{\Theta_i}} - 3 K_{D,i\ell} \Theta_i \quad (2.30)$$

where $K_{D,il}$ is the drag coefficient for the i - l solid-fluid pair and $g_{0,ij}$ is the radial distribution function.

The dissipation of granular energy between granular phases i and k is defined by (Huilin and Gidaspow, 2003)

$$\dot{\gamma}_{ik} = \left[\frac{3}{d_{ik}} \left(\frac{2m_0^2 \Theta_i \Theta_k}{\pi(m_i^2 \Theta_i + m_k^2 \Theta_k)} \right)^{1/2} - \frac{3m_0(m_i \Theta_i + m_k \Theta_k)}{4(m_i^2 \Theta_i + m_k^2 \Theta_k)} \nabla \cdot \mathbf{u}_i \right] (1 - e_{ik}) P_{s,ik} \quad (2.31)$$

where

$$m_i = \frac{\rho_i \pi d_i^3}{6} \quad (2.32)$$

$d_{ik} = (d_i + d_k)/2$, $m_0 = m_i + m_k$, e_{ik} is the coefficient of restitution between granular phases i and j , and $P_{s,ik}$ is the granular pressure between phases i and k . The drag, radial distribution function, and granular pressure are determined by models and will be discussed in Sec. 3.1 and Sec. 4. The specific manner as to how these source terms are solved can be found in Appendix B.7 of Houim and Oran (2016).

The energy transfer term, $\dot{e}_{ij}$, is defined as

$$\dot{e}_{ij} = \dot{e}_{\text{HT},ij} + \dot{\omega}_{i,j} I(e_i, e_j + h_{f_i} - h_{f_j}) \quad (2.33)$$

Lastly, the speed of sound for a granular material is given by

$$c_i^2 = \frac{1}{\rho_i} \left[\frac{\partial P_{\text{tot},i}}{\partial \alpha_i} + \frac{2}{3} \frac{\Theta_i \left(\frac{\partial P_{\text{tot},i}}{\partial \Theta_i} \right)^2}{\rho_i \alpha_i^2} \right] \quad (2.34)$$

where $P_{\text{tot},i} = P_{s,i} + P_{\text{fr},i}$. Because different models can be used for the granular pressure, frictional pressure, and additional models used within radial distribution functions, the specific form this takes is not discussed here.

The topic of granular flows, or kinetic theory, is quite extensive and thus it would be impractical to attempt to provide complete descriptions and derivations of the equations and models discussed here. For more details on this topic, the reader should consult the following resources:

- For derivations relating to molecular gasses, see Chapman et al. (1990)
- For more practical derivations, see Gidaspow (1994)
- For applications to compressible flows, see Fox (2019)
- For derivations of the polydisperse formulations, see Huilin and Gidaspow (2003) and Chao et al. (2011)

3 Interfacial models

The exchange of momentum and energy is determined by the use of interfacial models. These models are described below and include the equations used to calculate the transfer rates. In general, these models are solved using an explicit Euler integration method. Drag and heat transfer, however, are solved using an analytical solution; the reference used for these solution methods will be provided in the relevant sections.

NOTE: There are more models currently implemented that do not apply to gas-particle flows and, therefore, are not presented at this time. Their descriptions and equations will be added as they become relevant.

3.1 Drag models

The drag is responsible for relaxing the velocity difference between two phases to an equilibrium velocity at a rate determined by the drag coefficient, $K_{D,ij}$:

$$K_{D,ij} = 0.75 \frac{\alpha_d C_D \text{Re}_{ij} \rho_c \nu_c}{d_d^2} \quad (3.1)$$

where the Reynolds number between phases i and j is defined as $\text{Re}_{ij} = \frac{d_d |\mathbf{u}_c - \mathbf{u}_d|}{\nu_c}$. The subscripts c and d are used to denote the continuous and dispersed phases, respectively. The term $C_D \text{Re}_{ij}$ will be defined by the models discussed in the following sections.

The momentum transfer term is defined as $\dot{\mathbf{M}}_{D,ij} = K_{D,ij}(\mathbf{u}_j - \mathbf{u}_i)$. The solution to the drag relaxation can be found in Appendix B.7 of Houim and Oran (2016). Additionally, an ODE solver can be used to solve the drag relaxation, though this approach is much more computationally expensive.

Variable	Description
swarmCorrelation	Swarm model

3.1.1 SchillerNaumann

The drag model of Schiller and Naumann (1933) calculates the drag coefficient for flow over a single sphere:

$$C_D \text{Re}_{ij} = \begin{cases} 24(1 + 0.15 \text{Re}_{ij}^{0.687}) & \text{if } \text{Re}_{ij} < 1000 \\ 0.44 \text{Re}_{ij} & \text{else} \end{cases} \quad (3.2)$$

Variable	Description
residualRe	Minimum Reynolds number []

3.1.2 WenYu

The Wen and Yu drag model (Enwald et al., 1996) is used for dispersed particulate flow and the drag coefficient is defined as

$$C_D \text{Re}_{ij} = C_D \text{Re}_{SN,ij} \alpha_c^{-2.65} \quad (3.3)$$

where $C_D \text{Re}_{SN,ij}$ is the Schiller Naumann drag coefficient multiplied by the Reynolds number, and the Reynolds number used in the drag coefficient is weighted by the dispersed phase volume fraction, i.e.,

$$\text{Re}_{ij} = \text{Re}_{ij} \alpha_c \quad (3.4)$$

Variable	Description
residualRe	Minimum Reynolds number []

3.1.3 Ergun

The Ergun drag model (Enwald et al., 1996) is used for dispersed particulate flow and the drag coefficient is defined as

$$C_D \text{Re}_{ij} = \frac{4}{3} \frac{150(1 - \alpha_c)}{\alpha_c} + 1.75 \text{Re}_{ij} \quad (3.5)$$

3.1.4 Gibilaro

The Gibilaro drag model (Enwald et al., 1996) is used for dispersed particulate flow and the drag coefficient is defined as

$$C_D \text{Re}_{ij} = \frac{4}{3} \left(\frac{17.3}{\alpha_2} + 0.336 \text{Re}_{ij} \right) \alpha_c \alpha_2^{-2.8} \quad (3.6)$$

where $\alpha_2 = 1 - \alpha_d$.

3.1.5 SyamlalOBrien

The Syamlal O'Brien drag model (Syamlal et al., 1993) is used for dispersed particulate flow and the drag coefficient is defined as

$$C_D \text{Re}_{ij} = \frac{C_{Ds} \text{Re}_{ij} \alpha_c}{V_r^2} \quad (3.7)$$

where

$$C_{Ds} \text{Re}_{ij} = \left(0.63 \sqrt{\text{Re}_{ij}} + 4.8 \sqrt{V_r} \right)^2 \quad (3.8)$$

$$V_r = 0.5 \left(A - 0.06 \text{Re}_{ij} + \sqrt{(0.06 \text{Re}_{ij})^2 + 0.12 \text{Re}_{ij} (2B - A) + A^2} \right) \quad (3.9)$$

$$A = \alpha_2^{4.14} \quad (3.10)$$

$$B = \begin{cases} 0.8 \alpha_2^{1.28} & \text{if } \alpha_2 < 0.85 \\ \alpha_2^{2.65} & \text{else} \end{cases} \quad (3.11)$$

and $\alpha_2 = 1 - \alpha_d$.

3.1.6 GidaspowErgunWenYu

The drag model of Gidaspow (1994) is used for dispersed particulate flow and the drag coefficient is a combination of the Wen and Yu drag model (dilute) and Ergun drag model (dense):

$$C_D \text{Re}_{ij} = \begin{cases} C_D \text{Re}_{\text{WenYu},ij} & \text{if } \alpha_c > 0.80 \\ C_D \text{Re}_{\text{Ergun},ij} & \text{else} \end{cases} \quad (3.12)$$

where $C_D \text{Re}_{\text{WenYu},ij}$ is the Wen Yu drag coefficient and $C_D \text{Re}_{\text{Ergun},ij}$ is the Ergun drag coefficient, both multiplied by the Reynolds number.

Variable	Description
residualRe	Minimum Reynolds number []

3.1.7 GidaspowSchillerNaumann

The Gidaspow-Schiller Naumann drag model (Enwald et al., 1996) is used for dispersed particulate flow and the drag coefficient is defined as

$$C_D \text{Re}_{ij} = C_{Ds} \text{Re}_{ij} \alpha_c^{-1.65} \quad (3.13)$$

$$C_{Ds} \text{Re}_{ij} = \begin{cases} \frac{24(1+0.15 \text{Res}_{ij}^{0.687})}{\alpha_c} & \text{if } \text{Res}_{ij} < 1000 \\ 0.44 \text{Res}_{ij} & \text{else} \end{cases} \quad (3.14)$$

where the Reynolds number used in the drag coefficient is weighted by the dispersed phase volume fraction, i.e.,

$$\text{Res}_{ij} = \text{Re}_{ij} \alpha_c \quad (3.15)$$

Variable	Description
residualRe	Minimum Reynolds number []

3.2 Lift models

The lift interfacial momentum transfer model is used to account for acceleration due to rotation in the continuous phase and is defined as

$$\dot{\mathbf{M}}_{L,ij} = C_{L,ij} \alpha_d \rho_c [(\mathbf{u}_c - \mathbf{u}_i) \times (\nabla \times \mathbf{u}_c)] \quad (3.16)$$

where $C_{L,ij}$ is the lift coefficient between phases i and j and is defined by the model of choice.

3.2.1 constant

The lift coefficient is a constant value.

Variable	Description
Cl	Lift coefficient []

3.3 Heat transfer models

The heat transfer model is responsible for relaxing the temperature between two phases at a rate determined by the model of choice. The Prandtl number will be used and is defined as

$$\text{Pr}_{ij} = \frac{\nu_c C_{vc} \rho_c}{\kappa_c} \quad (3.17)$$

Again, the subscripts c and d represent the continuous and dispersed phases, respectively. C_{vc} is the continuous phase specific heat at constant volume and κ_c is the continuous phase thermal conductivity. The solution to the temperature relaxation can be found in Appendix B.7 of Houim and Oran (2016).

3.3.1 RanzMarshall

The Ranz Marshall heat transfer model (Houim and Oran, 2016) defines the heat transfer rate as

$$\dot{E}_{\text{HT},ij} = \frac{6\alpha_d \kappa_c \text{Nu}_{ij}}{d_d^2} \quad (3.18)$$

where the Nusselt number is a run-time selectable model, and all other properties are taken from the thermodynamic model.

Variable	Description
NuModel	Nusselt number model
residualAlpha	Stabilization value of volume fraction (default: phaseModel value)

3.3.2 noneNu

This should only be used if a Nusselt number is needed by some other model, but heat transfer is neglected.

3.4 Nusselt number models

3.4.1 constant

For the constant Nusselt number heat transfer model, Eq. (3.18) is again used, but the Nusselt number is a constant value provided by the user.

Variable	Description
Nu	Nusselt number []

3.4.2 duct

For hollow particles the Nusselt number of Dittus and Boelter (1930) is defined as

$$\text{Nu}_{ij} = 0.023 \text{Re}_{ij}^{0.8} \text{Pr}_{ij}^{0.4} \quad (3.19)$$

3.4.3 GelperinEinstein

The Nusselt number of Gelperin and Einstein (1971) is defined as

$$\text{Nu}_{ij} = 0.023 \text{Re}_{ij}^{2/3} \text{Pr}_{ij}^{1/3} \quad (3.20)$$

3.4.4 Gunn

The Nusselt number of Gunn (1978) is defined as

$$\begin{aligned} \text{Nu}_{ij} = & (7 - 10\alpha_c + 5.0\alpha_c^2)(1 + 0.7\text{Re}_{ij}^{0.2}\text{Pr}_{ij}^{1/3}) \\ & + (1.33 - 2.4\alpha_c + 1.2\alpha_c^2)\text{Re}_{ij}^{0.7}\text{Pr}_{ij}^{1/3} \end{aligned} \quad (3.21)$$

3.4.5 RanzMarschall

The Ranz-Marschall Nusselt number for spheres is defined as

$$\text{Nu}_{ij} = 2 + 0.6\text{Re}_{ij}^{1/2}\text{Pr}_{ij}^{1/3} \quad (3.22)$$

3.4.6 sphere

The Nusselt number for spheres is defined as

$$\text{Nu}_{ij} = 2 + 0.4\text{Re}_{ij}^{1/2}\text{Pr}_{ij}^{1/3} \quad (3.23)$$

3.5 Mass transfer models

Currently on particle to fluid mass transfer has been implemented. Fluid-fluid mass transfer is not currently supported.

3.5.1 none

No mass transfer.

3.5.2 reactingParticle

The mass transfer rate is explicitly calculated using the change in mass of a particle where the regression rate is determined by a reaction rate. the *reactingParticle* diameter model must be used.

3.6 Interfacial pressure models

The interfacial pressure model is used to determine the pressure at the interface between two phases.

3.6.1 single

The pressure of the given phase is used.

Variable	Description
phase	Name of the phase

NOTE: For gas-particle flows this option should be used, using the gas phase pressure.

3.6.2 volumeAveraged

The volume fraction averaged pressure of the two phases is used.

3.6.3 totalPressure

The volume fraction averaged thermodynamic pressure and the mass averaged dynamic pressure is used.

3.7 Interfacial velocity models

The interfacial velocity model is used to determine the velocity at the interface between two phases.

3.7.1 single

The velocity of the given phase is used.

Variable	Description
phase	Name of the phase

NOTE: For gas-particle flows this option should be used, using the particle phase velocity.

3.7.2 massAveraged

The mass averaged velocity of the two phases is used.

4 Granular models

For granular phases, additional models are required to account for particle-particle collisions in both the kinetic and frictional regimes. These models include radial distribution, viscosity, conductivity, granular pressure, frictional pressure, solid-solid drag, and packing models.

Some common variables used in the granular models will now be defined. Here, $\sum_k$ is the summation over all granular phase indices and α_s is the sum of all granular phase volume fractions. The strain is defined as

$$\mathbb{S} = \frac{1}{2} [(\nabla \mathbf{u})^\top + \nabla \mathbf{u}] - \frac{1}{3} (\nabla \cdot \mathbf{u}) \mathbb{I} \quad (4.1)$$

The symmetric strain is defined as

$$\mathbb{D} = \frac{1}{2} [(\nabla \mathbf{u})^\top + \nabla \mathbf{u}] \quad (4.2)$$

Finally, the total granular drag on a phase is defined as

$$K_{D,i} = \sum_k K_{D,ik} \quad (4.3)$$

4.1 Radial distribution models

The radial distribution gives the likelihood of a collision between two particles. This is defined between like and unlike particle phases.

4.1.1 CarnahanStarling

The Carnahan Starling radial distribution function is only valid for monodisperse flows and is defined as

$$g_{0,ii} = \frac{1}{1 - \alpha_i} + \frac{3\alpha_i}{2(1 - \alpha_i)^2} + \frac{\alpha_i^2}{2(1 - \alpha_i)^3} \quad (4.4)$$

4.1.2 SinclairJackson

The radial distribution of Sinclair and Jackson (1989) function is only valid for monodisperse flows and is defined as

$$g_{0,ii} = \frac{1}{1 - \left(\frac{\alpha_i}{\alpha_{i,\max}} \right)^{1/3}} \quad (4.5)$$

4.1.3 LunSavage

The radial distribution function of Lun et al. (1984) is only valid for monodisperse flows and is defined as

$$g_{0,ii} = \left(1 - \frac{\alpha_i}{\alpha_{\max,i}} \right)^{-2.5\alpha_{\max,i}} \quad (4.6)$$

4.1.4 Gao

The radial distribution function of Gao et al. (2008) is a modified version of the Sinclair Jackson radial distribution function and is valid for polydisperse flows. It is defined as

$$g_{0,ij} = \frac{d_i g_{0,i} + d_j g_{0,j}}{d_i + d_j} \quad (4.7)$$

$$g_{0,i} = \frac{1}{1 - \left(\frac{\alpha_i}{\alpha_{i,\max}} \right)^{1/3}} \quad (4.8)$$

4.1.5 Lebowitz

The Lebowitz radial distribution function (Benyahia et al., 2012) is valid for polydisperse flows and is defined as

$$g_{0,ij} = \frac{1}{1 - \alpha_s} + \frac{3d_i d_j}{(1 - \alpha_s)^2 (d_i + d_j)} \sum_k \frac{\alpha_k}{d_k} \quad (4.9)$$

4.2 Granular viscosity models

The modeling of granular viscosity is used to determine the stress seen in a granular phase. These models only account for single collisions (frictional models will have additional contributions for multiple collisions).

4.2.1 Gidaspow

The Gidaspow viscosity model (Gidaspow, 1994) is for monodisperse flows and is defined as

$$\begin{aligned}\nu_{s,i} = & \frac{4}{5}\alpha_i^2 g_{0,ii} \frac{(1 + e_{i,j})}{\pi} + \frac{1}{15}\sqrt{\pi} g_{0,ii} (1 + e_{ii}) \alpha_i^2 \\ & + \frac{1}{6}\sqrt{\pi} \alpha_i + \frac{10}{96} \frac{\sqrt{\pi}}{(1 + e_{ii}) g_{0,ii}}\end{aligned}\quad (4.10)$$

4.2.2 HrenyaSinclair

The Hrenya Sinclair viscosity model (Hrenya and Sinclair, 1997) is for monodisperse flows and is defined as

$$\begin{aligned}\nu_{s,i} = & \frac{4}{5}\alpha_i^2 g_{0,ii} \frac{(1 + e_{i,j})}{\pi} \\ & + \frac{1}{15}\sqrt{\pi} g_{0,ii} (3e_{ii} - 1) \frac{\alpha_i^2}{3 - e_{ii}} \\ & + \frac{1}{6} \frac{\sqrt{\pi} \alpha_i (0.5\lambda + 0.25(3e_{ii} - 1))}{0.5(3 - e_{ii})\lambda} \\ & + \frac{10}{96} \frac{\sqrt{\pi}}{(1 + e_{ii}) 0.5(3 - e_{ii}) g_{0,ii} \lambda}\end{aligned}\quad (4.11)$$

where

$$\lambda = 1 + \frac{d_i L}{6\sqrt{2}\alpha_i} \quad (4.12)$$

Variable	Description
L	characteristic length [m]

4.2.3 Princeton

The Princeton viscosity model (Agrawal et al., 2001) is for polydisperse flows and is defined as

$$\nu_{s,i} = \rho_i \frac{3.6}{3} \left[\frac{\mu_i^*}{g_{0,ii}\eta(2-\eta)} \left(1 + \frac{8}{5}\eta \sum_k (\alpha_k g_{0,ik}) \right) \times \left(1 + \frac{8}{5}\eta(3\eta-2) \sum_k (\alpha_k g_{0i,k}) \right) + \frac{3}{5}\eta\mu_b \right] \quad (4.13)$$

where

$$\mu_i^* = \frac{\rho_i \alpha_i g_{0,ii} \Theta_i \mu}{\rho_i \sum_k (\alpha_k g_{0,ik}) \Theta_i + \frac{2K_{d,i}\mu}{\rho_i \alpha_i}} \quad (4.14)$$

$$\mu = \frac{5}{96} \rho_i d_i \sqrt{\pi \Theta_i} \quad (4.15)$$

and

$$\mu_b = \frac{256}{5\pi} \mu \alpha_i \sum_k (\alpha_k g_{0,ik}) \quad (4.16)$$

4.2.4 Syamlal

The Syamlal viscosity model (Syamlal et al., 1993) is for monodisperse flows and is defined as

$$\nu_{s,i} = \frac{4}{5} \alpha_i^2 g_{0,ii} \frac{(1+e_{ii})}{\pi} + \frac{1}{15} \sqrt{\pi} g_{0,ii} (3e_{ii}-1) \frac{\alpha_i^2}{3-e_{ii}} + \frac{1}{6} \frac{\sqrt{\pi} \alpha_i}{(3-e_{ii})} \quad (4.17)$$

4.2.5 none

No granular viscosity is used. This also removes the conductivity contribution to the granular energy equation.

4.3 Granular conductivity models

4.3.1 Chao

The Chao granular conductivity model Chao et al. (2011) is defined as

$$\kappa_{s,i} = \frac{2\kappa_{\text{dilute},i}}{\frac{1}{N} \sum_k (1 + e_{ik}) g_{0,ik}} \left[1 + \frac{6}{5} \sum_k \alpha_k (1 + e_{ik}) g_{0,ik} \right]^2 \quad (4.18)$$

where N is the number of granular phases and

$$\kappa_{\text{dilute},i} = \frac{15\rho_i\Theta_i}{\sum_k [\pi\sqrt{2\pi}n_k d_{ij}^2 (\sqrt{\Theta_i} + \sqrt{\Theta_j} - 0.56\sqrt[4]{\Theta_i\Theta_j})]} \quad (4.19)$$

4.3.2 Gidaspow

The Gidaspow conductivity model (Gidaspow, 1994) is for monodisperse flows and is defined as

$$\begin{aligned} \kappa_{s,i} = & 2\alpha_i^2 g_{0,ii} \frac{(1 + e_{ii})}{\sqrt{\pi}} + \frac{9}{8} \sqrt{\pi} g_{0,ii} 0.5(1 + e_{ii}) \alpha_i^2 \\ & + \frac{15}{16} \sqrt{\pi} \alpha_i + \frac{25}{64} \frac{\sqrt{\pi}}{g_{0,ii}(1 + e_{ii})} \end{aligned} \quad (4.20)$$

4.3.3 HrenyaSinclair

The Hrenya Sinclair conductivity model (Hrenya and Sinclair, 1997) is for monodisperse flows and is defined as

$$\begin{aligned} \kappa_{s,i} = & 2\alpha_i^2 g_{0,ii} \frac{(1 + e_{ii})}{\sqrt{\pi}} \\ & + \frac{9}{8} \frac{\sqrt{\pi} g_{0,ii} 0.25(1 + e_{ii})^2 (2e_{ii} - 1) \alpha_i^2}{\frac{49}{16} - \frac{33}{16} e_{ii}} \\ & + \frac{15}{16} \frac{\sqrt{\pi} \alpha_i (0.5e_{ii}^2 + 0.25e_{ii} - 0.75 + \lambda)}{(\frac{49}{16} - \frac{33}{16} e_{ii}) \lambda} \\ & + \frac{25}{64} \frac{\sqrt{\pi}}{g_{0,ii} \lambda (1 + e_{ii}) (\frac{49}{16} - \frac{33}{16} e_{ii})} \end{aligned} \quad (4.21)$$

where

$$\lambda = 1 + \frac{d_i L}{6\sqrt{2}\alpha_i} \quad (4.22)$$

Variable	Description
L	characteristic length [m]

4.3.4 Princeton

The Princeton conductivity model (Agrawal et al., 2001) is for polydisperse flows and is defined as

$$\begin{aligned} \kappa_{s,i} = \frac{\kappa_i^*}{g_{0,ii}} & \left[\left(1 + \frac{12}{5}\eta \sum_k (\alpha_k g_{0,ik}) \right) \left(1 + \frac{12}{5}\eta^2 (4\eta - 3) \sum_k (\alpha_k g_{0,ik}) \right) \right. \\ & \left. + \frac{64}{25\pi} (41 - 33\eta)\eta^2 \left(\sum_k (\alpha_k g_{0,ik}) \right)^2 \right] \end{aligned} \quad (4.23)$$

where

$$\kappa_i^* = \frac{\rho_i \alpha_i g_{0,ii} \Theta_i \kappa_i}{\rho_i \sum_k (\alpha_k g_{0,ik}) \Theta_i + \frac{6K_{d,i} \kappa_i}{5\rho_i \alpha_i}} \quad (4.24)$$

and

$$\kappa_i = \frac{75\rho_i d_i \sqrt{\pi \Theta_i}}{48\eta(41 - 33\eta)} \quad (4.25)$$

4.3.5 Syamlal

The Syamlal conductivity model (Syamlal et al., 1993) is for monodisperse flows and is defined as

$$\begin{aligned} \kappa_{s,i} = 2\alpha_i^2 g_{0,ii} & \frac{(1 + e_{ii})}{\sqrt{\pi}} \\ & + \frac{9}{8} \frac{\sqrt{\pi} g_{0,ii} 0.25(1 + e_{ii})(2e_{ii} - 1)\alpha_i^2}{\frac{49}{16} - 3316e_{ii}} \\ & + \frac{15}{16} \frac{\sqrt{\pi} \alpha_i}{\frac{49}{16} - \frac{33}{16}e_{ii}} \end{aligned} \quad (4.26)$$

4.4 Granular pressure models

The granular pressure model is used to calculate the pressure due to single collisions between particles. Pressures that occur due to multiple collisions will be added using the frictional models.

4.4.1 Huilin

The Huilin granular pressure model (Huilin and Gidaspow, 2003) is for poly-disperse flows and is defined as

$$\begin{aligned}
P_{s,ij} = & \frac{\pi(1 + e_{ij})d_{ij}^3 g_{0,ij} n_i n_j m_i m_j m_0 \Theta_i \Theta_j}{3(m_i^2 \Theta_i + m_j^2 \Theta_j)} \\
& \times \left[\frac{m_0^2 \Theta_i \Theta_j}{(m_i^2 \Theta_i + m_j^2 \Theta_j)(\Theta_i + \Theta_j)} \right]^{3/2} \\
& \times (1 - 3\omega + 6\omega^2 - 10\omega^3)
\end{aligned} \tag{4.27}$$

with

$$\omega = \frac{m_i \Theta_i - m_j \Theta_j}{[(m_i^2 \Theta_i^2 + m_j^2 \Theta_j^2) + \Theta_i \Theta_j (m_i^2 + m_j^2)]^{1/2}} \tag{4.28}$$

4.4.2 Lun

The Lun granular pressure model (Lun et al., 1984) is for monodisperse flows and is defined as

$$P_{s,ij} = \alpha_i \alpha_j \rho_i \Theta_i 2(1 + e_{ij}) g_{0,ij} \tag{4.29}$$

4.5 Frictional stress models

The frictional stress model acts as a simple way to account for non-singular collisions between particles and results in very large stress when the granular phases approach their packing limit. This results in both a modeled pressure and viscosity to calculate a stress, and limits the ability of particles to move between computational cells. The frictional pressure and viscosity denoted P_{fr} and ν_{fr} act on all granular phases since the packing of particles is dependent on the total volume fraction of particles.

4.5.1 JohnsonJackson

The Johnson-Jackson frictional stress model (OpenCFD Ltd., 2018b) uses a smooth function to calculate the pressure

$$P_{\text{fr}} = \begin{cases} Fr \frac{(\alpha_s - \alpha_{\text{crit},s})^\eta}{(\alpha_{\text{max},s} - \alpha_s)^p} & \text{if } \alpha_s > \alpha_{\text{crit},s} \\ 0 & \text{else} \end{cases} \quad (4.30)$$

and the frictional viscosity is

$$\nu_{\text{fr}} = 0.5 P_{\text{fr}} \sin(\phi) \quad (4.31)$$

where ϕ is the angle of internal friction, and Fr , η , and p are model coefficients generally taking values of 0.05, 2, and 5 respectively. $\alpha_{\text{crit},s}$ is the granular particle fraction that the frictional pressure "turns on", and $\alpha_{\text{max},s}$ is the maximum packing limit of all granular phases. Generally $\alpha_{\text{crit},s}$ is taken to be around 0.5.

Variable	Description
Fr	Model coefficient []
eta	Model coefficient []
p	Model coefficient []
phi	Model coefficient []
alphaDeltaMin	Minimum value of $\alpha_s - \alpha_{\text{max},s}$ []
alphaMinFriction	Volume fraction friction is "turned on" []

4.5.2 Schaeffer

The Scheffer frictional stress model (Syamlal et al., 1993) uses a power law and the local stress is used to calculate the frictional viscosity. The critical volume fraction, $\alpha_{\text{crit},s}$ should be a value close to that of the packing limit. Because the local packing limit can vary in space when polydispersity is present, the ratio of the local volume fraction by the critical volume fraction is used, i.e. $\alpha_{\text{crit},s} = \left(\frac{\alpha_{\text{crit},s}}{\alpha_{\text{max},s}} \right)_{\text{crit}} \alpha_{\text{max},s}$ where $\left(\frac{\alpha_{\text{crit},s}}{\alpha_{\text{max},s}} \right)_{\text{crit}}$ is specified by the user and $\alpha_{\text{max},s}$ is the calculated local packing limit. The pressure is defined as

$$P_{\text{fr}} = \begin{cases} 10^{24} (\alpha_s - \alpha_{\text{crit},s})^{10} & \text{if } \alpha_s > \alpha_{\text{crit},s} \\ 0 & \text{else} \end{cases} \quad (4.32)$$

and the viscosity is

$$\nu_{\text{fr}} = \begin{cases} 0.5 \frac{P_{\text{fr}} \sin(\phi)}{\sqrt{\frac{1}{3}(I_1(\mathbb{D})^2 - I_2(\mathbb{D}))}} & \text{if } \alpha_s > \alpha_{\text{crit},s} \\ 0 & \text{else} \end{cases} \quad (4.33)$$

where $I_1(\mathbb{D})$ is the first invariant (or trace) and $I_2(\mathbb{D})$ is the second invariant.

Variable	Description
phi	Model coefficient []
alphaMinFrictionByAlphap	Volume fraction ratio friction is “turned on” []

4.5.3 JohnsonJacksonSchaeffer

For the Johnson Jackson-Scheffer frictional stress model (OpenCFD Ltd., 2018b), the frictional pressure of Johnson Jackson is used while the viscosity of the Schaeffer model is used.

$$P_{\text{fr}} = \begin{cases} Fr \frac{(\alpha_s - \alpha_{\text{crit},s})^\eta}{(\alpha_{\text{max},s} - \alpha_s)^p} & \text{if } \alpha_s > \alpha_{\text{crit},s} \\ 0 & \text{else} \end{cases} \quad (4.34)$$

$$\nu_{\text{fr}} = \begin{cases} 0.5 \frac{P_{\text{fr}} \sin(\phi)}{\sqrt{\frac{1}{3}(I_1(\mathbb{D})^2 - I_2(\mathbb{D}))}} & \text{if } \alpha_s > \alpha_{\text{crit},s} \\ 0 & \text{else} \end{cases} \quad (4.35)$$

Variable	Description
Fr	Model coefficient []
eta	Model coefficient []
p	Model coefficient []
phi	Model coefficient []
alphaDeltaMin	Minimum value of $\alpha_s - \alpha_{\text{max},s}$ []
alphaMinFriction	Volume fraction friction is “turned on” []

4.5.4 Princeton

The Princeton model Agrawal et al. (2001) uses a blending of the Johnson Jackson model when the local volume fraction is greater than $\alpha_{\text{crit},\text{JJ},s}$ but less

than $\alpha_{\text{crit,Sc},s}$. As with the Schaeffer model, the critical ratio of $\left(\frac{\alpha_{\text{crit,Sc},s}}{\alpha_{\text{max},s}}\right)_{\text{crit}}$ is used as an input rather than $\alpha_{\text{crit,Sc},s}$.

$$P_{\text{fr}} = \begin{cases} 10^{24}(\alpha_s - \alpha_{\text{crit,Sc},s})^{10} & \text{if } \alpha_s > \alpha_{\text{crit,Sc},s} \\ Fr \frac{(\alpha_s - \alpha_{\text{crit,JJ},s})^\eta}{(\alpha_{\text{max},s} - \alpha_s)^p} & \text{if } \alpha_{\text{crit,Sc},s} \leq \alpha_s > \alpha_{\text{crit,JJ},s} \\ 0 & \text{else} \end{cases} \quad (4.36)$$

The frictional viscosity is defined using

$$\frac{P_{c,k}}{P_{\text{fr}}} = \left(1 - \frac{\nabla \cdot \mathbf{u}_s}{n\sqrt{2}\sin(\phi)\sqrt{\mathbb{S} : \mathbb{S} + \Theta_k/d_k^2}}\right)^{n-1} \quad (4.37)$$

$$\nu_{\text{fr},k} = \frac{\sqrt{2}P_{c,k}\sin(\phi)}{\sqrt{\mathbb{S} : \mathbb{S} + \Theta_k/d_k^2}} \left[n - (n-1) \left(\frac{P_{c,k}}{P_{\text{fr}}} \right)^{\frac{1}{1-n}} \right] \frac{\alpha_k}{\alpha_s} \quad (4.38)$$

and

$$n = \begin{cases} \frac{\sqrt{3}}{2\sin(\phi)} & \text{if } \nabla \cdot \mathbf{u}_s \geq 0 \\ 1.03 & \text{else} \end{cases} \quad (4.39)$$

Variable	Description
Fr	Model coefficient []
eta	Model coefficient []
p	Model coefficient []
phi	Model coefficient []
alphaDeltaMin	Minimum value of $\alpha_s - \alpha_{\text{max},s}$ []
alphaMinFriction	Volume fraction friction is “turned on” []
alphaMinFrictionByAlphap	Volume fraction ratio friction is “turned on” []

4.6 Packing limit models

The modeling of the packing limit is important in polydisperse flows due to the fact that, when large variations in particle diameters exist locally, the packing limit can vary widely. This is due to the fact that smaller particles can be found in between larger particles, increasing the maximum allowable volume fraction. For monodisperse flows, this is not the case and a constant value packing limit should be used.

4.6.1 constant

A constant value for the packing limit is used.

Variable	Description
alphaMax	Packing limit []

4.6.2 YuStandish

The Yu Standish packing model (Yu and Standish, 1987) assumes that the particle phases are ordered from smallest diameter to largest for the calculation of the packing limit. The packing limit is defined as

$$\alpha_{\max,s} = \min_{i \in 1:N} \left[\frac{\alpha_{\max,i}}{1 - \sum_{k=1}^{i-1} \left(1 - \frac{\alpha_{\max,i}}{p_{ik}}\right) \frac{cx_i}{X_{ik}} - \sum_{k=i+1}^N \left(1 - \frac{\alpha_{\max,i}}{p_{ik}}\right) \frac{cx_i}{X_{ik}}} \right] \quad (4.40)$$

where

$$cx_i = \frac{\alpha_i}{\alpha_s} \quad (4.41)$$

$$X_{ik} = \begin{cases} \frac{1-r_{ik}^2}{2-\alpha_{\max,i}} & \text{if } k < i \\ 1 - \frac{1-r_{ik}^2}{2-\alpha_{\max,i}} & \text{if } k \geq i \end{cases} \quad (4.42)$$

$$p_{ik} = \begin{cases} \alpha_{\max,i} + \alpha_{\max,i}(1 - \alpha_{\max,i}) \\ \quad \times (1 - 2.35r_{ik} + 1.35r_{ik}^2) & \text{if } r_{ik} \leq 0.741 \\ \alpha_{\max,i} & \text{if } r_{ik} > 0.741 \end{cases} \quad (4.43)$$

and

$$r_{ik} = \begin{cases} \frac{d_i}{d_k} & \text{if } i \geq k \\ \frac{d_k}{d_i} & \text{if } i < k \end{cases} \quad (4.44)$$

Variable	Description
residualAlpha	Minimum volume fraction to divide by []

4.6.3 FedorsLandel

The Fedors-Landel packing model (Fedors and Landel, 1979) is only valid of for a binary mixture and is defined as

$$\alpha_{\max,s} = \begin{cases} \left[\begin{aligned} &(\alpha_{\max,1} - \alpha_{\max,2}) \\ &+ (1 - \sqrt{r_{21}}(1 - \alpha_{\max,1})\alpha_{\max,2}) \\ &\times [\alpha_{\max,1} + (1 - \alpha_{\max,1})\alpha_{\max,2}] \frac{cx_i}{\alpha_{\max,1}} \end{aligned} \right. \\ \left. \begin{aligned} &+ \alpha_{\max,2} \\ &(1 - \sqrt{r_{21}})[\alpha_{\max,1} \\ &+ (1 - \alpha_{\max,1})\alpha_{\max,2}]cx_2 + \alpha_{\max,1} \end{aligned} \right. \end{cases} \begin{matrix} \text{if } cx_1 \leq \frac{\alpha_{\max,1}}{\alpha_{\max,1} + (1 - \alpha_{\max,1})\alpha_{\max,2}} \\ \text{else} \end{matrix} \quad (4.45)$$

Where the definitions of r_{21} and cx_i are used from the previous section.

Variable	Description
residualAlpha	Minimum volume fraction to divide by []

4.7 Granular drag models

The drag force between granular phases is different than standard drags models due to the fact the the collisions which are a function of the granular energy strongly effect the rate at which the velocities relax. These granular drag models are only necessary when multiple granular phases are used.

4.7.1 Chao

The Chao granular drag model (Chao et al., 2011) defines the drag coefficient between phases as

$$K_{D,ij} = \frac{\alpha_i \alpha_j \rho_i \rho_j}{m_0} d_{ij} (1 + e_{ij}) g_{0,ij} \times \left[\sqrt{2\pi} (\sqrt{\Theta_i} + \sqrt{\Theta_j}) - \sqrt{2} (\Theta_i \Theta_j)^{1/4} + 0.5\pi |\mathbf{u}_i - \mathbf{u}_j| - 1.135 \sqrt{|\mathbf{u}_i - \mathbf{u}_j|} \left(\sqrt{4\Theta_i} + \sqrt[4]{\Theta_j} - 0.8 \sqrt[8]{\Theta_i \Theta_j} \right) \right] \quad (4.46)$$

4.7.2 Syamlal

The Syamlal granular drag model (Benyahia et al., 2012) defines the drag coefficient between phases as

$$K_{D,ij} = \frac{3(1 + e_{ij})(\frac{\pi}{2} + \frac{C_{f,ij}\pi^2}{8})\alpha_i\alpha_j\rho_i\rho_j(d_i + d_j)^2 g_{0,ij}|\mathbf{u}_i - \mathbf{u}_j|}{2\pi(\rho_i d_i^2 + \rho_j d_j^3)} + C_1 P_{fr} \quad (4.47)$$

where $C_{f,ij}$ is the coefficient of friction between granular phases i and j, and C_1 is the coefficient of frictional pressure.

Variable	Description
C1	Coefficient of frictional pressure (default: 0) [s/m ²]

5 Fluid and solid thermodynamic models

Within blastFoam solvers, two different types of basic thermodynamic models are used. The primary is that of a fluid where internal energy and density, or temperature and density, are used to find the resulting pressure and speed of sound. The other, is the solid thermodynamic model, which is only used for solid phases within multi-fluid solvers and multi-region solvers. These two models will share the basic thermodynamic classes (eConst, hConst, etc), as well as the choice in the type (**basic**, **detonating**, or **multicomponent**), however the solid phase only can use a constant density equation of state, and the transport model describes the of heat conduction, rather than flow coefficients (like viscosity). These specific choices will be described below.

To calculate the pressure and temperature given the conservative quantities (mass, momentum, and energy) three components are required: an equation of state, a thermodynamic model, and a transport model. These models are defined in the *phaseProperties* file located in the *constant* directory of each blastFoam case. Each phase within the *phaseProperties* file has several dictionary entries, and these models are specified in the "thermoType" sub-dictionary; the molecular weight must also be defined in the "specie" sub-dictionary.

```
phase_name
{
    type          basic;
    thermoType
    {
        transport  const;
        thermo     eConst;
        equationOfState idealGas;
    }
    specie
    {
        molWeight  28.97;
    }
    transport
    {
        ...
    }
}
```

```

thermodynamics
{
    ...
}
equationOfState
{
    gamma      1.4;
    ...
}
}

```

NOTE: Not all combinations of thermodynamic model, equation of state, transport, and mixture type are compiled, however using the dynamic compilation available in OpenFOAM, if a valid option is requested it will be compiled at run time and the library will be stored in the *dynamicCode* folder in the case. If any changes are made to the source code (updating/pulling) will not affect these libraries, and the *dynamicCode* folder will need to be removed for re-compilation.

NOTE: By default, any thermodynamic combination that has not previously been compiled will be compiled using the files in *blastfoam/etc/code-Templates*. These files include support for the standard model, however using the "templateDir" entry a different folder can be used, so that user modified files can be used. This allows for use of user implemented equations of state, thermodynamic, or transport models.

5.1 Thermodynamic models

The thermodynamic model is used to calculate the internal energy based solely on the thermal contribution:

$$e_T = \int C_v(T) dT \quad (5.1)$$

An additional contribution from the equation of state is given by:

$$e_{\text{eos}}(\rho, T) = \int \left(T \frac{\partial p(\rho, T)}{\partial T} - p(\rho, T) \right) dv \quad (5.2)$$

where $v_i = 1/\rho_i$ is the specific volume. The total internal energy is

$$e = e_T(T) + e_{\text{eos}}(\rho, T) \quad (5.3)$$

Because the internal energy is the tracked quantity, the temperature is solved using a Newton-Raphson root finding method given the local internal energy and density.

Either internal energy or enthalpy-based coefficients can be used, but only internal energy is used to define the total energy. The conversion from an enthalpy base to an internal energy base is calculated using the specified equation of state. All transport models, with the exception of the tabulated methods were ported from OpenCFD Ltd. (2018b).

$$C_v = C_p - CpMCv \quad (5.4)$$

where $CpMCv$ is determined from the equation of state using

$$CpMCv = \frac{\partial(p(\rho, e)v)}{\partial T} \quad (5.5)$$

For an ideal gas, $CpMCv = \bar{R}/W$ where $\bar{R}$ is the ideal gas constant (8314 [J/mol/K]) and W is the molecular weight in [kg/kmol]. Equations of state that have a pressure with a non-linear dependence on temperature will deviate from this.

When enthalpy is used, the internal energy is calculated using

$$e = h - \frac{p}{\rho} \quad (5.6)$$

NOTE: By default the internal energy is limited so that the temperature is greater than 0. If "limitT" is set to "no" the temperature and energy will not be limited. Caution should be used with this option.

5.1.1 eConst

Internal energy is calculated using a constant value of specific heat at constant volume.

$$e_T(T) = C_v(T - T_{ref}) + E_{s,ref} \quad (5.7)$$

Variable	Description
Cv	Specific at constant volume [J/kg/K]
Tref	Reference temperature (default: 293.15) [K]
Esref	Reference sensible internal energy [J/kg/] (default: $T_{ref} C_v$)
Hf	Heat of formation [J/kg]

5.1.2 ePolynomial

Internal energy is calculated using polynomial with powers of T.

$$e_T(T) = \sum_{i=0}^7 C_{v,i} T^i \quad (5.8)$$

Variable	Description
CvCoeffs<8>	Specific heat at constant volume polynomial coefficients
Hf	Heat of formation [J/kg]
Sf	Standard entropy [J/kg/K]

5.1.3 ePower

Internal energy using a power function of temperature.

$$e_T(T) = C_v(T)T \quad (5.9)$$

$$C_v(T) = C_0 \left(\frac{T}{T_{ref}} \right)^{n_0} \quad (5.10)$$

Variable	Description
C0	Reference heat capacity at constant volume [J/kg/K]
n0	Exponent of the power function []
Tref	Reference temperature [K]
Hf	Heat of formation [J/kg]

5.1.4 eIcoTabulated

The incompressible tabulated thermodynamic model can be used to calculate the temperature using temperature to lookup the internal energy. The data is read from a one dimensional table. The specific heat are calculated using finite difference derivatives to the tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x" and "f" names are "T" and "e" respectively. A heat of formation, "Hf", must also be provided.

5.1.5 eTabulated

The tabulated thermodynamic model can be used to calculate the temperature using density and temperature to find internal energy. The data is read from a two dimensional table. The specific heat are calculated using finite difference derivatives to the tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x", "y", and "f" names are "rho", "T", and "e" respectively. A heat of formation, "Hf", must also be provided.

NOTE: The equation of state contribution of internal energy is not included so this must be included in the tables provided.

5.1.6 hConst

Enthalpy is calculated using a constant value of specific heat at constant pressure.

$$h_T(T) = C_p(T - T_{ref}) + H_{s,ref} \quad (5.11)$$

Variable	Description
Cp	Specific at constant pressure [J/kg/K]
Tref	Reference temperature (default: 293.15) [K]
Hsref	Reference sensible enthalpy (default: $T_{ref} C_p$) [J/kg]
Hf	Heat of formation [J/kg]

5.1.7 hPolynomial

Enthalpy is calculated using polynomial with powers of T.

$$h_T(T) = \sum_{i=0}^7 C_{p,i} T^i \quad (5.12)$$

Variable	Description
CpCoeffs<8>	Specific heat at constant pressure polynomial coefficients
Hf	Heat of formation [J/kg]
Sf	Standard entropy [J/kg/K]

5.1.8 hPower

Internal energy using a power function of temperature.

$$h_T(T) = C_p(T)T \quad (5.13)$$

$$C_v(T) = C_0 \left(\frac{T}{T_{ref}} \right)^{n_0} \quad (5.14)$$

Variable	Description
C0	Reference heat capacity at constant volume [J/kg/K]
n0	Exponent of the power function
Tref	Reference temperature [K]
Hf	Heat of formation [J/kg]

5.1.9 hIcoTabulated

The incompressible tabulated thermodynamic model can be used to calculate the temperature using temperature to lookup the enthalpy. The data is read from a one dimensional table. The specific heat are calculated using finite difference derivatives to the tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x" and "f" names are "T" and "h" respectively. A heat of formation, "Hf", must also be provided.

5.1.10 hTabulated

The tabulated thermodynamic model can be used to calculate the temperature using density and temperature to find enthalpy. The data is read from a two dimensional table. The specific heat are calculated using finite difference derivatives to the tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x", "y", and "f" names are "rho", "T", and "h" respectively. A heat of formation, "Hf", must also be provided.

NOTE: The equation of state contribution of internal energy is not included so this must be included in the tables provided.

5.1.11 JANAF

The JANAF thermodynamic model can be used in which the enthalpy is represented by a temperature based polynomial (OpenCFD Ltd., 2018b). Seven coefficients must be specified for both the low temperature and high temperature states, as well as the high, low, and switching temperatures. Currently, this model is only available for ideal gases.

Variable	Description
highCpCoeffs	High enthalpy coefficients
lowCpCoeffs	Low enthalpy coefficients
Tlow	Minimum temperature [K]
Tcommon	Switching temperature [K]
Thigh	Maximum temperature [K]

5.2 Equation of states

In order to more easily handle a variety of equations of state, both temperature-based equations of state and internal energy based equations of state (e.g. in Mie-Grüneisen form) can be used.

5.2.1 rhoConst

For a solid phase, it is assumed to have a constant density, ρ_0 , and maintains a reference pressure p_{ref} . Because a solid thermodynamic model is never used

to transport mass, the reference pressure is arbitrary and therefore set to 0 [Pa] by default.

Variable	Description
ρ_0	reference density [kg/m ³]
p_{ref}	Reference pressure (default: 101235) [Pa]

5.2.2 Internal energy based (Mie-Grüneisen form)

The Mie-Grüneisen form uses two contributions to calculate the pressure, and ideal gas term contribution, and a deviation term.

$$p = (\Gamma_i - 1)\rho_i e - \Pi_i \quad (5.15)$$

Γ_i , also called the Grüneisen coefficient, and Π_i is the pressure deviation from an ideal gas and is specific to the equation of state. Using these quantities, the speed of sound is written as

$$c_i^2 = \frac{\Gamma_i p + \Pi_i}{\rho_i} - \delta_i(\Gamma_i - 1) \quad (5.16)$$

where δ_i is defined

$$\delta_i = -\frac{[p + \Pi_i]\Gamma'_i}{[\Gamma_i - 1]^2} + \frac{\Pi'_i}{\Gamma_i - 1} \quad (5.17)$$

and where Γ'_i and Π'_i are the derivatives with respect to the phase density.

5.2.2.1 idealGas

For an ideal gas, the pressure is defined as

$$p_i = (\gamma_i - 1)\rho_i e_i \quad (5.18)$$

The functions as defined to represent an ideal gas in Mie-Grüneisen form are:

$$\Gamma_i = \gamma_i \quad (5.19)$$

$$\Pi_i = 0 \quad (5.20)$$

$$\delta_i = 0 \quad (5.21)$$

Variable	Description
gamma	Specific heat ratio []

5.2.2.2 stiffenedGas

For a material obeying the stiffened EOS, the pressure is defined as

$$p_i = (\gamma_i - 1)\rho_i e_i - \gamma_i a_i \quad (5.22)$$

where a_i and γ_i are material properties (Zheng et al., 2011). In the Mie-Grüneisen form

$$\Gamma_i = \gamma_i \quad (5.23)$$

$$\Pi_i = \gamma_i a_i \quad (5.24)$$

and

$$\delta_i = 0 \quad (5.25)$$

Variable	Description
a	Reference pressure [Pa]
gamma	Specific heat ratio []

5.2.2.3 linearTillotson

The linear Tillotson equation of state was presented by (Wardlaw et al., 1998), and the pressure is defined as

$$p_i = p_{0,i} + \omega_i \rho_i (e_T - e_{0,i}) + A_i \mu + B_i \mu^2 + C_i \mu^3 \quad (5.26)$$

where A_i , B_i , C_i , and ω_i are material coefficients, and $p_{0,i}$ and $e_{0,i}$ are the reference pressure and internal energy. μ is defined as $\rho_i/\rho_{0,i} - 1$. The cavitation pressure, p_{cav} is used as the minimum pressure which occurs when the density is sufficiently below the reference density.

In the Mie-Grüneisen form

$$\Gamma_i = \omega_i + 1 \quad (5.27)$$

and

$$\Pi_i = \omega_i \rho_i e_{0,i} - p_{0,i} - A_i \mu - B_i \mu^2 - C_i \mu^3 \quad (5.28)$$

and

$$\delta_i = e_{0,i} - \frac{A + 2B\mu + 3.0C\mu^2}{\rho_{0,i}\omega_i} \quad (5.29)$$

NOTE: Because the linear Tillotson EOS calculates deviation pressure from p_0 , in the thermodynamic dictionary, Tref should be set to the ambient temperature and Esref should be set to 0.

Variable	Description
p0	Reference pressure [Pa]
rho0	Reference density [kg/m ³]
e0	Reference internal energy [kJ/kg]
omega	Model coefficient []
A	Model coefficient [Pa]
B	Model coefficient [Pa]
C	Model coefficient [Pa]
pCav	Cavitation pressure [Pa]

5.2.2.4 Tillotson

The Tillotson equation of state was originally developed for metals by Tillotson (1962), and the pressure is defined as

$$p_i = \max \left(\rho_i e \left(a + \frac{b}{\frac{e}{e_{0,i}\eta^2} + 1} \right) + A_i \mu + B_i \mu^2, p_{cav} \right) \quad (5.30)$$

where A_i , B_i , C_i , and ω_i are material coefficients, and $p_{0,i}$ and $e_{0,i}$ are the reference pressure and internal energy (Wardlaw et al., 1998). μ is defined as $\rho_i/\rho_{0,i} - 1$. The cavitation pressure, p_{cav} is used as the minimum pressure which occurs when the density is sufficiently below the reference density.

In the Mie-Grüneisen form

$$\Gamma_i = \frac{a_i x + b_i e_{0,i} \rho_i^2}{x} - \frac{b e e_{0,i} (\rho_i \rho_{0,i})^2}{x^2} + 1 \quad (5.31)$$

where

$$x_i = e \rho_{0,i}^2 + e_{0,i} \rho_i^2 \quad (5.32)$$

and

$$\Pi_i = -A_i \mu - B_i \mu^2 - b_i \frac{(e \rho_i \rho_{0,i})^2 e_{0,i} \rho_i}{(e \rho_{0,i}^2 + e_{0,i} \rho_i^2)^2} \quad (5.33)$$

and δ_i is calculated using (5.17)

NOTE: Because the Tillotson EOS calculates deviation pressure from p_0 , in the thermodynamic dictionary, Tref should be set to the ambient temperature and Esref should be set to 0.

Variable	Description
rho0	Reference density [kg/m ³]
e0	Reference internal energy [kJ/kg]
a	Model coefficient []
b	Model coefficient []
A	Model coefficient [Pa]
B	Model coefficient [Pa]
pCav	Cavitation pressure [Pa]

5.2.2.5 Tait

For a material obeying the stiffened EOS, the pressure is defined as

$$p_i = (\gamma_i - 1) \rho_i e_i - \gamma_i (b_i - a_i) \quad (5.34)$$

where a_i , b_i , and γ_i are material properties (Zheng et al., 2011). In the Mie-Grüneisen form

$$\Gamma_i = \gamma_i \quad (5.35)$$

$$\Pi_i = \gamma_i (b_i - a_i) \quad (5.36)$$

and

$$\delta_i = 0 \quad (5.37)$$

Variable	Description
a	Material parameter [Pa]
b	Material parameter [Pa]
gamma	Specific heat ratio []

5.2.2.6 vanderWaals

For a gas described by the generalized van der Waals equation of state (Zheng et al., 2011), the pressure is defined as

$$p_i = \frac{\gamma_i - 1}{1 - b_i \rho_i} (\rho_i e_i + a_i \rho_i^2) - (a_i \rho_i^2 + c_i) \quad (5.38)$$

Putting the pressure in the Mie-Grüneisen form, we obtain

$$\Gamma_i = \frac{\gamma_i - 1}{1 - b_i \rho_i} + 1 \quad (5.39)$$

$$\Pi_i = \left[1 - \frac{\gamma_i - 1}{1 - b_i \rho_i} \right] a_i \rho_i^2 + \left[\frac{\gamma_i - 1}{1 - b_i \rho_i} + 1 \right] c_i \quad (5.40)$$

and

$$\delta_i = -b_i \frac{p_i + a_i \rho_i^2}{\gamma_i - 1} + \left(\frac{1 - b_i \rho_i}{\gamma - 1} - 1 \right) 2a_i \rho_i \quad (5.41)$$

Variable	Description
a	Attraction between particles [$\text{kg}^{-1} \text{m}^5 \text{s}^{-2}$]]
b	Specific volume excluded due to particle volume [$\text{kg}^{-1} \text{m}^3$]
c	Reference pressure [Pa]
gamma	Specific heat ratio []

5.2.2.7 LSZK

The Landau, Stanyukovich, Zeldovich, and Kompaneets equation of state is a simple model for describing an ideal explosive (Lutzky, 1964; Needham, 2018). It takes a form similar to a stiffened gas or Tait EOS, with

$$\Gamma_i = \gamma_i \quad (5.42)$$

$$\Pi_i = a_i \rho_i^{b_i} \quad (5.43)$$

and

$$\delta_i = 0 \quad (5.44)$$

Variable	Description
a	Model constant [$\text{kg}^{1-b} \text{s}^{-2} \text{m}^{3b-1}$]
b	Pressure exponent []
gamma	Specific heat ratio []

5.2.2.8 JWL

The more complicated Jones Wilkins Lee EOS is often used to define energetic materials (Zheng et al., 2011), and has a pressure deviation given by

$$\Pi_i = A_i e^{-R_{1,i}V} \left(\frac{\omega_i}{R_{1,i}V} - 1 \right) + B_i e^{-R_{2,i}V} \left(\frac{\omega_i}{R_{2,i}V} - 1 \right) - \rho_i \omega_i e_{0,i} \quad (5.45)$$

where $V = \rho_{0,i}/\rho_i$.

$$\Gamma_{r,i} = \omega_i + 1 \quad (5.46)$$

and

$$\begin{aligned} \delta_{r,i} = A_i e^{-\frac{R_{1,i}\rho_{0,i}}{\rho_i}} & \left[\omega_i \left(\frac{1}{R_{1,i}\rho_{0,i}} + \frac{1}{\rho_i} \right) - \frac{R_{1,i}\rho_{0,i}}{\rho_i^2} \right] \frac{1}{\omega_i} \\ & + B_i e^{-\frac{R_{2,i}\rho_{0,i}}{\rho_i}} \left[\omega_i \left(\frac{1}{R_{2,i}\rho_{0,i}} + \frac{1}{\rho_i} \right) - \frac{R_{2,i}\rho_{0,i}}{\rho_i^2} \right] \frac{1}{\omega_i} - e_{0,i} \end{aligned} \quad (5.47)$$

NOTE: The JWL model can also be used to describe the unreacted state if a detonating model is used with the `solidJWL` type. For the solid case, $e_{0,i}$ is calculated so that there is zero pressure at the reference density, and uses an offset pressure, "pRef". Thermal energy is also not considered for the solid case

Variable	Description
A	Model coefficient [Pa]
B	Model coefficient [Pa]
R1	Model coefficient []
R2	Model coefficient []
rho0	Unreacted or reference density [kg / ³]
omega	Grüneisen coefficient []
e0	Offset energy (gas: (default: 0), solid: 0 pressure at reference density) [J/kg]
pRef	Offset pressure (only for solid) [Pa]

5.2.2.9 CochranChan

The Cochran Chan EOS can be used to describe a solid material (Zheng et al., 2011), and has a reference pressure given by

$$p_{\text{ref},i} = A_i V_i^{1-\varepsilon_{1,i}} - B_i V_i^{1-\varepsilon_{2,i}} \quad (5.48)$$

The functions used to defined the Mie-Grüneisen equations are

$$\Gamma_i = \Gamma_{0,i} + 1, \quad (5.49)$$

$$\begin{aligned} \Pi_i = \Gamma_{0,i} \rho_i \left(- \frac{A_i}{(\varepsilon_{1,i} - 1) \rho_{0,i}} V_i^{1-\varepsilon_{1,i}} \right. \\ \left. + \frac{B_i}{(\varepsilon_{2,i} - 1) \rho_{0,i}} V_i^{1-\varepsilon_{2,i}} - e_{0,i} \right) - p_{\text{ref},i} \end{aligned} \quad (5.50)$$

and

$$\delta_i = \frac{A_i}{\mathcal{E}_{1,i}} \left[\mathcal{E}_{1,i} V_i^{-\mathcal{E}_{1,i}} \frac{\mathcal{E}_{1,i} - \Gamma_{0,i} - 1}{\rho_i} + \frac{\Gamma_{0,i}}{\rho_{0,i}} \right] \frac{1}{\Gamma_{0,i}} + \frac{B_i}{\mathcal{E}_{2,i}} \left[\mathcal{E}_{2,i} V_i^{-\mathcal{E}_{2,i}} \frac{\mathcal{E}_{2,i} - \Gamma_{0,i} - 1}{\rho_i} + \frac{\Gamma_{0,i}}{\rho_{0,i}} \right] \frac{1}{\Gamma_{0,i}} - e_{0,i} \quad (5.51)$$

Variable	Description
A	Model coefficient [Pa]
B	Model coefficient [Pa]
E1	Model coefficient []
E2	Model coefficient []
rho0	Reference density [kg/m ³]
Gamma0	Grüneisen coefficient []
e0	Reference energy (default: 0) [J/kg]

5.2.2.10 DoanNickel

The Doan-Nickel EOS includes changes in the composition of air at high temperatures and the changing specific heat ratio due to dissociation of molecules Doan and Nickel (1963). Due to the way this model is defined, it can currently only be used with inviscid fluids and a constant specific heat at constant volume (eConst). The specific heat is a function of the density and internal energy and is defined as

$$\Gamma_i = 1 + \left[0.161 + 0.255e^{\tilde{e}/e_1} f_1 + 0.280e^{\tilde{e}/e_2} (1 - f_1) + 0.137e^{\tilde{e}/e_3} f_2 + 0.050f_3 \right] \left(\frac{\rho_i}{\rho_{0,i}} \right)^{\alpha(\rho_i, \tilde{e})} \quad (5.52)$$

where $\tilde{e} = e \cdot 10^{-6}$. $e_1 = 4.46$, $e_2 = 6.63$, and $e_3 = 25.5$ are the internal energies at which oxygen dissociates, everything is dissociated, and electrons contribute, respectively. $\rho_{0,i}$ is the density of air at sea-level (1.293 [kg/m³], and

$$\alpha(\rho_i, \tilde{e}) = [0.048f_1 + 0.032(1 - f_1)(1 - f_2)] \log_{10}(\tilde{e}) + 0.045f_3 \quad (5.53)$$

with

$$f_1 = \frac{1}{\exp\left(\frac{\tilde{e} - e_{O_2 \rightarrow 2O}}{\Delta e_{O_2 \rightarrow 2O}}\right) + 1} \quad (5.54)$$

$$e_{O_2 \rightarrow 2O} = 8.5 + 0.357 \log_{10} \left(\frac{\rho_i}{\rho_{0,i}} \right) \quad (5.55)$$

$$\Delta e_{O_2 \rightarrow 2O} = 0.975 \left(\frac{\rho_i}{\rho_{0,i}} \right)^{0.05} \quad (5.56)$$

$$f_2 = \frac{1}{\exp\left(\frac{e_{N_2 \rightarrow 2N} - \tilde{e}}{\Delta e_{N_2 \rightarrow 2N}}\right) + 1} \quad (5.57)$$

$$e_{N_2 \rightarrow 2N} = 45.0 \left(\frac{\rho_i}{\rho_{0,i}} \right)^{0.0157} \quad (5.58)$$

$$\Delta e_{N_2 \rightarrow 2N} = 4.0 \left(\frac{\rho_i}{\rho_{0,i}} \right)^{0.085} \quad (5.59)$$

$$f_3 = \frac{1}{\exp\left(\frac{e_e - \tilde{e}}{\Delta e_e}\right) + 1} \quad (5.60)$$

$$e_e = 160.0 \quad (5.61)$$

and

$$\Delta e_e = 6.0 \quad (5.62)$$

Due to the complexity of the model, and the fact that it was *specifically* developed as a model for air, all coefficients are hard-coded and cannot be changed. This equation of state should only be used to represent standard air.

5.2.3 Temperature-based

While it is preferable to use an internal energy based equation of state for compressible flows, some equations of state cannot be converted, and therefore use the local temperature to define the thermodynamic pressure. Generally the Grüneisen coefficient used to calculate the shared pressure is defined as

$$\Gamma_i = \frac{v_i}{C_v} \frac{\partial p}{\partial T} \quad (5.63)$$

where C_v is the specific heat at constant volume, and the speed of sound is defined

$$c_i^2 = v_i^2 \left[\left(\frac{\partial p_i}{\partial T} \right)^2 \frac{T}{C_v} - \frac{\partial p}{\partial v_i} \right] \quad (5.64)$$

Unless otherwise specified, these relations are used to calculate the relevant quantities.

5.2.3.1 perfectGas

The perfect gas equation of state uses the standard ideal gas pressure relation, however the specific heat ratio is not held constant, allowing for the use of non-constant heat capacity models.

$$p_i = RT\rho_i \quad (5.65)$$

5.2.3.2 JWLC

The C-form of the Jones Wilkins Lee equation of state (Souers et al., 2000) does not have a dependence on the temperature or internal energy and has a pressure defined as

$$p_i = A_i e^{-R_{1,i}V} + B_i e^{-R_{2,i}V} + \frac{C_i}{V^{1+\omega_i}} \quad (5.66)$$

the speed of sound if defined

$$c_i^2 = \frac{V^2}{\rho_{0,i}} \left(A_i e^{-R_{1,i}V} + B_i e^{-R_{2,i}V} + \frac{C_i(1+\omega_i)}{V^{2+\omega_i}} \right) \quad (5.67)$$

The Grüneisen coefficient is the same as the standard JWL equation of state.

Variable	Description
A	Model coefficient [Pa]
B	Model coefficient [Pa]
C	Model coefficient [Pa]
R1	Model coefficient []
R2	Model coefficient []
rho0	Unreacted or reference density [kg/m ³]
omega	Grüneisen coefficient []

5.2.3.3 AbelNobel

The equation of state of Abel-Nobel is commonly used for modeling simulate explosive materials, specifically for propellants (Johnston, 2005). Unlike the JWL equation of state, the BKW models uses the temperature, rather than the internal energy, to define the pressure.

$$p_i = \frac{RT\rho_i}{1 - b_i\rho_i} \quad (5.68)$$

where b is the co-volume of the fluid.

Variable	Description
b	Co-volume [m ³ /kg]

5.2.3.4 VanderWaals

The equation of state of Abel-Nobel is commonly used for modeling simulate explosive materials, specifically for propellants (Johnston, 2005). Unlike the JWL equation of state, the BKW models uses the temperature, rather than the internal energy, to define the pressure.

$$p_i = \frac{RT\rho_i}{1 - b_i\rho_i} \quad (5.69)$$

where b is the co-volume of the fluid.

Variable	Description
b	Co-volume [m ³ /kg]

5.2.3.5 BKW

The Becker-Kistiakowsky-Wilson equation of state is another equation of state that is commonly used to simulate explosive materials (Mader, 1963). Unlike the JWL equation of state, the BKW models uses the temperature, rather than the internal energy, to define the pressure.

$$p_i = RT\rho_i (1 + xe^{\beta x}) \quad (5.70)$$

where

$$x = \frac{\kappa k\rho_i}{W_i(T + \theta)^a} \quad (5.71)$$

and W_i is the molecular weight in [kg/kmol].

Variable	Description
beta	Model coefficient []
a	Temperature exponent []
Theta	Model coefficient [K]
kappa	Model coefficient [K ^a]
k	Specie co-volume [m ³ /kmol]

5.2.3.6 BWR

The Benedict-Webb-Rubin equation of state was developed by (Benedict et al., 1942) to describe real gases using a set of eight coefficients that must be experimentally determined. The pressure is defined as

$$p_i = RT\rho_i + \left(B_0RT - A_0 - \frac{C_0}{T^2}\right)\rho_i^2 + (bRT - a)\rho_i^3 + a\alpha\rho_i^6 + c\frac{\rho_i^3}{T^2}(1 + \gamma\rho_i^2)e^{-\gamma\rho_i^2} \quad (5.72)$$

Variable	Description
A0	Model coefficient
B0	Temperature exponent
C0	Model coefficient
a	Model coefficient
b	Model coefficient
c	Model coefficient
alpha	Model coefficient
gamma	Specific heat ratio []

5.2.3.7 Murnaghan

The Murnaghan (or Munahan) equation of state is used for solid material and has no temperature dependence (Souers et al., 2000). The pressure is a function of only the density and is defined as

$$p_i = p_{\text{ref}} + \frac{1}{\kappa n} \left(\frac{\rho_i}{\rho_{0,i}} - 1 \right)^n \quad (5.73)$$

Variable	Description
pRef	Reference pressure [Pa]
rho0	Reference density [kg/m ³]
kappa	Model coefficient ($1/K_0$) [Pa ⁻¹]
K0	Bulk modulus (can be used in place of κ) [Pa]
n (or K0)	
Gamma	Grüneisen coefficient []

5.2.4 tabulatedEOS

The tabulated equation of state can be used to calculate the pressure using the density and temperature. The data is read from a two dimensional tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x", "y", and "f" names are "rho", "T", and "p" respectively.

5.2.4.1 BirchMurnaghan2

The second order Birch-Murnaghan equation of state is the second order expansion of the Birch-Murnaghan model. The pressure is defined as

$$p_i = p_{\text{ref}} + \frac{3K_0}{2K'_0} \left[\left(\frac{\rho_i}{\rho_{0,i}} \right)^{7/3} - \left(\frac{\rho_i}{\rho_{0,i}} \right)^{5/3} \right] \quad (5.74)$$

Many coefficients can be found for this in Zheng and Zhao (2016).

Variable	Description
pRef	Reference pressure [Pa]
rho0	Reference density [kg/m ³]
K0	Bulk modulus [Pa]
Gamma	Grüneisen coefficient []

5.2.4.2 BirchMurnaghan3

The Third order Birch-Murnaghan equation of state is the third order expansion of the Birch-Murnaghan (see Zheng and Zhao (2016)). The pressure is defined as

$$p_i = p_{\text{ref}} + \frac{3K_0}{2K'_0} \left[\left(\frac{\rho_i}{\rho_{0,i}} \right)^{7/3} - \left(\frac{\rho_i}{\rho_{0,i}} \right)^{5/3} \right] \times \left\{ 1 + 0.75(K'_0 - 4) \left[\left(\frac{\rho_i}{\rho_{0,i}} \right)^{2/3} - 1 \right] \right\} \quad (5.75)$$

Variable	Description
pRef	Reference pressure [Pa]
rho0	Reference density [kg/m ³]
K0	Bulk modulus [Pa]
K0Prime	Change in bulk modulus w.r.t. pressure []
Gamma	Grüneisen coefficient []

5.2.5 eTabulated/tabulatedMG

In addition to the previously mentioned equations of state and thermodynamic models, tabulated models may also be used. This is useful when simulating real gasses, and high temperatures and pressures when standard models are no longer sufficient. Any combination of pressure and temperature tables can be used, as long as they are a function of density and internal energy (x and y , respectively). The internal energy is initialized using a reverse look up given a pressure and a density. See 13.1 for more information on usage.

NOTE: Because of the way the selection of models is handled, this option will not show up as an available option if other models are used. Currently only `const`, `icoTabulated`, and `tabulated` transport models are available. An example is shown below:

```
phase
{
    type          basic;
    thermoType
    {
        transport  const;
```

```

        thermo      eTabulated;
        equationOfState tabulatedMG;
    }
    specie
    {
        molWeight    28.97;
    }
    transport
    {
        mu           0;
        Pr           0;
    }
    thermodynamics
    {
        file         "T.csv";
        mod          ln;      // log(p_i)=p_table

        rhoMod       log10;   // log_10(rho_i)=minRho+i*deltaRho
        nRho         7;       // Number of columns
        minRho       -3.0;    // 0.001 kg/m^3
        deltaRho     1.0;

        eMod         ln;      // log(e)=mine+j*deltaE
        nE           40;      // Number of rows
        minE         11.8748;
        deltaE       1.0;
    }
    equationOfState
    {
        file         "p.csv";
        mod          ln;      // log(p_i)=p_table

        rhoMod       log10;   // log_10(rho_i)=minRho+i*deltaRho
        nRho         7;       // Number of columns
        minRho       -3.0;    // 0.001 kg/m^3
        deltaRho     1.0;
        rhoIsReal    false;
    }

```

```

    eMod    ln;      // log(e)=mine+j*deltaE
    nE      40;      // Number of rows
    minE    11.8748;
    deltaE  1.0;
    eIsReal false;
  }
}

```

5.3 Fluid Transport

The transport models are used to define quantities such as viscosity and thermal diffusion. Only Newtonian viscosity models are currently available. All transport models, with the exception of the tabulated methods ported from OpenCFD Ltd. (2018b).

5.3.1 const

Constant values for viscosity (μ) and Prandtl number are used.

$$\kappa = \mu C_p / \text{Pr} \quad (5.76)$$

Variable	Description
mu	Dynamic viscosity [Kg/m/s]
Pr	Prandtl number []

5.3.2 sutherland

Viscosity is calculated using Sutherland's formula for viscosity

$$\mu(T) = A_s \frac{\sqrt{T}}{1 + T_s/T} \quad (5.77)$$

$$\kappa = \mu C_v \left(1.32 + \frac{1.77R}{C_v} \right) \quad (5.78)$$

Variable	Description
Ts	Sutherland temperature [K]
As	Sutherland coefficient [kg/m/s/K ^{1/2}]

5.3.3 logPolynomial

Transport package using polynomial functions of the natural logarithm of temperature for the natural logarithm of dynamic viscosity and thermal conductivity:

$$\log(\mu(T)) = \sum_{i=0}^7 \mu_i (\log(T))^i \quad (5.79)$$

$$\log(\kappa(T)) = \sum_{i=0}^7 \kappa_i (\log(T))^i \quad (5.80)$$

Variable	Description
muCoeffs<8>	Dynamic viscosity coefficients
kappaCoeffs<8>	Conductivity coefficients

5.3.4 polynomial

Transport package using polynomial functions of temperature for the dynamic viscosity and thermal conductivity:

$$\mu(T) = \sum_{i=0}^7 \mu_i T^i \quad (5.81)$$

$$\kappa(T) = \sum_{i=0}^7 \kappa_i T^i \quad (5.82)$$

Variable	Description
muCoeffs<8>	Dynamic viscosity coefficients
kappaCoeffs<8>	Conductivity coefficients

5.3.5 WLF

Transport package using the Williams-Landel-Ferry model for viscosity of polymer melts (Williams et al., 1955):

$$\mu(T) = \mu_0 \exp \left(-\frac{C_1(T - T_r)}{C_2 + T - T_r} \right) \quad (5.83)$$

$$\kappa(T) = Cp(T) \frac{\mu(T)}{Pr} \quad (5.84)$$

Variable	Description
mu0	Reference dynamic viscosity [Pa s]
TRef	Referece temperature [K]
C1	WLF constant []
C2	WLF constant [K]
Pr	Prandtl number []

5.3.6 icoTabulated

The tabulated, incompressible transport model can be used to calculate the viscosity and conductivity using only the temperature. The data is read from two one dimensional tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x" and "f" names are "T" and "mu"/"kappa" respectively.

5.3.7 tabulated

The tabulated transport model can be used to calculate the viscosity and conductivity using the density and temperature. The data is read from two, two dimensional tables. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x", "y", and "f" names are "rho", "T", and "mu"/"kappa" respectively.

5.4 Solid Transport

The transport models are used to define the thermal conductivity of the material. All transport models, with the exception of the tabulated methods are ported from OpenCFD Ltd. (2018b).

5.4.1 constIso

A constant, scalar value (isotropic), is used for for thermal conductivity.

Variable	Description
kappa	Thermal conductivity [W/m/K]

5.4.2 `constAnIso`

A constant, vector value (anisotropic), is used for thermal conductivity.

Variable	Description
kappa	Thermal conductivity (vector) [W/m/K]

5.4.3 `exponential`

Thermal conductivity is calculated using an exponential function

$$\kappa(T) = \kappa_0 \left(\frac{T}{T_{ref}} \right)^{n_0} \quad (5.85)$$

Variable	Description
kappa0	Reference thermal conductivity [W/m/K]
n0	Exponential coefficient []
Tref	Reference temperature [K]

5.4.4 `polynomial`

Thermal conductivity is calculated using polynomial functions of temperature.

$$\kappa(T) = \sum_{i=0}^7 \kappa_i T^i \quad (5.86)$$

Variable	Description
kappaCoeffs<8>	Conductivity coefficients

5.4.5 `tabulatedIso`

The tabulated, isotropic solid transport model can be used to calculate the conductivity using the temperature. The data is read from a one dimensional table. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x" and "f" names are "T" and "kappa" respectively.

5.4.6 tabulatedAnIso

The tabulated, anisotropic solid transport model can be used to calculate the conductivity using the temperature. The data is read from a one dimensional table. See Sec. 13.1 for more information on the available options for modifiers and interpolation schemes. The "x" and "f" names are "T" and "kappa" respectively.

5.5 Basic thermodynamic model

The `basicFluid` and `basicSolid` models ("type" basic) uses a transport model, thermodynamic model, and equation of state to describe a phase. The basic model should be used for non-reacting phases.

5.6 Detonating thermodynamic model

In order to better describe a detonating material, a 'detonating' thermodynamic model can be used. This model transitions between an unreacted state and a reacted state using an activation model. Each state (unreacted and reacted) is described by its own basic thermodynamic model, allowing for different models and coefficients to be used for values such as viscosity or specific heats. In addition to the activation model, afterburn models can also be used to add additional energy into the system after the initial detonation reaction has taken place.

This same methodology is applied to solid phases where energy is released locally to increase the thermal energy (internal energy). Only activation models are currently available.

Some standard combinations of unreacted and reacted equations of states include the `JWL++` (Murnaghn and `JWLC`), and the `blendedJWL` (`solidJWL` and `JWL`).

NOTE: Whilst it will not give the correct temperature, using an ideal gas is a simple option if the solid properties of the unreacted material are not known.

```
phase1
{
    type                detonating;
```

```

reactants
{
    thermoType
    {
        ...
    }
    ...
}
products
{
    thermoType
    {
        ...
    }
    ...
}

activationModel linear;
initiation
{
    E0      9.0e9;
    points  ((0 0 0) (0.1 0.1 0.1));
    delays  (0 1e-5);
    radii   (0.001 0.005);
    vDet    5000;
}

afterburnModel Miller;
MillerCoeffs
{
    Q0      1e7;
    m       0.1667;
    n       0.5;
    a       0.0195e6;
    pMin    1.1e5;
}
}

```

5.6.1 Activation Models

In order to accurately model the release of energy, different models can be used to approximate the speed at which the solid reactants become the product gases. The currently available models include the linear model, a pressure based model, and finally an Arrhenius rate reaction. These models all use a reaction progress variable, λ_i , to describe the state of individual cell activation. Additionally, in order to add numerical stability, the initial energy is defined using the unreacted equation of state, and the reaction energy, E_0 , is added based on the time rate of change of the reaction progress variable, i.e.

$$\dot{E} = \frac{E_0 \rho_i}{\rho_{0,i}} \frac{d\lambda}{dt} \quad (5.87)$$

The phase properties are calculated using

$$x_i = \lambda_i^\ell x_{r,i} + (1 - \lambda_i^\ell) x_{u,i} \quad (5.88)$$

where $x_{r,i}$ is the reacted phase quantity, $x_{u,i}$ is the unreacted phase quantity, and ℓ is the exponent used to blend the unreacted and reacted states (lambdaPow). By default ℓ is one. The advection scheme and limiters for λ_i need to be specified in the *fvSchemes* in the same manner as the other transported quantities.

The setting of detonation points can be done for any activation model. By setting detonation points, the cells within the charge with the radii specified are set to full activated when run time reaches the given delays. If no radii are provided, only the cell containing the point will be activated. Additionally, delays can be used to specify the time at which the given point and cells within the radius are activated. For a single detonation point, the "useCOM" option can be used to calculate the center of mass of an individual charge.

Transport of the reaction progress variable is described by the transport equation

$$\frac{\partial \alpha_i \rho_i \lambda_i}{\partial t} + \nabla \cdot (\alpha_i \rho_i \mathbf{u} \lambda_i) = \alpha_i \rho_i \dot{R}_i \quad (5.89)$$

where $\dot{R}_i$ is the reaction rate determined by the model of choice.

Variable	Description
E_0	Detonation energy [Pa]
points	Detonation points [m]
radii (radius)	Initiation radii around detonation points (default: 0) [m]
delays	Delay time of detonation points (default: 0) [s]
lambdaPow	Exponent used for blending (default: 1) []
useCOM	Is the center of mass used as the detonation point (default: false)

5.6.1.1 none

The **none** activation model assumes that all cells are activated when the simulation begins. This is the equivalent to an infinitely fast detonation velocity. The use of delays and detonation points are not valid here because the detonation is assumed to be instantaneous. There is no transport of the reaction progress variable.

5.6.1.2 linear

The simplest activation model uses a list of detonation points and a constant detonation velocity to determine which cells have been activated. This gives

$$\lambda_{i,j} = \begin{cases} 1 & \text{if } |\mathbf{x} - \mathbf{x}_{\text{det},i,j}| < v_{\text{det},i}t \\ 0 & \text{else} \end{cases} \quad (5.90)$$

where $\mathbf{x}$ is the cell center, $\mathbf{x}_{\text{det},i,j}$ is the j^{th} detonation point, $v_{\text{det},i}$ is the detonation velocity, and t is the time. In order to account for multi-point detonation, the actual λ is defined as

$$\lambda_i = \max(\lambda_{i,j}) \quad (5.91)$$

Variable	Description
vDet	Speed of propagation within the material [m/s]

5.6.1.3 programmedIgnition

The **programmedIgnition** activation model is slightly more complex than the linear activation model and includes the option to use activation due to compression of material, and is based on the work of Giroux (1973). The

reaction progress variable is defined using a combination of two models, the compression

$$\lambda_{\beta,i} = \frac{1 - \frac{\rho_{0,i}}{\rho_i}}{1 - V_{\text{CJ},i}} \quad (5.92)$$

where $V_{\text{CJ},i}$ is the Chapman-Jouguet specific volume which is computed using the Chapman-Jouguet pressure. The reference density, $\rho_{0,i}$ is read from the product equation of state. The programmed definition of the reaction progress variable is a modified version of the linear activation, where a cell reacts at a time-rate determined by the detonation velocity and the local grid size

$$\lambda_{\text{programmed},i} = \max(t - t_{\text{ign},j}, 0) \frac{v_{\text{det}}}{1.5dx} \quad (5.93)$$

where t_{ign} is the time of ignition determined by the physical location and the detonation velocity, and dx is the local grid size.

If the **beta** burn option is used, $\lambda_i = \lambda_{\beta,i}$; if the **programmed** burn option is used, $\lambda_i = \lambda_{\text{programmed},i}$; and if the **programmedBeta** burn option is used $\lambda_i = \max(\lambda_{i,\beta}, \lambda_{\text{programmed},i})$.

Variable	Description
vDet	Speed of propagation within the material [m/s]
Pcj	Chapman-Jouguet pressure [Pa]
burnModel	Burn model used (beta , programmed , or programmedBeta)

5.6.1.4 pressureBased

A more advanced activation model uses the local cell pressure, determined by the equation of state, and follows the evolution equation

$$\dot{R}_i = I \left(\frac{\rho_i}{\rho_{0,i}} - 1 - a \right)^x (1 - \lambda)^b + G_1(1 - \lambda)^c \lambda^d p^y + G_2(1 - \lambda)^e \lambda^f p^z \quad (5.94)$$

The first term is responsible for the initial detonation based on the compression of the material. The following two terms account for the subsequent reaction based on the local pressure. All terms can be turned on or off by specifying the minimum and maximum values of lambda at which the stages occur.

Variable	Description
I	Ignition rate [s ⁻¹]
a	Compression ratio offset []
b	Ignition (1 - λ) exponent []
x	Compression ratio exponent []
maxLambdaI	Maximum value of λ that the ignition term is used []
G1	First activation rate [Pa ^{-y} s ⁻¹]
c	First (1 - λ) exponent []
d	First λ exponent []
y	First pressure exponent []
minLambda1	Minimum value of λ that the first step is used [] (default: 0)
maxLambda1	Maximum value of λ that the first step is used [] (default: 1)
G2	Second activation rate [Pa ^{-z} s ⁻¹]
e	Second (1 - λ) exponent []
f	Second λ exponent []
z	Second pressure exponent []
minLambda2	Minimum value of λ that the second step is used [] (default: 0)
maxLambda2	Maximum value of λ that the second step is used [] (default: 1)
pScale	Coefficient used to scale pressure [] (default: 1)

5.6.1.5 ArrheniusRate

A more physical representation of the activation rate uses the local temperature, defined by the internal energy and the specific heat of the mixture. The evolution equation for the reaction progress variable is

$$\dot{R}_i = (1 - \lambda) \begin{cases} 0 & T < T_{\text{ign}} \\ d_p^2 A_{\text{low}} \exp\left(\frac{E_{a,\text{low}}}{RT}\right) & T_{\text{ign}} \leq T < T_s \\ A_{\text{high}} \exp\left(\frac{E_{a,\text{high}}}{RT}\right) & T \geq T_s \end{cases} \quad (5.95)$$

Where the particle diameter is determined by the chosen diameter model (see Sec. 8), and R is the ideal gas constant.

Variable	Description
diameterModel	Diameter model used to determine the particle diameter
ALow	Low activation rate [$\text{m}^{-2} \text{s}^{-1}$]
EaLow	Low activation energy [m^2/s^2]
AHigh	High activation rate [s^{-1}]
EaHigh	High activation energy [m^2/s^2]
Ts	Switch temperature [K]
Tign	Minimum temperature of reaction

NOTE: This model is still in development and has not been validated

5.6.2 Afterburn Models

When reactions occur over a period of time after the initial charge has been activated, additional energy can be added that results in higher pressure and temperature, and is referred to *afterburn*. Afterburn is typical in under-oxygenated and/or non-ideal explosives, which may release energy at a later time. The advection scheme and limiters for c_i need to be specified in the *fvSchemes* in the same manner as the other transported quantities.

5.6.2.1 none [default]

No additional energy is added. If the keyword "afterburnModel" is not found in the phase dictionary, this is the model that is used.

5.6.2.2 constant

A constant amount of afterburn energy is added where $\dot{Q}$ is specified by the user.

Variable	Description
Qdot	Afterburn energy addition rate[J/kg/s]

5.6.2.3 linear

Afterburn energy is added linearly from a start time to an end time where

$$\dot{Q} = \begin{cases} \frac{Q_0}{\Delta t} & \text{if } t_{\text{start}} < t < t_{\text{end}} \\ 0 & \text{else} \end{cases} \quad (5.96)$$

Variable	Description
Q0	Afterburn energy [J/kg]
tStart	Time energy begins being added [s]
tEnd	Time energy stops being added [s]

5.6.2.4 Miller

The model of Miller (1995) uses an evolution equation to determine the fraction of the total amount of afterburn energy added to the system due to unburnt material. c_i is the reaction progress variable that has a value between 0 and 1, with an evolution equation defined as

$$\frac{\partial \alpha_i \rho_i c_i}{\partial t} + \nabla \cdot (\alpha_i \rho_i \mathbf{u} c_i) = \alpha_i \rho_i a (1 - c_i)^m (p \times p_{\text{scale}})^n \quad (5.97)$$

and the total afterburn energy is found using

$$\dot{Q} = \frac{dc_i}{dt} Q_0 \quad (5.98)$$

Variable	Description
a	Model constant [Pa^{-n}]
m	Reaction progress variable exponent []
n	Pressure exponent []
Q0	Afterburn energy [J/kg]
pMin	Minimum pressure that afterburn occurs [Pa]
pScale	Scale factor for pressure (default: 1) []

5.7 Multicomponent thermodynamic model

The `multicomponent` fluid thermodynamic model allows for the use of multiple species that use the same equation of state by mass averaging equation of state coefficients of the species. The mass fractions of the species are required as initial conditions (`Ydefault` can be used to give initial conditions for multiple species). Additionally, a "defaultSpecie" must be provided in the thermodynamic dictionary to specify a *slave* specie

Below is an example of a multicomponent fluid

```
type      multicomponent;
species   (ignitor air);
```

```

defaultSpecie air;
calculateDensity yes;
thermoType
{
    transport    const;
    thermo       eConst;
    equationOfState AbelNobel;
}

ignitor
{
    equationOfState
    {
        b          0.99e-3;
    }
    specie
    {
        molWeight   22.935;
    }
    transport
    {
        mu          1.81e-5;
        Pr          0.7;
    }
    thermodynamics
    {
        Cv          1450;
        Hf          1;
    }
}

air
{
    equationOfState
    {
        b          1.02e-3;
    }
    specie
    {

```

```

        molWeight      28.6689;
    }
    transport
    {
        mu              1.81e-5;
        Pr              0.7;
    }
    thermodynamics
    {
        Cv              1450;
        Hf              0;
    }
}

```

5.8 Two/multiphase models

To simulate a two-phase or multiphase system, the keyword "phases" must be defined in the *phaseProperties* dictionary and defines the list of phases used in a simulation. The selection of a two-phase or multiphase system is determined by the number of defined phases. The mixture pressure is defined by

$$p = \frac{\sum_i \alpha_i \xi_i p_i}{\sum_i \alpha_i \xi_i} \quad (5.99)$$

where

$$\xi_i(\rho_i) = \frac{1}{\Gamma_i - 1} \quad (5.100)$$

and p_i is the pressure from a single equation of state (Zheng et al., 2011). The speed of sound within a given phase is given by

$$c_i = \sqrt{\frac{\sum_i y_i \xi_i c_i^2}{\sum_i \xi_i}} \quad (5.101)$$

with

$$y_i = \frac{\alpha_i \rho_i}{\rho} \quad (5.102)$$

Both the initial energy and temperature are calculated using a root finding method (Newton-Raphson by default) that include the contributions of all phases so the calculated values obey the mixture equations. See 13.3 for additional options.

5.9 Initialization

The initialization of fields can be handled in several ways, but ρ_i , p , and T must be given as initial conditions. Some of these fields may be overwritten, but the boundary conditions are used to construct the conserved quantities. For equations of state in the Mie-Grüneisen form, by default the pressure and density are used to initialize the internal energy. Temperature based equations of state initialize the internal energy using the density and temperature. If the "calculateDensity" keyword is defined, and is set to "yes" in the given equation of state dictionary, then the density is calculated from the pressure and temperature prior to the initialization of the internal energy. Each of the initialization methods use a Newton-Raphson root finding method to find the necessary quantities. If the internal energy field is provided in the initial conditions folder, the pressure and temperature fields are calculated directly from the density and internal energy.

5.10 Example

An example of a single phase *phaseProperties* file can be seen below

```
mixture
{
    type          basic;
    calculateDensity yes;
    thermoType
    {
        transport  const;
        thermo      eConst;
        equationOfState idealGas;
    }
    transport
    {
        mu          0.0;
```

```

        Pr      1.0;
    }
    thermodynamic
    {
        Cv      718;
        Hf      0.0;
    }
    idealGas
    {
        gamma    1.4;
    }
}

```

For a two or multiphase simulation, the *phaseProperties* file is defined as

```

phases (air water);
air
{
    type          basic;
    thermoType
    {
        transport  const;
        thermo     eConst;
        equationOfState idealGas;
    }
    transport
    {
        mu        0.0;
        Pr        1.0;
    }
    thermodynamic
    {
        Cv        718;
        Hf        0.0;
    }
    equationOfState
    {
        gamma     1.4;
    }
}

```

```

    }
}
water
{
    type          basic;
    thermoType
    {
        transport  const;
        thermo     hConst;
        equationOfState stiffenedGas;
    }
    specie
    {
        molWeight  18.0;
    }
    transport
    {
        mu         0.0;
        Pr         1.0;
    }
    thermodynamic
    {
        Cp         4186;
        Hf         0.0;
    }
    equationOfState
    {
        gamma      4.4;
        a          7e8;
    }
}

```

6 Turbulence

In addition to the hyperbolic flux terms, viscous effects can be included in simulations. This includes the use of turbulence models such as Reynolds averaged Navier-Stokes (RANS) or Large Eddy simulations (LES) models. All single phase turbulence models standard to OpenFOAM including k-epsilon, k-omega, Smagorinsky, as well as many others are available. The specified model introduces additional transport equations, as well as modifications to transport coefficients like viscosity (ν) and thermal diffusivity (α) which modify the viscous terms in the conservation equations. When the maximum viscosity is zero, the *constant/momentumTransport* (or *constant/turbulence-Properties*) does not need to be included, however if any phase has viscous terms, this dictionary needs to be included along with the appropriate turbulence model (laminar is the same as no turbulence model). The available turbulence models are listed below, and these can also be found using the '-listMomentumTransportModels' option when running **blastFoam**.

Additional **kOmega** turbulence models have been added that include compressibility corrections based on Sarkar et al. (1991) and Zeman (1990).

Turbulence models: LES, RAS, laminar (e.g. no model)

laminar models:

- Giesekus
- Maxwell
- Stokes (default)
- generalizedNewtonian

RAS models:

- LRR
- LaunderSharmaKE
- RNGkEpsilon
- SSG
- SpalartAllmaras
- buoyantKEpsilon
- kEpsilon
- kOmega
- kOmega2006
- kOmegaSST
- kOmegaSSTLM
- kOmegaSSTSAS
- compressible::kOmega
- compressible::kOmega2006

- `compressible::kOmegaSST`
- `realizableKE`
- `kOmegaSSTLM`
- `kOmegaSSTSAS`
- `v2f`

LES models:

- `DeardorffDiffStress`
- `dynamicKEqn`
- `Smagorinsky`
- `dynamicLagrangian`
- `SpalartAllmarasDDES`
- `kEqn`
- `SpalartAllmarasDES`
- `kOmegaSSTDES`
- `SpalartAllmarasIDDES`
- `compressible::kOmegaSSTDES`
- `WALE`

NOTE: Many of the turbulence models require additional field (*k*, *epsilon*, *omega*, etc.) to be specified in the initial conditions folder.

7 Region models

When multiple regions are used, blastFoam solvers use global poly patches which do not require any special mesh patches to transfer data between field. Instead, the information is transferred using field specific boundary conditions. The interpolation method, sampled patch, and optional displacement field is specified in the *constant/regionProperties* file. An example of *regionProperties* can be seen below

```
regions
(
    fluid      (air)
    solid      (building)
);

interfaces
(
    air
    {
        air_to_building
        {
            sampleRegion    building;
            samplePatch      building_to_air;
            mappingType       intersection;
            mappingType       inverseDistance;
        }
    }
    building
    {
        building_to_air
        {
            sampleRegion    air;
            samplePatch      air_to_building;
            displacementField pointD;
        }
    }
);
```

Additionally the mapping method can be provided in the region specific *fvSchemes* under the interpolationSchemes via

```
interpolationSchemes
{
    default linear;
    patchName
    {
        sampleRegion    building;
        samplePatch      building_to_air;
        mappingType       intersection;
        mappingType       inverseDistance;
    }
};
```

7.1 Boundary conditions

Several boundary conditions have been implemented that make use of the global patches and are described below.

7.1.1 globalMapped

Both vol and point field can use the `globalMapped` boundary condition which directly maps values from one patch to another where the neighbor patch is specified in *regionProperties*.

Variable	Description
nbrName	Name of field to map from

7.1.2 globalInterpolated

Point field can use the `globalInterpolated` boundary condition which maps a volume field to a point field where the neighbor patch is specified in *regionProperties*.

Variable	Description
nbrName	Name of field to map from

7.1.3 globalTemperatureCoupled

Using the same methodology as the `globalMapped` boundary condition, the `globalTemperatureCoupled` boundary condition computes the temperature of a patch using the neighbor temperature, heat transfer coefficient, and radiation heat flux. Optionally when the entire boundary does not overlap the neighbor patch and unmapped temperature can be specified.

Variable	Description
TNbr	Name of neighbor temperature (default: T)
hNbr	Name of neighbor convective heat transfer coefficient (default: none)
h	Name of convective heat transfer coefficient (default: none)
qrNbr	Name of neighbor radiation heat flux (default: none)
qr	Name of radiation heat flux (default: T)
limitGrad	Is the temperature gradient limited (default: false)
unmappedT	Unmapped temperature (default: 293.15) [K]

8 Diameter models

The diameter of a dispersed phase is used to calculate surface reaction rates in the Arrhenius activation model as well as interfacial transfer sources in the **blastEulerFoam** solver. Below are is a short description of the models currently available.

8.1 constant

A constant diameter is used.

NOTE: The diameter field is not written

Entry	Description
d	Diameter of the phase [m]

8.2 constantMass

The diameter is determined using a reference diameter at a reference density. The mass is defined as

$$M = \frac{\pi d_0^3 \rho_0}{6} \quad (8.1)$$

and the true diameter is calculated using

$$d = \left(\frac{6M}{\rho\pi} \right)^{1/3} \quad (8.2)$$

Entry	Description
rho0	Reference density [kg/m ³]
d0	Reference diameter [m]

8.3 reacting

The **reacting** diameter model adds an additional source term in the transport equation for the diameter to increase/decrease the local diameter based

on the given reaction rate

$$\frac{\partial d_i}{\partial t} + \mathbf{u}_i \cdot \nabla d_i = k \quad (8.3)$$

where k is the specified reaction rate (see Sec. 8.4)

Entry	Description
d	Initial diameter [m]
reactionRate	Reaction rate model

8.4 Reaction Rates

Reaction rates are used to give rates based on temperature and/or pressure currently there are two options, pressure based, and Arrhenius rate

8.4.1 pressureBased

The **pressureBased** reaction rate uses a coefficient, pressure scale, pressure offset (in scaled units) and pressure exponent to compute the rate

$$k = \beta (\bar{p})^\alpha \quad (8.4)$$

and $\bar{p} = p \times p_{scale}$. For example, if the pressure relation is in MPa, $p_{scale} = 1e-6$.

Entry	Description
alpha	Pressure exponent []
beta	Coefficient [Pa ^{-α}]
pScale	Pressure scale []
offset	Pressure offset [Pa/pScale]

8.4.2 Arrhenius

The rate given by the Arrhenius reaction rate is

$$k = AT^\beta \exp\left(\frac{-T_a}{T}\right) \quad (8.5)$$

Entry	Description
A	Rate constant [1/s]
beta	Temperature exponent []
Ta	Activation temperature [K]

9 Flux Evaluation

blastFoam relies on explicit solutions for the evolution of conservative variables. This has several benefits in terms of order of accuracy and computational cost. First, higher-order, explicit Runge-Kutta time integration schemes can be used which allow for higher order time-integration accuracy. Secondly, because of the required time steps of the problems of interest, iterative matrix solutions are not necessary to ensure accuracy or stability. Instead, stability can be improved through use of SSP time integration methods or more restrictive limiters.

All of the methods currently available to calculate the conservative, hyperbolic fluxes rely on the owner (own) and neighbor (nei) interpolated values on a face (also called left and right states). These interpolated values are found using a combination of a base interpolation scheme and flux limiters. OpenFOAM currently has a wide selection of available limiters such as `upwind`, `Minmod`, `vanLeer`, `SuperBee`, etc. for scalars, and `upwind`, `MinmodV`, `vanLeerV`, `SuperBeeV`, etc. for vectors; all are run-time selectable.

In addition to the standard OpenFOAM interpolation schemes, MUSCL reconstruction has also been implemented. There are two main options which include `upwindMUSCL` and `quadraticMUSCL`. The upwind version uses standard upwinding for owner and neighbor interpolations. The quadratic version uses limiters in combination with the gradients and the gradient of the gradients (Hessians) to reconstruct face values. All of the standard limiters are available for MUSCL reconstruction. The vector limiters (`MinmodV` for example) are not used with the MUSCL schemes since the limiting is done for each component of the variables.

An example of the *fvSchemes* dictionary is shown below

```
fluxScheme HLLC;

ddtSchemes
{
    default Euler;
    timeIntegrator RK2SSP;
}

gradSchemes
{
```

```

    default cellMDLimited leastSquares 1.0;
}

divSchemes
{
    default none;
}

laplacianSchemes
{
    default Gauss linear corrected;
}

interpolationSchemes
{
    default                linear;        //Base interpolation scheme

    // Limiters
    reconstruct(scalar) Minmod;           // Limiter applied to all scalars
    reconstruct(vector) MinmodV;          // Limiter applied to all vectors

    reconstruct(alpha)  vanLeer;           // Volume fraction
    reconstruct(rho)    upwindMUSCL;       // Density
    reconstruct(p)      linearMUSCL  vanLeer; // Pressure
    reconstruct(U)      quadraticMUSCL vanLeer; // Velocity
    reconstruct(e)      quadraticMUSCL vanLeer; // Internal energy
    reconstruct(speedOfSound) vanLeer;     // Speed of sound
}

```

NOTE: By default, if the OpenFOAM standard reconstruction is used, linear interpolation will be used. The `limitWith` method can instead be used to specify a different interpolation scheme such as `cubic`, but these have proven to be unstable in the past. Please use with caution.

9.1 Riemann Solvers

The currently available Riemann solvers are described below. For all schemes, $\mathbf{n}$ denotes the surface normal vector, and $u = \mathbf{u} \cdot \mathbf{n}$. There is the inclusion of volume fraction in the solution of multi-fluid problems, however with the exception of the AUSM+up flux scheme for granular phases these changes will not be included in the descriptions below.

9.1.1 HLL

The HLL scheme is based on Toro et al. (1994) in which the contact wave is neglected. The fluxes take the form:

$$\mathbf{F}^{HLL} = \begin{cases} \mathbf{F}_{own} & \text{if } 0 \leq S_{own} \\ \frac{S_{nei}\mathbf{F}_{own} - S_{own}\mathbf{F}_{nei} + S_{own}S_{nei}(\mathbf{U}_{nei} - \mathbf{U}_{own})}{S_{nei} - S_{own}} & \text{if } S_{own} \leq 0 \leq S_{nei} \\ \mathbf{F}_{nei} & \text{if } 0 \geq S_{nei} \end{cases} \quad (9.1)$$

The owner and neighbor states to approximate the wave propagation speeds are defined as

$$S_{own} = \min(u_{own} - c_{own}, \tilde{u} - \tilde{c}) \quad (9.2)$$

$$S_{nei} = \max(u_{nei} + c_{nei}, \tilde{u} + \tilde{c}) \quad (9.3)$$

where u_K is the normal flux of the respective state, c_K is the speed of sound, and $\tilde{u}$ and $\tilde{c}$ are the Roe averages velocities and speed of sounds, respectively.

9.1.2 HLLC

The approximate HLLC Riemann was developed by Toro et al. (1994), and improves upon the HLL method by providing an estimation of the contact wave between the owner and neighbor waves. The specific implementation used is taken from Zheng et al. (2011, 2008) This means that the state between the owner and neighbor waves now has two states rather than one. The following fluxes are thus used, using the fact that both pressure and

velocity are constant across the contact wave,

$$\mathbf{F}^{HLLC} = \begin{cases} \mathbf{F}_{own} & \text{if } 0 \leq S_{own} \\ \mathbf{F}_{*own} & \text{if } S_{own} \leq 0 \leq S_* \\ \mathbf{F}_{*nei} & \text{if } S_* \leq 0 \leq S_{nei} \\ \mathbf{F}_{nei} & \text{if } 0 \geq S_{nei} \end{cases} \quad (9.4)$$

The contact wave speed is given by

$$S_* = \frac{\rho_{own} u_{own} (S_{own} - U_{own}) + \rho_{nei} u_{nei} (S_{nei} - U_{nei}) + p_{nei} - p_{own}}{\rho_{own} (S_{own} - u_{own}) - \rho_{nei} (S_{nei} - u_{nei})} \quad (9.5)$$

and the pressure in the

$$p_{*K} = \rho_K (S_K - U_K) (S_* - u_K) + p_K \quad (9.6)$$

The final fluxes are determined by

$$\mathbf{F}_{*K}^{HLLC} = \frac{S_* (S_K \mathbf{U}_K - \mathbf{F}_K) + S_K p_{*K} \mathbf{D}_*}{S_K - S_*} \quad (9.7)$$

$$\mathbf{D}_* = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \mathbf{n} \\ S_* \end{pmatrix} \quad (9.8)$$

9.1.3 HLLCP

Following the work of Xie et al. (2019), a low-speed correction of the standard HLLC flux scheme has been added. Currently it is only available for single phase simulations.

$$\mathbf{F}^{HLLC-P} = \mathbf{F}^{HLLC} + \Phi^P \quad (9.9)$$

$$\Phi_P = (f - 1) \frac{S_{own} S_{nei}}{S_{nei} - S_{own}} \frac{1}{1 + |\tilde{M}|} \frac{p_{nei} - p_{own}}{\tilde{c}^2} \begin{pmatrix} 1 \\ \tilde{\mathbf{u}} \\ \frac{1}{2} \tilde{u}^2 \end{pmatrix} \quad (9.10)$$

$$f = \min \left(\frac{p_{own}}{p_{nei}}, \frac{p_{nei}}{p_{own}} \right)^3 \quad (9.11)$$

where the minimum of all cell faces is used for each cell.

$$p_{**} = \theta p_* + (1 - \theta) \frac{p_{own} + p_{nei}}{2} \quad (9.12)$$

$$\theta = \min(M_*, 1) \quad (9.13)$$

$$p_{***} = f p_{**} + (1 - f) p_* \quad (9.14)$$

where p_{***} replaces p_{*K} in Eq. (9.7)

9.1.4 AUSM+

The AUSM+ scheme was developed by Luo et al. (2004) and constructs the fluxes based on the mass flux across the face.

$$\mathbf{F}^{AUSM+} = 0.5 [M_* c_* (\mathbf{U}_{own} + \mathbf{U}_{nei})] - 0.5 [|M_*| c_* (\mathbf{U}_{own} + \mathbf{U}_{nei})] + \mathbf{F}_p \quad (9.15)$$

$$\mathbf{F}_p = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ p_* \mathbf{n} \\ p_* \end{pmatrix} \quad (9.16)$$

The star state variables are written as

$$c_* = 0.5(c_{own} + c_{nei}) \quad (9.17)$$

$$M_* = \mathcal{M}_{4,own}^+ + \mathcal{M}_{4,nei}^- \quad (9.18)$$

and

$$P_* = \mathcal{P}_{5,own}^+ p_{own} + \mathcal{P}_{5,nei}^- p_{nei} \quad (9.19)$$

where

$$\mathcal{M}_1^\pm(M) = 0.5(M \pm |M|) \quad (9.20)$$

$$\mathcal{M}_2^\pm(M) = \begin{cases} \mathcal{M}_1^\pm(M) & \text{if } |M| \geq 1 \\ \pm 0.25(M \pm 1)^2 & \text{else} \end{cases} \quad (9.21)$$

$$\mathcal{M}_4^\pm(M) = \begin{cases} \mathcal{M}_1^\pm(M) & \text{if } |M| \geq 1 \\ \mathcal{M}_2^\pm(M) [1 \mp 16\beta\mathcal{M}_2^\mp(M)] & \text{else} \end{cases} \quad (9.22)$$

$$\mathcal{P}_5^\pm(M) = \begin{cases} \frac{1}{M}\mathcal{M}_1^\pm(M) & \text{if } |M| \geq 1 \\ \pm\mathcal{M}_2^\pm(M) [(2 \mp M) - 16\alpha M\mathcal{M}_2^\mp(M)] & \text{else} \end{cases} \quad (9.23)$$

with $\alpha = 3/16$ and $\beta = 1/8$.

9.1.5 AUSM+up

The AUSM+ scheme extended to include a low speed correction by Liou (2006). The previous section is used to describe the high speed portion, however additional diffusion terms are added resulting in the following definitions of the mass and pressure fluxes

$$M_* = \mathcal{M}_{4,own}^+ + \mathcal{M}_{4,nei}^- + M_p \quad (9.24)$$

and

$$P_* = \mathcal{P}_{5,own}^+ p_{own} + \mathcal{P}_{5,nei}^- p_{nei} + P_u \quad (9.25)$$

where

$$M_p = -\frac{K_p}{f_a} \max(1 - \sigma \bar{M}a^2, 0) \frac{p_{nei} - p_{own}}{\rho_* c_*^2} \quad (9.26)$$

$$P_u = -K_u \mathcal{P}_{5,own}^+ \mathcal{P}_{5,nei}^- (\rho_{own} + \rho_{nei}) f_a c_* (u_{nei} - u_{own}) \quad (9.27)$$

and $f_a = 1$, $\sigma = 1$, $K_p = 0.25$, $K_u = 0.75$, and α is redefined as $\frac{3}{16}(5f_a - 4)$. The interface density, ρ_* , is defined as $\frac{\rho_{own} + \rho_{nei}}{2}$, and the average Mach number is defined as.

$$\bar{M}a^2 = \sqrt{\frac{u_{own}^2 + u_{nei}^2}{2c_*^2}} \quad (9.28)$$

This correction is very important when considering blasts because the Mach numbers can become much less than unity, resulting in numerical checker-boarding.

9.1.6 AUSM+up (granular)

A modified version of the AUSM+up flux scheme has been developed by Houim and Oran (2016) and includes flux limiters in the event of packing. The flux of conservative variables, (mass, momentum, thermal energy, and granular energy) across a face is given by

$$\mathbf{F}_s^{AUSM+up} = \mathbf{F}_p + \dot{m} \begin{cases} \mathbf{U}_{own} & \text{if } \dot{m} > 0 \\ \mathbf{U}_{nei} & \text{if } \dot{m} \leq 0 \end{cases} \quad (9.29)$$

where

$$\dot{m} = \mathcal{F} c_{1/2} M_{1/2} \begin{cases} \alpha_{own} \rho_{own} & \text{if } M_{1/2} > 0 \\ \alpha_{nei} \rho_{nei} & \text{if } M_{1/2} \leq 0 \end{cases} \quad (9.30)$$

$$\mathcal{F} = \frac{(c_{1/2} - \epsilon)[1 + |M_{1/2}|(1 - \mathcal{G}/2)]}{2} \frac{\max(\alpha_{s,own}, \alpha_{s,nei})}{\alpha_{s,max}} [\alpha_{own} \rho_{own} - \alpha_{nei} \rho_{nei}] \quad (9.31)$$

and $\mathbf{F}_p = \langle 0, P_{tot,1/2} \mathbf{I}, 0, 0 \rangle$, with α_s being the total granular volume fraction (sum of all granular phases), $\alpha_{s,max}$ being the maximum volume fraction at which packing occurs, and ϵ being a small number (1e-10). $\mathcal{G}$ is defined as

$$\mathcal{G} = \max(2(1 - \mathcal{D}\zeta^2)) \quad (9.32)$$

and ζ is defined as

$$\zeta = \begin{cases} \frac{\alpha_s^M - \alpha_{s,crit}}{\alpha_{s,max} - \alpha_{s,crit}} & \text{if } \alpha_s^M > \alpha_{s,crit} \\ 0 & \text{if } \alpha_s^M < \alpha_{s,crit} \end{cases} \quad (9.33)$$

where α_s^M is the maximum granular volume fraction of the own/nei value, and $\alpha_{s,crit}$ being the minimum granular volume fraction that frictional stresses occur.

$c_{1/2}$ is defined as

$$c_{1/2} = \sqrt{\frac{\alpha_{own} \rho_{own} c_{own}^2 + \alpha_{nei} \rho_{nei} c_{nei}^2}{\alpha_{own} \rho_{own} + \alpha_{nei} \rho_{nei}}} + \epsilon \quad (9.34)$$

The split Mach number is defined as

$$M_{1/2} = \mathcal{M}_{4,own}^+ + \mathcal{M}_{4,nei}^- - 2 \frac{K_p}{f_a} \max(1 - \sigma \bar{M}^2, 0) \frac{P_{tot,nei} - P_{tot,own}}{(\alpha_{own}\rho_{own} + \alpha_{nei}\rho_{nei} + \epsilon)c_{1/2}^2} \quad (9.35)$$

where $M_4^\pm$ is calculate the same way as in the AUSM+ scheme and $f_a = 1$. K_p , and σ are defined as

$$K_p = 0.25 + 0.75(1 - \mathcal{G}/2) \quad (9.36)$$

$$\sigma = 0.75\sigma/2 \quad (9.37)$$

The pressure is defined as

$$\begin{aligned} P_{tot,1/2} = & -K_u f_a (c_{1/2} - \epsilon) \mathcal{P}_{5,own}^+ \mathcal{P}_{5,nei}^- (\alpha_{own}\rho_{own} + \alpha_{nei}\rho_{nei})(u_{nei} - u_{own}) \\ & + \mathcal{P}_{5,own}^+ p_{tot,own} + \mathcal{P}_{5,nei}^- p_{tot,nei} \end{aligned} \quad (9.38)$$

with $\mathcal{P}_5^\pm$ being the same as in the AUSM+ scheme and

$$K_u = 0.75 + 0.25(1 - \mathcal{G}/2) \quad (9.39)$$

9.1.7 Tadmor/Kurganov

The Tadmor/Kurganov fluxes were developed by Kurganov and Tadmor (2000). The **rhoCentralFoam** solver (OpenCFD Ltd., 2018a; Greenshields et al., 2010) utilizes this scheme, which has been ported to the current framework. Two run-time selectable options (e.g. **Kurganov** or **Tadmor**) are available; both follow the same general procedure with a slight difference in the calculation of coefficients.

The Kurganov scheme defines

$$a^+ = \max(\max(u_{own} + c_{own}, u_{nei} + c_{nei}), 0) \quad (9.40)$$

$$a^- = \min(\min(u_{own} - c_{own}, u_{nei} - c_{nei}), 0) \quad (9.41)$$

$$a_{own} = \frac{a^+}{a^+ + a^-} \quad (9.42)$$

$$a_{nei} = \frac{a^-}{a^+ + a^-} \quad (9.43)$$

and

$$a = \frac{a^- a^+}{a^+ + a^-} \quad (9.44)$$

The Tadmor scheme defines $a_{own} = a_{nei} = 0.5$ and $a = \max(|a^-|, |a^+|)$.

The volumetric fluxes are written

$$\phi_{own} = u_{own} a_{own} - a \quad (9.45)$$

and

$$\phi_{nei} = u_{nei} a_{nei} + a \quad (9.46)$$

The resulting fluxes are

$$\mathbf{F}^{KT} = (\phi_{own} \mathbf{U}_{own} + \phi_{nei} \mathbf{U}_{nei}) + \mathbf{F}_p \quad (9.47)$$

with

$$\mathbf{F}_p = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (a_{own} p_{own} + a_{nei} p_{nei}) \mathbf{n} \\ a(p_{own} - p_{nei}) \end{pmatrix} \quad (9.48)$$

9.2 Riemann advection scheme

In order to allow for consistency between the conserved quantities (mass, momentum, and energy), and additional transport quantities such as turbulence fields or species mass fractions, an additional advection scheme using the Riemann fluxes has been added. This makes use of the quantities calculated in transport of the conserved variable which then allows for the use of higher order interpolation schemes to be used for additional scalar transport. This is a major advantage over traditional methods which are generally limited to upwinding for additional transport due to presence of strong gradients.

```

divSchemes
{
    default                                none;
    div(alphaRhoPhi.tnt,lambda.tnt)  Riemann;
    div(alphaRhoPhi.tnt,c.tnt)      Riemann;
}

interpolationSchemes
{
    default linear;
    ...

    reconstruct(lambda.tnt)  linearMUSCL vanLeer;
    reconstruct(c.tnt)      linearMUSCL vanLeer;
}

```

In the interpolation schemes sub-dictionary, the limiters should be specified for each field using the Riemann fluxes, but by default the limiter specified by "reconstruct(rho)" will be used.

NOTE: One limitation of using the Riemann fluxes is that, because an explicit advection term for the transported quantities is used, when very stiff implicit terms are present (such as production term in turbulence models), the advection term can become negligible leading to no advection occurring. For this reason, implicit divergence schemes are still recommended for the advection of turbulence quantities and other equations with large implicit source terms.

9.3 Time integration

Higher order time integration has been added allowing for fully third order accurate solutions to the evolution equations. This is a major benefit in comparison to standard OpenFOAM time integration which is at most second order due to the limitation on the implicit time evolution. This functionality applies to both conservative quantities such as mass, momentum, and energy, as well as the activation and afterburn models. The transport of turbulent quantities are still limited by the standard time integration schemes in OpenFOAM. Below are the currently available time integration schemes

and the mathematical steps associated with them. The superscript n denotes the state at the beginning of the current time step, (i) denotes the i -th step, and $n + 1$ denotes final state. The time integration scheme is specified in the *fvSchemes* dictionary using

```
ddtSchemes
{
    default Euler;
    timeIntegrator RK2SSP 3; // Three stages, second order accurate
}
```

NOTE: The strong stability preserving (SSP) methods offer several different variation that allow for a greater number of steps to be used to add additional stability to the solution. This is specified by adding the number of requested steps after the SSP scheme. If a number is provided that is higher or lower than the available number of steps, the scheme will used the closest number of steps.

9.3.1 Euler

Standard first order time integration

$$\mathbf{U}^{n+1} = \mathbf{U}^n - \Delta t \nabla \cdot \mathbf{F}^n \quad (9.49)$$

9.3.2 RK1SSP

First-order, strong-stability-preserving Runge-Kutta method (Spiteri and Ruth, 2002):

One stage:

$$\mathbf{U}^{n+1} = \mathbf{U}^n - \Delta t \nabla \cdot \mathbf{F}^n \quad (9.50)$$

Two stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{2} \Delta t \nabla \cdot \mathbf{F}^n \quad (9.51)$$

$$\mathbf{U}^{n+1} = \frac{1}{2} \mathbf{U}^{(1)} - \frac{1}{2} \Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.52)$$

Three stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^n \quad (9.53)$$

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.54)$$

$$\mathbf{U}^{n+1} = \mathbf{U}^{(2)} - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^{(2)} \quad (9.55)$$

9.3.3 RK2

Standard second-order Runge-Kutta method (mid point):

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^n \quad (9.56)$$

$$\mathbf{U}^{n+1} = \mathbf{U}^n - \Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.57)$$

9.3.4 RK2SSP

Second-order, strong-stability-preserving Runge-Kutta method (Spiteri and Ruuth, 2002):

Two stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \Delta t \nabla \cdot \mathbf{F}^n \quad (9.58)$$

$$\mathbf{U}^{n+1} = \frac{1}{2} (\mathbf{U}^n + \mathbf{U}^{(1)} - \Delta t \nabla \cdot \mathbf{F}^{(1)}) \quad (9.59)$$

Three stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^n \quad (9.60)$$

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.61)$$

$$\mathbf{U}^{n+1} = \frac{1}{3}\mathbf{U}^n + \frac{2}{3}\mathbf{U}^{(1)} - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^{(2)} \quad (9.62)$$

Four stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^n \quad (9.63)$$

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.64)$$

$$\mathbf{U}^{(3)} = \mathbf{U}^{(2)} - \frac{1}{3}\Delta t \nabla \cdot \mathbf{F}^{(2)} \quad (9.65)$$

$$\mathbf{U}^{n+1} = \frac{1}{4}\mathbf{U}^n + \frac{3}{4}\mathbf{U}^{(3)} - \frac{1}{4}\Delta t \nabla \cdot \mathbf{F}^{(3)} \quad (9.66)$$

9.3.5 RK3SSP

Third-order, strong-stability-preserving Runge-Kutta method (Spiteri and Ruuth, 2002):

Three stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \Delta t \nabla \cdot \mathbf{F}^n \quad (9.67)$$

$$\mathbf{U}^{(2)} = \frac{1}{4} (3\mathbf{U}^n + \mathbf{U}^{(1)} - \Delta t \nabla \cdot \mathbf{F}^{(1)}) \quad (9.68)$$

$$\mathbf{U}^{n+1} = \frac{1}{3} (\mathbf{U}^n + 2\mathbf{U}^{(2)} - 2\Delta t \nabla \cdot \mathbf{F}^{(1)}) \quad (9.69)$$

Four stages:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^n \quad (9.70)$$

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.71)$$

$$\mathbf{U}^{(3)} = \frac{2}{3}\mathbf{U}^n + \frac{1}{3}\mathbf{U}^{(2)} - \frac{1}{6}\Delta t \nabla \cdot \mathbf{F}^{(2)} \quad (9.72)$$

$$\mathbf{U}^{n+1} = \mathbf{U}^{(3)} - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^{(3)} \quad (9.73)$$

9.3.6 RK4

Standard fourth-order Runge-Kutta method:

$$\mathbf{U}^{(1)} = \mathbf{U}^n - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^n \quad (9.74)$$

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} - \frac{1}{2}\Delta t \nabla \cdot \mathbf{F}^{(1)} \quad (9.75)$$

$$\mathbf{U}^{(3)} = \mathbf{U}^n - \Delta t \nabla \cdot \mathbf{F}^{(2)} \quad (9.76)$$

$$\mathbf{U}^{n+1} = \mathbf{U}^n - \frac{1}{6}\Delta t \nabla \cdot (\mathbf{F}^n + 2\mathbf{F}^{(1)} + 2\mathbf{F}^{(2)} + \mathbf{F}^{(3)}) \quad (9.77)$$

9.3.7 RK4SSP

Fourth-order, strong-stability-preserving Runge-Kutta method (Spiteri and Ruuth, 2002):

$$\mathbf{U}^{(i)} = \sum_{k=0}^{i-1} (a_{ik} \mathbf{U}^{(k)} + \Delta t \beta_{ik} \nabla \cdot \mathbf{F}^{(k)}) \quad (9.78)$$

Table 1: a_{ik}

0	1			
1	$\frac{649}{1600}$	$\frac{951}{1600}$		
2	$\frac{53989}{2500000}$	$\frac{4806213}{20000000}$	$\frac{23619}{32000}$	
3	$\frac{1}{5}$	$\frac{6127}{30000}$	$\frac{7873}{30000}$	$\frac{1}{3}$
	0	1	2	3

Table 2: β_{ik}

0	1			
1	$\frac{-10890423}{25193600}$	$\frac{5000}{7873}$		
2	$\frac{-102261}{5000000}$	$\frac{-5121}{20000}$	$\frac{7873}{10000}$	
3	$\frac{1}{10}$	$\frac{1}{6}$	0	$\frac{1}{6}$
	0	1	2	3

10 fvMesh Motion Solvers

Modified versions of the standard OpenFOAM displacement motion solvers have been added to enable use of adaptive mesh refinement with mesh motion. The *libblastFvMeshMotionSolvers.so* must be used to enable these models. The available models are:

- `blastDisplacementComponentLaplacian`
- `blastDisplacementLaplacian`
- `blastDisplacementSBRStress`

```
mover
{
    libs
    (
        "libfvMeshMoversMotionSolver.so"
        "libfvMotionSolvers.so"
    );

    type            motionSolver;
    motionSolver    blastDisplacementLaplacian;

    diffusivity     inverseDistance (patchName1 patchName2);
}
```

11 fvMesh Topo-Changers

Topological changes (an change to the definition of cells, faces, or patches) is handled through "topoChangers". Below are the topoChangers available in the *libbbLastFvMeshTopoChangers.so* library.

11.1 fvMeshRefiner

Adaptive mesh refinement can be used by specifying the **fvMeshRefiner** topoChanger. By default the hex/quad refiner is used (can also be specified using the **hexRefiner** topoChanger or refiner). The classes are modified versions of the github repository [SOFTX_2018_143](#) and is based on the work of Rettenmaier et al. (2019). This library introduces 1-D, 2-D, 2-D axisymmetric, and 3-D (present in OpenFOAM-12) all with the option to include dynamic load balancing. This mesh type has been further developed, specifically for compressible flows, and includes modifications for improved stability with meshes created with **snappyHexMesh**. Additional polyhedral refinement has been ported from [foam-extend](#)'s and allows for general polygon refinement (specified by the **polyRefiner** topoChanger or refiner). Both variations automatically include constraints for load balancing when dynamic load balancing is turned on.

Because the detonation of explosives may not occur when the simulation starts, the option to begin unrefinement at a designated time ("beginUnrefine") has been added. This allows the mesh to remain refined in specific regions even though the error is initially small.

NOTE: Whilst AMR may be used with meshes created using **snappyHexMesh**, regions can only be cut at level 0 when using either 2-D or axisymmetric refinement. This is due to the fact that **snappyHexMesh** uses a 3-D refinement, regardless of the number of solution directions. If refinement is used, the simulation will crash when the mesh is refined in these areas.

NOTE: Meshes that have been "snapped" with **snappyHexMesh** may work, however crashes may occur. Use at your own risk.

Entry	Description
protectedPatches	List of patch names that will not be refined or unrefined (default: none)
nRefinementBufferLayers	Number of layers added to refinement (default: 0)
nUnrefinementBufferLayers	Number of layers added to unrefinement (default: 0)
refine	Use refinement (default: true)
unrefine	Use unrefinement (default: true)
refineInterval	Number of time steps between refinement (default: 1)
unrefineInterval	Number of time steps between unrefinement (default: 1)
balanceInterval	Number of time steps between balancing (default: 1)
beginRefine	Time when refinement can begin (default: 0 s)
endRefine	Time when refinement stops (default: great s)
beginUnrefine	Time when unrefinement can begin (default: 0 s)
endUnrefine	Time when unrefinement stops (default: great s)
dumpLevel	Write the cell and point level as viewable fields (default: false)

11.1.1 Error estimators

In order to determine the cells that can be refined or unrefined, error estimators have been added. Generally, this is achieved by looking for a derivative or difference of a specified fields across all the faces a cell and it neighbors. The maximum value of all face errors of a given cell is taken as the cell center-based value, and is used to define the cells that should be refined. Probe locations are automatically refined up to the maximum level of refinement to remove discontinuities in probe sampling.

Entry	Description
lowerRefineLevel	Error that mesh is refined at
unrefineLevel	Error that mesh is unrefined
upperRefineLevel	Error that refinement is stopped (default: great)
upperUnrefineLevel	Error that unrefinement begins to occur again (default: great)
maxRefinement	Maximum number of refinement levels
minDx	Minimum cell size
refineProbes	Are probes refined (default: true)

11.1.1.1 cellSize

Refinement is determined based on the cell size. Either the volume, magnitude of relevant directions (mag), components (cmpt), or the characteristic length (characteristic) can be used. Only the options listed below are used. Because the refinement is based on the cell size the maximumRefinement can be a large number.

Entry	Description
type	size type (volume, mag, cmpt, or characteristic)
minVolume	Minimum volume (for volume)
maxVolume	Maximum volume (for volume)
cmpts	Components (x, y, or z for cmpts)
minDX	Minimum size as a vector (for cmpts)
maxDX	Maximum size as a vector (for cmpts)
minDx	Minimum size (for mag or characteristic)
maxDx	Maximum size (for mag or characteristic)
maxRefinement	Maximum number of refinement levels (default: 10)

11.1.1.2 fieldValue

The error is set to the value of the given field

$$\epsilon_x = x \quad (11.1)$$

where x is a field chosen by the user, and can be any scalar field stored within the simulation.

Entry	Description
field	Error field

11.1.1.3 volumeFraction

When the volume fraction of the given phase is between threshold and 1-threshold, the mesh is refined, otherwise it is able to be unrefined

$$\epsilon_x = x \quad (11.2)$$

where x is a field chosen by the user, and can be any scalar field stored within the simulation.

Entry	Description
phase	Name of the phase
threshold	Threshold value of the volume fraction

11.1.1.4 delta

A basic method to detect sharp changes across faces uses a difference approach, where the error is defined by

$$\epsilon_\delta = |x_{own} - x_{nei}| \quad (11.3)$$

where x is a field chosen by the user, and can be any scalar field stored within the simulation. The subscript own denotes values at the current face, while nei denotes the value at the neighboring face.

Entry	Description
field	Field used to compute error

11.1.1.5 scaledDelta

A basic method to detect sharp changes across faces uses a scaled difference approach, where the error is defined by

$$\epsilon_{\bar{\delta}} = \frac{|x_{own} - x_{nei}|}{\max(\min(|x_{own} - x_c|, |x_{nei} - x_c|), x_{min})} \quad (11.4)$$

where x is a field chosen by the user, and can be any field stored within the simulation. The minimum value (x_{min}) can be specified by the user and is the minimum value that the error will be scaled by.

Entry	Description
field	Field used to compute error
minValue (x_{min})	Minimum value to scale difference by (default: small)
offset (x_c)	Offset value to centre error around (default: 0)

11.1.1.6 gradient

A modified gradient was defined by Zheng et al. (2008), and uses an error defined by

$$\epsilon_{\nabla x} = \max \left(\frac{|(\nabla l_n x)_C - (\nabla l_n x)_{own}|}{\alpha_f x_C / dl + |(\nabla l_n x)_{own}|}, \frac{|(\nabla l_n x)_C - (\nabla l_n x)_{nei}|}{\alpha_f x_C / dl + |(\nabla l_n x)_{nei}|} \right)_f \quad (11.5)$$

with

$$(\nabla l_n x)_C = \frac{x_{nei} - x_{own}}{|\mathbf{r}_{nei} - \mathbf{r}_{own}|} \quad (11.6)$$

$$(\nabla l_n x)_{nei,own} = (\nabla x)_{nei,own} \frac{\mathbf{r}_{nei} - \mathbf{r}_{own}}{|\mathbf{r}_{nei} - \mathbf{r}_{own}|} \quad (11.7)$$

$\mathbf{r}_{own,nei}$ is the surface normal vector, and $dl = |\mathbf{r}_{nei} - \mathbf{r}_{own}|$, is the distance between cell centers. This error estimator uses the total density.

Entry	Description
fieldName	Field used to compute error

11.1.1.7 densityGradient

The **densityGradient** error estimator use the gradient error estimator with the **fieldName** set to "rho" (mixture density).

11.1.1.8 Lohner

A final error estimation method based on Löhner (1987) is also available. It uses a scaled Laplacian to ensure an error bounded between 0 and 1, and indicates the smoothness of the solution. Due to the fact that the method was initially developed for a Cartesian grid, some modifications were required. Namely, the face value of the tracked field is interpolated using a cubic interpolation method. This allows non-zero smoothness across the faces while still allowing for the method to be used on a non-Cartesian grid.

$$\epsilon_{Lohner} = \frac{|x_{own} - 2x_f + x_{nei}|}{|x_{own} - x_f| + |x_{nei} - x_f| + \varepsilon(|x_{own}| + 2|x_f| + |x_{nei}|)} \quad (11.8)$$

where ε is a coefficient which is problem dependent, and x is any scalar field saved within the simulation.

Entry	Description
field	Field name used to compute error
epsilon	Scaling coefficient

11.1.1.9 multiComponent

The multicomponent error estimator allows for multiple errors to be used to define which cells should be refined. Any of the previous methods can be used. A list of names for the error estimators need to be provided, which must correspond to a dictionary with the necessary entries included. The maximum of error at each cell is used for refinement. Each error estimator can use different levels of maximum refinement that are specified in their respective dictionaries.

```
// Error is calculated using the density gradient
errorEstimator multiComponent;
errorEstimators
(
    density
    {
        errorEstimator densityGradient;
        lowerRefineLevel 0.01;
        unrefineLevel    0.01;
        maxRefinement    2;
    }
    pressure
    {
        errorEstimator scaledDelta;
        scaledDeltaField p;
        minValue        1e3;

        lowerRefineLevel 0.1;
        unrefineLevel    0.01;
        maxRefinement    2;
    }
    volumeFraction
    {
        errorEstimator volumeFraction;
        phase          tnt;

        //- Only refine when 0.1 < alpha.tnt < 0.9
        threshold 0.1;
        maxRefinement 1;
    }
}
```

```

    }
);

```

Entry	Description
errorEstimators	List of error estimators to be used

11.1.2 Example

An example of the optional *dynamicMeshDict* for use with `meshRefiner` class

```

topoChanger
{
    libs ("libblastFvMeshTopoChangers.so");

    // type of topoChanger
    type meshRefiner;
    refiner hexRefiner;

    // Error is calculated using the density gradient
    errorEstimator densityGradient;

    // How often to refine
    refineInterval 1;

    // When to begin unrefinement
    beginUnrefine 1e-5;

    // Refine field in between lower..upper
    lowerRefineLevel 1e-2;

    // If value < unrefineLevel unrefine
    unrefineLevel 1e-2;

    // Number of cells between level
    nBufferLayers 1;

    // Refine cells only up to maxRefinement levels

```

```

maxRefinement    4;

// Write the refinement level as a volScalarField
dumpLevel        true;
}

```

11.2 burst

The ability to have a "bursting" patches can be used with the burst topoChangers. The burst conditions are specified using pairs or intact and slave patches (for non-baffles patches) and intactMaster/intactSlave and burstMaster/burstSlave patches (for baffles). Each pair of patches needs uses an associated burst model to determined the burst condition.

Entry	Description
patches	List of patch pairs and associated burstModels

11.2.1 Burst models

The method to calculate where burst occurs is runtime-selectable and the inputs are placed in the *constant/polyMesh/boundary* file under the corresponding boundary. This can also be added in either the patch entry within *blockMeshDict* of *snappyHexMeshDict*.

Variable	Description
intactPatch	Name of the intact patch (for non-coupled)
burstPatch	Name of the burst patch (for non-coupled)
intactMasterPatch	Name of the master-side intact patch (for coupled)
intactSlavePatch	Name of the slave-side intact patch (for coupled)
burstMasterPatch	Name of the master-side burst patch (for coupled)
burstSlavePatch	Name of the slave-side burst patch (for coupled)
partialBurst	Can individual face burst (default: false)
useDelta	Use the difference across coupled patches rather than the maximum value (default: true)
useAverage	Use the average value to determine burst (only for non-partialBurst) (default: false)
regionize	Split patch in to connected regions (only for non-partialBurst) (default: false)

11.2.1.1 pressure

Only pressure is used to determine if burst occurs.

Variable	Description
pName	Name of the pressure field (p by default)
pRef	Reference pressure (0 by default)
pBurst	Pressure at which burst occurs [Pa]

11.2.1.2 impulse

Only impulse is used to determine if burst occurs. Because an impulse field must be available while running the simulation, it is recommended to use the `impulse` functionObject (Sec. 16), otherwise some other method of calculating impulse needs to be used.

Variable	Description
impulseName	Name of the impulse field (impulse by default)
impulseBurst	Impulse at which burst occurs

11.2.1.3 pressureAndImpulse

A combination of the pressure and impulse models are used. If either model results in a burst, then burst occurs. Both sets of inputs are required.

11.2.1.4 field

A list of fields and burst values are used to determine if the patch has burst. The inputs can either be given as a single entry ("burstValue"), multiple burst conditions ("burstValues"), or as a list of names and values ("fields" and "values"). See below for an example.

```
// Model
burstModel field;

// Single Entry
burstValue p 4e7;

// Multiple Entries
burstValues
(
```

```

    p      4e7
    impulse 10000
);

// Multiple Entries, split
fields (p  impulse);
values (4e7 10000);

// Reference values
refValues
(
    p 1e5
);

```

Variable	Description
burstValue	Used for single field burst condition [name value]
refValue	Reference value for single burst condition
burstValues	Used for multiple field burst condition
	[(name1 value1 name2 value2 ... nameN valueN)]
refValues	Used for multiple field burst condition
	[(name1 value1 name2 value2 ... nameN valueN)]
fields	Name of the fields
values	Values at which burst occurs
patchToPatch	Mapping methof

12 fvMesh Distributors

12.1 redistributor

The ability to use dynamic load balancing is available for 2-D, 2-D axisymmetric, and 3-D meshes. Dynamic load balancing can greatly reduce the computational time required by balancing the cells equally across all processors. However, there are known limitations: due to the way OpenFOAM transfers data between processors, the MPI buffer limit can be exceeded for large cases, leading to the simulation crashing. There are also known problems in the transferring of data for calculated non-scalar fields and movingWall boundary conditions. The fields used within the standard **blastFoam** solver have addressed this, but if **adaptiveFvMesh** is used with a different solver, its stability with balancing is not guaranteed.

Variable	Description
balance	Dynamic load balancing enabled? (default: true)
force	Always balance (balanceInterval is 1, maxImbalance is 0) (default: false)
decomposer	Decomposition method (default: specified in <i>decomposeParDict</i>)
balanceInterval	Number of mesh modifications between attempts to balance (default: 1)
beginBalance	Time when balancing begins (default: 0)
endBalance	Time when balancing stops (default: great)
maxImbalance	Maximum imbalance that distribution occurs (default: 0.1)

12.2 Example

An example of the optional distributor dictionary in the *dynamicMeshDict* file.

```
distributor
{
    // Load library
    libs ("libblastFvMeshDistributors.so");

    // Method of redistribution
    type redistributor;

    // Is the mesh dynamically balanced
```

```
balance yes;

// How many refinement steps between rebalancing
balanceInterval 20;

// Allowable difference between average load
// and max load
allowableImbalance 0.2;

// Decomposition method
method scotch;
}
```

13 Numerics

A collection of numerical methods is provided that are used within the models describe prior, and can also be used for new application and solvers. Many of the methods are only listed and the reader should view the provided references for a more complete description.

13.1 Lookup tables

1D, 2D, and 3D lookup tables are provided, all including support for modified data types, runtime selectable interpolation schemes, and the ability to read or construct the necessary data. The name of the given dictionaries and inputs are dictated by the specific models using the tables, "x" and "f" will be used as place holders for these names. If the dictionary named "xCoeffs" is provided "x" is not used to read the given input (i.e. "xMod" is "mod").

When 1D data is read the field can be constructed with 3 methods: directly read, read from a file given a column, or constructed with a spacing.

Variable	Description
file	Name of a file to read from
delim	Delimiter used to seperate data (',' by default)
xCol	Column of the data (first column by default)
x	Data
xMod	Modifier (none by default)
nX	Number of values
minX	Minimum value
maxX	Maximum value
deltaX	Spacing

13.1.1 Modifiers

Different modifiers can be used to interpolate scalar data in different *spaces*, and include

- none
- log10
- pow10 (10^x)
- ln
- exp

Variable	Description
xMod	Modifier for the data

13.1.2 Interpolation schemes

Several interpolation schemes are provided to give the user control over the method which is used to interpolate data including

- floor
- ceil
- linearClamp
- linearExtrapolated
- quadraticClamp
- quadraticExtrapolated
- cubicClamp
- cubicExtrapolated

Clamp methods limit the interpolation to the minimum and maximum values of the provided table, and extrapolated methods will use the closest values to extrapolate the given data out of bounds. Different methods can be used for all directions, and "interpolationScheme" is used to define the default method.

Variable	Description
xInterpolationScheme (interpolationScheme)	Interpolation scheme (linearClamp by default)

13.1.3 1D lookup tables

Data is looked up given an x value. First and second derivatives have been implemented and a reverse lookup can be used for scalar data (find f given x).

13.1.4 2D lookup table

Data is looked up given x and y values. The data can be read from a table or from a list. The default delimiter is ",". The table can also be flipped when reading if the input is reversed (y by x) and a check is done on the size when reading. First and second derivatives (pure and mixed) have been implemented and a reverse lookup can be used for scalar data (find x or y given f and y or x).

13.1.5 3D lookup tables

Data is looked up given x, y, and z values. The data can be read from a table or a list. The default delimiters are "," for columns and ";" for rows. The table can also be flipped when reading if the input is reversed (z by y by x) and a check is done on the size when reading. First and second derivatives (pure and mixed) have been implemented.

13.2 Equations

A simple equation class has been implemented to support many of the numerical methods. The equations take an input and return a result.

Equation: A single scalar input is used to return a single value. This can be used as a univariate or multivariate equation.

Univariate equation: A List of scalar inputs are used to return a single value. This can be used as a multivariate equation.

Multivariate equation: A list of scalar inputs are used to return a list of values.

13.3 Root solvers

A collection of root solvers are provided for finding the roots of univariate, scalar equations. Both the x (absolute and relative) and y (absolute only) values are checked for convergence. The available methods are:

Univariate root solvers:

- | | |
|--------------|-----------------|
| • bisection | • NewtonRaphson |
| • Brent | • secant |
| • falsePoint | • Steffensen |
| • Halley | • step |

Multivariate root solvers:

- | | |
|-------------------------------|-----------------|
| • goodBroyden (Broyden, 1965) | • NewtonRaphson |
| • badBroyden (Kvaalen, 1991) | |

Variable	Description
xTolerance(s)	Relative tolerance for the difference in x (1e-6 by default)
xAbsTolerance(s)	Absolute tolerance for the difference in x (1e-6 by default)
yTolerance(s)	Absolute tolerance for the y value (1e-6 by default)
maxSteps	Maximum number of iteration (100 by default)

13.4 Minimization

A collection of minimization methods are providing for finding the minimums of scalar equations. All methods were implemented using Kochenderfer and Wheeler (2019). The available models are

Univariate minimization:

- bisection
- Fibonacci
- goldenRatio
- NewtonRaphson
- quadraticFit
- ShubertPiyavskii
- step

Multivariate minimization:

- gradientDescent
- NelderMead
- particleSwarm

Nelder-Mead options:

Variable	Description
reflectionCoeff	Scale for reflection (1 by default)
expansionCoeff	Scale for expansion (2 by default)
contractionCoeff	Scale for contraction (0.5 by default)

Particle Swarm options:

Variable	Description
nParticles	Number of particles used (100 by default)
cLocal	Velocity weight toward the best local point
cGlobal	Velocity weight toward the best global point
vWeight	Weight of previous velocity

General options:

Variable	Description
xAbsTolerance(s)	Absolute convergence tolerance for the independent variables (default: 0.0)
xRelTolerance(s)	Relative convergence tolerance for the independent variables (default: 1e-6)
yAbsTolerance(s)	Absolute convergence tolerance for the evaluated function (default: 0.0)
yRelTolerance(s)	Relative convergence tolerance for the evaluated function (default: 1e-6 by)
maxSteps	Maximum number of iteration (default: 100)
nSamples	Number of pre-sample intervals (default: 0)
normaliseError	Is the relative error checked for convergence (default: true)
tau	Line search (and step) reduction coefficient (default: 0.5)

13.5 Integrators

Numerical integrators have been implemented for arbitrary data types (not only scalars).

Univariate integrators (single input) have the ability to use either adaptive integration or a fixed number of intervals. These methods have also been optimised to reduce the number of function evaluations. The available methods are:

Univariate integrators:

- Boole
- Gaussian
- midPoint
- Simpson13
- Simpson38
- trapezoidal

Variable	Description
tolerance	Convergence relative tolerance (default: 1e-6)
absTolerance	Convergence absolute tolerance (default: 0)
adaptive	Is the integration adaptive (default: true)
maxSplits	Maximum number splits per interval (default: 10)
nIntervals	Number of intervals (default: 10)

Because of the complexity of integrating an arbitrary number of dimensions, only a small subset of the univariate integrators have been implemented for multiple inputs. These methods also do not have the same optimisation as seen in the univariate case. The available options are:

Multivariate integrators:

- Gaussian
- trapezoidal
- midPoint

Variable	Description
tolerance	Convergence tolerances (default: 1e-6)
absTolerance	Convergence absolute tolerances (default: 0)
adaptive	Is the integration adaptive (default: true)
maxSplits	Maximum number splits (default: 5)
nIntervals	Number of intervals (default: 5)

14 Compressible systems

All fluids within `blastFoam` are solved via a compressible system. These systems handle the storage of fields, updating of fluxes, and solving of conservative variables. The three primary types are the `single`, `twoPhase`, and `multiphase` models. The selection of the basic model type is done by a check on the number of phases specified in the *physicalProperties* dictionary. For single phase systems, only mass, momentum, and energy are solved. For two phase systems one volume fraction is solved (α_1), two phase masses ($\alpha\rho_1$ and $\alpha\rho_2$), and the mixture momentum and mixture momentum. Finally for multiphase systems, all volume fractions and phase masses are solved along with mixture momentum and mixture energy.

For two and multiphase system there are several options available for how fluxes are computed. `"transportPhaseDensity"` is used to specify that the individual phase density are transported in addition to the phase mass. This can help alleviate problems with stiff equations of that have numerical issues when the phase densities approach low values. `"densityReconstruction"` specified that instead of reconstructing the phase masses, both the volume fractions (always reconstructed) and the phase densities are reconstructed at the faces. Finally, `"MULESLimiting"` can be used to additionally limit volume fraction fluxes to ensure conservation, but is in general is not needed.

Variable	Description
<code>transportPhaseDensity</code>	Transport phase densities (default: false)
<code>densityReconstruction</code>	Reconstruct phase mass using volume fraction and phase density separately (default: false)
<code>MULESLimiting</code>	Limit volume fraction flux with MULES (default: false)

15 Solid models

A port of [solids4foam](#) has been incorporated into the **blastFSI** solver. These models allow for linear and non-linear deformation using the finite-volume methodology described in Cardiff et al. (2018); Cardiff and Demirdzic (2021) and Simo and Hughes (2000) for plasticity.

NOTE: These models are still under development and have not been extensively validated *specifically* for blast.

16 Function objects

In addition to the standard OpenFOAM functionObjects, additional blast specific function objects have been added. The standard function objects from the sampling library are automatically included, but additional libraries can also be included. A list of these libraries can be seen in 19. These can be included using the following *controlDict* entry

```
functions
{
    impulse
    {
        // Additional libraries to include
        libs ("libblastFunctionObject.so");

        // Function object type
        type    impulse;

        // writeTime - Writes every output time
        // timeStep - Number of time steps
        // adjustableRunTime - Specified output interval
        writeControl    writeTime;

        // Interval between writes
        writeInterval 1;

        // Reference pressure
        pRef 101325;

        ...
    }
}
```

16.1 blastProbes

A modified version of the standard OpenFOAM probe utility is also available, where the basic functionality remains the same, with several key additions. First, the ability to append to exiting probe files is now possible, allowing

for a single probe file to be used even if a simulation is restarted from a later time. Second, probes that are not found within the mesh can be moved to the nearest face so they can produce valid results. This can be very useful if a probe is needed on a curved face. Lastly, the warning of probes existing on multiple processors has been removed by selecting a single processor on which to probe.

NOTE: When refinement and unrefinement occurs, the probe locations will "jump" to the nearest cell resulting in fluctuations when plotted. This can be eliminated by allowing refinement of probes in *dynamicMeshDict* ("refineProbes") (this is the default option).

Entry	Description
probeLocations	List of locations to probe
fields	Fields to probe at the given points
append	Are probes appended to and existing file (default: false)
fixLocation	Probes follow cell motion (not stationary)
adjustLocations	Can probes be moved into the mesh (default: false)
writeVTK	Probed points are written to a VTK file (default: false)
interpolationScheme	Method to interpolate probe.
	Options are cell, cellPoint, cellPointFace, cellPatchConstrained, cellPointWallModified, and pointMVC (default: cell)

16.2 fieldMinMax (fieldMax)

The minimum or maximum value at every cell in a domain is output, allowing for quantities such as maximum impulse and peak pressure to be visualized. This can be done for any field currently stored within the simulation. These fields are automatically saved within the calculation.

Entry	Description
fields	Name of the fields (i.e (rho U p impulse))
restartOnRestart	Reset the min/max on simulation restart (default: false)
mode	Type of min/max (component, magnitude, componentMag)
minMax	Minimum or maximum (default: max)

16.3 writeTimeList

Additional write times are added using a list of user specified times. This is useful when fields are needed at time that do not correspond to a standard write interval. In addition to single times, a list of times can be set, either linearly with a perscribed start time, time spacing, and end time, or with a functional spacing given a start time, number of write times in the interval, and end time. Available functions are log, log10, exp, pow10, or linspace (linear). Times are sorted and duplicate times are removed. See [validation/blastFoam/woodwardColella](#) for an example.

Example:

```
functions
{
    times
    {
        // Additional libraries to include
        libs ("libblastFunctionObject.so");

        // Function object type
        type    writeTimeList;

        // Times to write
        times
        (
            0.005
            log10(0.001:3:0.1)
            linspace(0.1:11:1)
            1:1:10
        );
    }
}
```

Entry	Description
times	List of times to write

16.4 conservedQuantities

The volume integrated value of fields can be computed and printed in the output of the solver. The initial, current, total difference, and relative difference is printed. Any fields can be tracked by specifying the names of the fields in the dictionary.

Entry	Description
fields	Fields to track

NOTE: Currently, the removal and injection of values into a domain is not considered.

16.5 vtkTimeSeries surface writer

An extension of the vtk surface writer for writing sampled surfaces can be used so that an additional **.vtk.series* file is written in the *postProcessing/surfacesName* folder with the relative paths to all write times of the given surfaces. This file can then be opened in **ParaView** to visualize all files in the correct time sequence.

16.6 Schlieren (grad)

The gradient of the given field is calculated and written. Only scalar and vector fields can be used.

Entry	Description
field	Fields to take the gradient of
result	Name of the result field (default: Schlieren/grad(field))

16.7 shadowgrap (laplacian)

The Laplacian of the given field is calculated and written

Entry	Description
field	Fields to take the Laplacian of
result	Name of the result field (default: shadowgraph/Laplacian(field))

16.8 blastMachNo

The **blastMachNo** replaces the standard **machNo** available in OpenFOAM by using the speed of sound calculated from the blastFoam specific thermodynamic models.

Entry	Description
phaseName	Name of the phase (default: none)
UName	Name of the velocity field (default: U or U.phaseName)
result	Name of the Mach number field (default: Ma or Ma.phaseName)

16.9 dynamicPressure

It can be useful in blast simulations to track the dynamic pressure (kinetic energy) of a phase defined as

$$\mathbf{p}_{dyn} = \frac{1}{2} \rho_i \mathbf{U} |\mathbf{U}|. \quad (16.1)$$

The **dynamicPressure** function object returns the vector of dynamic pressure, where the magnitude of this output is the standard dynamic pressure. Additionally the **pressure** function object (standard OpenFOAM) can be used to calculate static (if using incompressible) or total pressure and their respective coefficients.

Entry	Description
rhoName	Name of the density field (default: rho)
UName	Name of the velocity field (default: U)

16.10 overpressure

This function object calculates and writes the overpressure (e.g. the pressure relative to a given reference pressure, usually ambient) given a pressure field and reference pressure. The field can also be saved to allow for additional sampling. If wanted, a non-uniform pressure field can be used by specifying "nonUniformPRef". By default the pressure field at the start of the simulation will be used, this can be replaced by creating a new field, p0.

Entry	Description
pName	Name of the pressure field (default: p)
pRef	Reference pressure [Pa]
nonUniformPRef	Is a non-uniform reference pressure used (default: false)
store	Is the field saved (default: false)

16.11 impulse

This function object calculates the impulse in every cell by integrating the overpressure over time using the specified pressure field and reference pressure. The impulse field is automatically saved to allow additional sampling by default. If wanted, a non-uniform pressure field can be used by specifying "nonUniformPRef". By default the pressure field at the start of the simulation will be used, this can be replaced by creating a new field, p0.

Entry	Description
pName	Name of the pressure field (default: p)
pRef	Reference pressure value [Pa]
nonUniformPRef	Is a non-uniform reference pressure used (default: false)

16.12 timeOfArrival

This function object tracks the maximum pressure and records the time when the maximum pressure is reached.

Entry	Description
pName	Name of the pressure field (default: p)

16.13 blastSurfaces

This function object is a modified version of the **surfaces** available in standard OpenFOAM and includes the ability to sample point fields.

16.14 stopProbes

This function object tracks field values at a list of locations and stops the simulation when any of the given criteria is met. This is useful when a lower

dimensional simulation will be mapped to a higher dimension simulations (aka 1D -> 3D)

Entry	Description
locations	List of locations to track
fields	List of field names and stop values
stopAt	How the simulation is stopped (default: next write time)

16.15 TExposure

This function object calculates the "exposure" given local temperature based on either the CEM43 model Sapareto and Dewey (1984), or model of Henriques and Moritz (1947). If the `Henriques` model is used coefficients can be used from Henriques and Moritz (1947), Stoll and Greene (1959), Takata et al. (1977), Mehta (1973) which requires what skin layer is used (epidermis or dermis), or `custom` with uses a user defined set of coefficients.

Entry	Description
restartOnRestart	Reset the min/max on simulation restart (default: false)
burnModel	Burn model to use (Henriques or CEM43)
source	Source of coefficients (Henriques, WeaverStoll, Takata, MehtaWong, or custom) (default: Henriques)
skinLayer	Skin layer (epidermis or dermis) (only for MehtaWong)
preExponential	For custom coefficients
deltaEbyR	For custom coefficients
TSwitch	For custom coefficients
Tmin	For custom coefficients
TMax	For custom coefficients

16.16 thermalImpulse

This function object calculates thermal impulse by integrating the temperature gradient and radiation heat flux. The thermal diffusivity is not included so post processing is required to obtain the true integrated heat flux.

Entry	Description
restartOnRestart	Reset the min/max on simulation restart (default: false)
TName	Temperature field name (default: T)
qrName	Name of radiation heat flux field (default: qr)

17 Utilities

Listed below are the utilities that have been developed for `blastFoam` to simplify case setup and post-processing.

17.1 `setRefinedFields`

This is a modified version of the `setFields` utility in the standard OpenFOAM that includes the ability to refine the mesh based on the fields that are set. This is extremely beneficial when a small volume needs to be set within a set of very large computational cells. The same functionality presented in Sec. 11.1 is present, allowing for 2D and 3D geometries to be set and refined, and the refinement options are specified in *setFieldsDict*, not *dynamicMeshDict*. The utility works by setting the default field values provided, setting the requested fields, then checking for local variations in specific fields. Region may have a level defined where the mesh is refined up to this level. The default field values for any fields being set must also be defined so that the region is updated at each iteration. The utility will iterate until the mesh is no longer changing, or the maximum number of iterations is reached. A comparison using the standard `setFields` and `setRefinedFields` can be seen in Fig. 1.

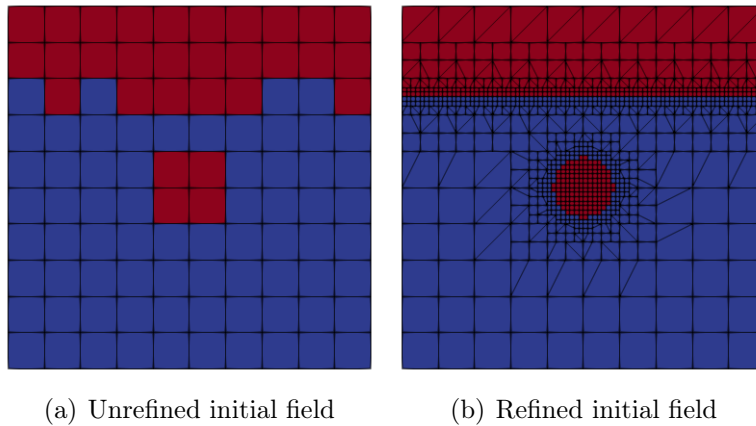


Figure 1: Comparison of initial fields using `setFields` and `setRefinedFields`

By default, no error estimator will be used so any local change in a tracked

field will be refined. However, by specifying an error estimator, the refinement will be determined by the method requested.

By default only fields that are specified within *setFieldsDict* will be modified, meaning that all other fields must use uniform field values to be read. The '-updateAll' command line option will read in all fields present in the current time step and update the sizes of the fields which is necessary if non-uniform fields will be used. If balancing is used this will automatically be enabled to ensure that all fields are correct.

In addition to the standard shapes that can be set with **setFields**, several additional options have been added and are listed below with their required entries. All ***MassToCell** options require "rho" and "mass" to be specified.

Usage: setRefinedFields [OPTIONS]

New cell sets:

- sphereListToCell: centres, radii, or radius
- cylinderCellToList: p1s, p2s, radii, or radius
- boxMassToCell: centre and aspectRatio (span for backup)
- cylindricalMassToCell: centre, LbyD and direction (radius and L for backup)
- sphericalMassToCell: centre (radius for backup)
- hemisphericalMassToCell: centre (radius for backup)
- searchableSurfaceToCell: surface (searchable surface type)
- searchableSurfaceToPoint: surface (searchable surface type)

The option to specify non-uniform values for the fields has also been included and are specified by using the field type (i.e. volScalarField, pointVectorField, etc.) followed by the type (i.e. Value, Density, etc.), for example pointScalarFieldValue, volVectorFieldDensity, etc.

Field set types:

- Density: A density is calculated using the given volume integrated quantity, and dividing by the volume of the selected cells

- Function: Values are set based on the specified function.
- InitialValue: The current field is read and saved to ensure the true values are used after refinement.
- MassIntegrated: The mass integrated quantity read and is divided by the mass (density times volume) of the given cell set to determine the field value.
- Sum: The field value is calculated given a summed value and the list of fields to use in the sum
- Value: A uniform value is set

Command line options:

Entry	Description
-forceHex8	Force 3D refinement for compatibility with standard OpenFOAM solvers
-debug	Write intermediate step with error and max cell level
-updateAll	Resize all fields from the current time
-overwrite	Write the mesh to previous location
-noFields	Do not write fields
-noRefine	Do not refine the mesh
-noHistory	Do not write the refinement history (fix the mesh)

Dictionary options:

Entry	Description
fields	Field names that are used to compute error and refine mesh
level	Level of each cell set
refineInternal	Are the cells inside each region refined
refineFaces	Are the owner and neighbor cells of faces refine (requires selectionMode)
refinePoints	Cells containing the points refined (requires selectionMode)
nBufferLayer	Number of cells between level boundaries (default: 0)
maxIter	Maximum number of iterations (default: 2*max(level))
errorEstimator	Type of error estimator used (default: none)

Field options:

Entry	Description
-setBoundaries ("patches")	Set the boundaries provided
-setAllBoundaries	Set all boundary values
-noInternal	Do not set cell values
-evaluateBoundaries	Correct boundary conditions after setting

Selection modes:

Entry	Description
-all	Select all entries
-internal	Select all entries not on boundaries
-interface	Selected entries between selected and not
-boundary	Select entries on boundaries in the 'patches' entry
-interfaceAndBoundary	Select entries using 'interface' and 'boundary'

Set and zone creation: The functionality of the **topoSet** utility is available in *setRefinedFields* allowing for creation of sets and zones using the refinement shapes. cell/face/point sets/zones are specified in separate lists, and multiple *actions* can be performed on the given sets. The usage is identical to that of **topoSet** and the *tutorials/setRefinedFields* case will offer an extensive list of usage options.

An example *setFieldsDict* is shown below

```
fields (alpha.air);
nBufferLayers 1;

defaultFieldValues
(
    volVectorFieldInitialValue U ( 0 0 0 )
    volScalarFieldFunction p
    {
        // Function3 (i.e. uses x, y, and z)
        type coded;
        code
        #{
            return -x + 2.0*y*z
        };
    }
)
```

```

volScalarFieldValue alpha.air 0

// alpha.water = 1 - (alpha.air + alpha.gas)
volScalarFieldSum alpha.water 1 (alpha.air alpha.gas)
volScalarFieldValue rho.air 1.225
);

regions
(
    cylinderToCell
    {
        p1 (0 0 -1);
        p2 (0 0 1);
        radius 0.1;
        level 3;
        refineInternal yes;
        backup
        {
            p1 (0 0 -1);
            p2 (0 0 1);
            radius 0.2;
        }

        fieldValues
        (
            volScalarFieldValue p 9.12e8

            // alpha.air = 10 [kg] / sum(V * rho.air) [kg]
            volScalarFieldMassIntegrated alpha.air 10

            // rho.water = 500 [kg]/ Volume
            volScalarFieldDensity rho.water 500.0
        );
    }

    // Refine the faces in the "walls" patch
    patchToFace

```

```

{
    name walls;
    level 2;
    refineFaces yes;
    selectionMode boundary;
}
);

```

The tutorial in [tutorials/setRefinedField](#) details how this can be used as well as the outputs of the '-debug' option.

NOTE: Because OpenFOAM does not decompose both the time-based mesh (e.g. *0, 0.001, ...*) and the base mesh (located in *constant*), where both are required to use the mesh modified by **setRefinedFields**. Thus, if it is run in parallel, **setRefinedFields** must be run in parallel after the case has been decomposed.

17.2 blastToVTK

In order to create smaller files for post-processing, **blastToVTK** can be used to create VTK files of a given patch with a set of fields. By Default the write interval will be the write interval of the VTK files, but optionally the '-dt' flag can be used to write interpolated, intermediate steps at the given interval. A **.vtk.series* file will automatically be written for simplified viewing. By default the VTK files will be written in binary.

Usage: blastToVTK <patchName> [OPTIONS]

Command line options:

Entry	Description
-ascii	Write ascii VTK files
-dt	Optional write interval
-interpolationScheme	Scheme used to interpolate between time steps (default: linearClamp)
-fields	List of fields to be written in the VTK files
-outputDir	Directory to write files to (default: VTK)

17.3 createTimeSeries

This utility simplifies the previous utility by simply creating a **.*.series* file which provides the path to each time step of a VTK or STL file and allows viewing of the time series with the time in **ParaView**. Only one additional file is created for each sampled field in the directory provided. The given path must include files within time step folders. By default the relative path to the files will be used

Usage: createTimeSeries <path/to/timeFolders> [OPTIONS]

Command line options:

Entry	Description
-absolute	Use absolute file paths
-outputDir	Directory to write the *.series file to

17.4 mergeProbes

This is a simple utility that can be used to calculate combine probes started at different times. By default, all files found in the start time folder will be merged.

Usage: mergeProbes <path/to/probes> [OPTIONS]

Command line options:

Entry	Description
-force	Overwrite existing merged probes
-probeNames	Probed fields to merge (default: all)

17.5 calculateImpulse

This is a simple utility that can be used to calculate the impulse using a pressure probe file. If an overpressure file is found, that will be the default field to integrate, but this can be overridden with the '-p' option. If an overpressure file is provided the reference pressure will be 0 by default, other with '-pRef' should be provided.

Usage: calculateImpulse <path/to/probes> [OPTIONS]

Command line options:

Entry	Description
-p	Name of pressure field
-pRef	Reference pressure [Pa]
-impulse	Name of file to write
-force	Overwrite existing impulse file

17.6 initializeAtmosphere

The **initializeAtmosphere** utility is used to initialize stable pressure and density fields such that the hydrostatic pressure gradient and gravitational acceleration force are balanced. The hydrostatic initialization procedure is a modified version of that available in OpenFOAM (OpenCFD Ltd., 2018a). Currently, there are two available models: hydrostatic and tabulated. The hydrostatic model dynamically solves for the hydrostatic pressure gradient and adjusts the density accordingly. The tabulated model reads a list of pressures and temperatures versus height to determine the density fields. An optional hydrostatic correction can be used once the table values have been set. See [tutorials/initializeAtmosphere](#) for an example.

Inputs can either be read from a dictionary or through the command line. If the tabulated option is requested, [system/atmosphereProperties](#) must be available.

NOTE: If there are no pressure boundaries that fix the pressure, a reference state must be specified by setting "pRefValue" and either specifying a cell index ("pRefCell") or point in the mesh ("pRefPoint").

Usage: initializeAtmosphere [OPTIONS]

Command line options:

Entry	Description
-hRef	Reference height, or the height at the lowest point of the mesh [m]
-phase	Name of the phase to set (only for multiphase)
-zone	Optional cellZone to set (default: all cells)
-type	Optional override of the model. (default: hydrostatic if not read from dict)
-pRefValue	
-pRefCell	
-pRefPoint	
	Reference pressure [Pa]
	Reference cell index
	Reference point

17.7 rotateFields

The **rotateFields** utility can be used to map the fields of a 1-D or 2-D solution to that of a 2-D or 3-D solution, where the source and target meshes are used to determine the rotation. The rotation of vector and tensor fields are mapped using the local rotation tensor to ensure that the correct radial components are used. Only field present in the target case are mapped, however additional fields can be mapped. Because there are no values for these fields, the closest value on the source mesh is used for cells outside of the mapped cells. Iterative refinement can be used to iteratively rotate fields and refine the mesh using the refinement criteria found in *dynamicMeshDict* allowing for easier initialization when using AMR.

NOTE: The mapping is not conservative currently so only the nearest cell value will be used which can result in certain flow features either being over or under emphasised.

Usage: rotateFields <sourceCase> [OPTIONS]

Command line options:

Entry	Description
-sourceTime	Time in the source case to map (default: startFrom time in source case)
-sourceRegion	Region of the source case to map (only used for multi-region)
-targetRegion	Region of the target case to map to (only used for multi-region)
-extend	If a cell is outside of the source range, map to the closest cell (default: false)
-maxR	Cutoff radius to map (default: great)
-centre	Center of rotation on the source mesh (default: center of the source mesh)
-uniform	Copy uniform fields to the target case (default: false)
-additionalFields	List of additional fields to map
-refine	Iteratively map, rotate and refine the mesh
-parallelSource	Map from a parallel source
-tets	Use cell tet decomposition (default: false)

17.8 rotateConservativeFields

This utility acts the same as **rotateFields** with the difference that all conservative fields are read, mapped then decoded so that conservation of mass, momentum and energy should be better since mapped pressures and temperatures will not necessarily correspond to the same mapped density and total energy.

NOTE: This utility can only be used with a serial source case.

Usage: rotateConservativeFields <sourceCase> [OPTIONS]

17.9 convertLagrangianPositions

Because OpenFOAM has changed the write format of Lagrangian particles to barycentric coordinated in place of a point in physical space, **ParaView** is no longer able to read the positions of particles any more. To alleviate this problem, and allow **ParaView** to read Lagrangian fields (in serial and

parallel), the **convertLagrangianPositions** will read in the standard Open-FOAM positions file, convert it to a format that **ParaView** can read and rewrite the *positions* file. If the positions need to be reverted to continue a simulation, the '-revert' optional will rewrite the original positions of the particles.

Usage: convertLagrangianPositions [OPTIONS]

Command line options:

Entry	Description
-revert	Convert point positions back to original positions

17.10 blastPruneTimes

The **blastPruneTimes** utility is a modified version of the **foamListTimes** with the additional ability to select times with a given time spacing.

Usage: blastPruneTimes [OPTIONS]

Command line options:

Entry	Description
-processor	Use decomposed times
-rm	Remove selected times
-withFunctionObjects	Use function objects
-startTime	Start of time interval (default: startTime)
-endTime	End of time interval (default: latest time)
-dt	Time spacing
-n	Number of times to keep between start and end
-tolerance	Tolerance to of time selection (default: minimum of differences between times * 1e-3)
-invert	Select non-interval times
-Clean	Remove times not in processor0 folder (for parallel cases)

17.11 addInternalPatch

The **addInternalPatch** utility creates an **internal** patch named internal-Patch that is used for balancing. The mesh is overwritten by default.

Usage: addInternalPatch [OPTIONS]

Command line options:

Entry	Description
-noFields	Do not modify field files
-increment	Do not overwrite mesh, write to time + dt folder
-first	Add internal patch as the first patch (default: after last non-processor patch)
-patchName	Name of the new patch (default: internalPatch)

17.12 addEmptyPatch

The **addEmptyPatch** utility creates new patch with the given name and type with 0 size. The mesh is not overwritten by default

Usage: addEmptyPatch <patchName> <patchType> [OPTIONS]

Command line options:

Entry	Description
-noFields	Do not modify field files
-overwrite	Overwrite mesh
-first	Add internal patch as the first patch (default: after last non-processor patch)

17.13 blastConvertCaseFiles

The **blastConvertCaseFiles** utility converts case files from OpenFOAM-9 to OpenFOAM-OpenFOAM-12 format. For single phase cases (i.e. **blastFoam**) *phaseProperties* is renamed to *physicalProperties* and for multiphase cases (i.e. **blastEulerFoam**) *phaseProperties* is kept and the thermodynamic entries for each phase are split into *physicalProperties.phaseName* files. For both, if a *turbulenceProperties* file exists, it is renamed *momentumTransport*. Finally, if *dynamicMeshDict* is present and the type is **adaptiveFvMesh**, **movingAdaptiveMesh**, or **dynamicMotionSolverFvMesh** is used, new dictionaries are added: "mover" for mesh movement (if the **movingAdaptiveMesh** or **dynamicMotionSolverFvMesh** was used), "topoChanger" for mesh refinement (if **adaptiveFvMesh** or **movingAdaptiveMesh** was used), and "distributor" for dynamic load balancing (if balancing was enabled). If files would be overwritten, they are copied to *fileName_old*

Usage: blastConvertCaseFiles [OPTIONS]

Command line options:

Entry	Description
-solver	Solver name (blastFoam or blastEulerFoam) ((default: blastFoam))
-multiphase	Sets solver as blastEulerFoam

18 fvModels and fvConstraints

In addition to the physics present in the standard solver, additional source terms and models can be included by adding the optional *constant/fvModels* and *system/fvConstraints*. All of these models must have their associated libraries included within the given entries in the files. A list of the libraries and a brief description can be found in Sec. 19

19 Optional libraries

Below is a list of optional libraries to include for functionObjects, fvMeshMovers, fvTopoChangers, fvMeshDistributors, fvModels, and fvConstraints. When libraries are included additional options will become available in the list of runtime-selectable models.

- "libutilityFunctionObjects.so": General utilities
- "libfieldFunctionObjects.so": Function objects used for field quantities
- "liblagrangianFunctionObjects.so": Simple lagrangian particles
- "libforces.so": Calculate viscous and pressure forces acting on boundaries
- "libfvMeshMoversInterpolator.so": Motion interpolation between known meshes
- "libfvMotionSolvers.so": Motion solvers
- "libfvMeshMoversMotionSolver.so": General motion solvers
- "libfvMeshTopoChangers.so": Standard OpenFOAM refinement and meshToMesh topoChangers
- "libfvMeshDistributorss.so": Standard OpenFOAM distributors, includes (loadBlancer time based, i.e. chemistry) and distributor (number of cells).
- "libfvConstraints.so": General fvConstraints
- "libfvModels.so": General fvModels
- "liblagrangianParcel.so": Coupled lagrangian particles (one way coupling)
- "libblastMotionSolvers.so": Motion solver that have additional support for AMR and load balancing
- "libblastFvTopoChangers.so": blastFoam specific topoChangers including hex, quad, and polyhedral refinement, and the **burst** topoChanger.
- "libblastFvMeshDistributors.so": BlastFoam specific load balancer (**redistributor**)

20 Solvers

blastFoam offers a wide variety of solvers for simulating transient, highly compressible flows. A short description of each is given below.

20.1 blastFoam

The **blastFoam** solver is the standard solver and has a variety of uses, including single phase, two phase and multiphase flows, where the phases and thermodynamic models are specified in the *constant/phaseProperties*. When a list of phases is provided (and the list has more than one entry), the two phase or multiphase solver is selected. If the key word phases is not specified, then the single phase solver is selected. All phases use the blastFoam specific equations of states (see Section 5).

20.2 blastEulerFoam

The **blastEulerFoam** solver is the Euler-Euler (two/multifluid) variant of **blastFoam**. Currently only fluid-solid simulations are supported; however, the addition of fluid-fluid simulations is under development. This solver is based on the work of Houim and Oran (2016) and Lai et al. (2018). One major advantage over the standard **multiphaseEulerFoam** solver is that any number of particle phases can be used. Currently there are three phase models that can be used: fluid, multiphase, and granular. The fluid phase model is used to describe a single component fluid (such as air). The multiphase phase model is used to describe a fluid phase consisting of multiple phases, each described by a unique equation of state. This allows simulations of standard detonation problems with the addition of particles. The granular phase model is used to describe a solid phase. Both the fluid and multiphase models can use any of the fluid thermodynamic models, while the granular model can use any of the solid thermodynamic models. The models describing interactions between fluids are described in Sec. 3 and Sec. 4

NOTE: While multiple fluid phases will not cause the solver to fail, theses types of simulations are not currently supported.

20.3 **blastFSIFoam**

The **blastFSIFoam** solver uses the standard **blastFoam** solver to solve the fluid phase, and the models mentioned in Sec. 15 to handle the solid deformation. The fluid phase uses a moving mesh, while the solid phase can be solved using a total lagrangian or linear (stationary), or updated Lagrangian (moving) method. In order to correctly transfer the information between the patches, the **coupledGlobalPolyPatch** Cardiff et al. (2018) is used and does not require any changed to the mesh patch types. Fields that require coupling information (temperature, displacement, etc.) should use a global patch type (see Sec. 7.1). If meshes are made for each individual region, the patch types can be specified as usual. However, if the **splitMeshRegions** utility is used to separate a single region mesh into multiple region meshes, the patch names and types may need to be modified

NOTE: This solver is still in development. Please use with caution.

20.4 **blastXiFoam (Experimental)**

In addition to the standard **blastFoam** solver, an additional solver has been added to solve combustion as an extension of the standard OpenFOAM **XiFoam** solver. The only difference is that the flux schemes presented earlier are used to transport the conserved quantities. This allows for a more accurate description of combustions, highly compressible flows (i.e. deflagration to detonation transition). It is recommended to use the **adibaticFlameT** utility to calculate the model coefficients used in the required *constant/thermophysicalProperties* dictionary.

20.5 **blastReactingFoam (Experimental)**

blastReactingFoam uses the standard OpenFOAM thermodynamic classes, and can be used to solve multi-specie systems that include reactions. The use of combustion models are also available. The *constant/thermophysicalProperties* dictionary must be included under the constant dictionary along with the relevant dictionaries (*constant/combustionProperties* and *constant/chemistryProperties*) and initial value files (mass fractions). The conserved quantities (mass, momentum, and energy) as well as the mass fractions are all transported using the flux schemes presented herein.

20.6 **blastMultiRegionFoam** (Experimental)

The **blastMultiRegionFoam** solver is derived from the **chtMultiRegionFoam** available in OpenFOAM. The major differences are in the use of ODE time integration and Riemann fluxes. The thermal transport in solids is still treated implicitly through a matrix solution.

20.7 **blastParcelFoam** (Experimental)

blastParcelFoam makes use of the standard OpenFOAM Lagrangian classes to solve a coupled Eulerian fluid and collection of Lagrangian parcels. This includes interaction terms like drag, heat transfer, and particle collisions. Not all of the OpenFOAM standard Lagrangian options are available due to the use of a different thermodynamic model. While AMR can be used, dynamic load balancing cannot.

20.8 **blastEquation**

blastEquation makes use of the numerics library and can be used to interrogate simple equations, or more complicated, coded function evaluations. This utility can either be used strictly through the command line, or using dictionaries to define equations and the operations they are used with. More control is available when using dictionaries, specifically with respect to root solving, minimization, and integration schemes. Please see [tutorials/blastEquation](#) for more complete examples on how to use this utility.

Entry	Description
-func	Dictionary used to define the equation
-dict	Dictionary used to define operations on the equation
-clean	Remove dynamicCode library
-noClean	Do not remove dynamicCode library
-name	Name of the equation
-fx "pow(x, 3)" "x[0]*x[1]"	Function
-dfdx "3*sqr(x)" "x[1]; x[0]"	First derivative of function (entries seperated by ";" for multiple inputs)
-d2fdx2 "6*x"	Second derivative of function (only for single inputs)
-d3fdx3 "6"	Third derivative of function (only for single inputs)
-P "(c b a)"	Polynomial coefficient ($a + bx + cx^2$)
-eval "x0" or "(x0 ... xn)"	Evaluate at a list of points
-findRoots "(x0 x1 x)"	Find roots between x0 and x1, optionally starting at x
-findAllRoots "(x0 x1)"	Find all roots between x0 and x1
-rootSolver	Root solver
-findPolyRoots	Find roots of a polynomial (only valid with "P")
-minimize "(x0 x1 x)"	Minimize function with bounds x0 and x1, optionally starting at x
-minimizer	Method of minimization
-integrate "(x0 x1)"	Integate between x0 and x1
-integrator	Integration method
-time	Use time (only for cases using meshes)

NOTE: When using command line options, all function definitions and list entries should be in quotes.

NOTE: When multiple inputs are used, "x0" becomes "(x00 ... x0m)", and applies to all entries that take points/positions.

21 Examples

Several validation and tutorial cases are presented to demonstrate the approach for setting up typical problems. Other validation and tutorial cases are available in the *validation* and *tutorials* directories, which can be found in the top level directory where `blastFoam` is installed. The cases presented here are

1. A single-phase, double Mach reflection. Located in *validation/blastFoam/doubleMachReflection*
2. A 1-D shock tube consisting of air and water. Located in *validation/blastFoam/shockTube_twoFluid*
3. A 2-D, three-phase case with two different detonating phases where the first explosive material is used to detonate the second charge using the pressure based activation model. Located in *tutorials/blastFoam/twoChargeDetonation*
4. An explosive charge of C-4 buried in sand. Located in *tutorials/blastEulerFoam/mine*
5. A charge causing the breakage of windows using burst patches. Located in *tutorials/blastFoam/burstingWindow_workshop*

Only modified, relevant input files are shown here, as all additional files (such as *blockMeshDict* and initial conditions) are contained in the respective directory for each case.

21.1 Double Mach reflection

The double Mach reflection problem is a complex case based on Woodward and Colella (1984), which is meant to demonstrate the capabilities of the single phase solver within **blastFoam**. Coded boundary conditions for velocity, pressure, and density are used to correctly describe the shock behavior at the top boundary. The case uses the a single phase with the RK2-SSP ODE time integrator and HLLC flux scheme.

For the double Mach reflection, the boundary conditions must be set so that the fields on the top boundary follow the correct upstream and downstream shock conditions, which are

$$\rho_R = 1.4 \text{ kg/m}^3, \quad p_R = 1 \text{ Pa}, \quad T_R = 1 \text{ [K]}, \quad \mathbf{U}_R = (0, 0, 0) \text{ m/s}$$

$$\rho_L = 8 \text{ kg/m}^3, \quad p_L = 116.5 \text{ Pa}, \quad T_L = 20.3887 \text{ [K]}, \quad \mathbf{U}_L = (7.1447, -4.125, 0) \text{ m/s}$$

These conditions change based on the shock location, leading to a modified boundary condition for density, pressure, temperature, and velocity, which uses a `codedFixedValue`. The initial density field is defined using

```
dimensions      [1 -3 0 0 0 0 0];

internalField   uniform 1.4;

boundaryField
{
    inlet1
    {
        type      fixedValue;
        value      $internalField;
    }
    inlet2
    {
        type      fixedValue;
        value      $internalField;
    }
    outlet
    {
        type      codedFixedValue;
        value      $internalField;
        name      shockRho;
        code
        #{
            // Shock location (x)
            scalar xS =
                db().time().value()*11.2799
```

```

        + 0.74402;
    scalarField x =
        this->patch().Cf().component(0);

    operator==
    (
        pos(x - xS)*1.4
        + neg0(x-xS)*8.0
    );
    #};
}
walls
{
    type          zeroGradient;
}

defaultFaces
{
    type          empty;
}
}

```

The velocity and temperature fields are initialized using the upstream and downstream values with the same evolution equations. The pressure field uses **zeroGradient** boundary conditions. The internal and boundaries are set using **setFields** with the *setFieldsDict* defined as

```

defaultFieldValues
(
    volVectorFieldValue U (7.1447 -4.125 0)
    volScalarFieldValue T 20.3887
    volScalarFieldValue p 116.5
    volScalarFieldValue rho 8.0
);

regions
(
    rotatedBoxToCell

```

```

{
    origin (0.16667 0 -1);
    i (8.66025 -5.0 0);
    j (5.0 8.66025 0 );
    k (0 0 2);
    fieldValues
    (
        volScalarFieldValue p 1
        volScalarFieldValue T 1
        volVectorFieldValue U (0 0 0)
        volScalarFieldValue rho 1.4
    );
}
boxToFace
{
    box (0.74402 0.999 -1) (5 1.001 1);
    fieldValues
    (
        volScalarFieldValue p 1
        volScalarFieldValue T 1
        volVectorFieldValue U (0 0 0)
        volScalarFieldValue rho 1.4
    );
}
);

```

The *phaseProperties* file is

```

mixture
{
    type          basic;
    thermoType
    {
        transport    const;
        thermo        eConst;
        equationOfState idealGas;
    }
    specie

```

```

{
    molWeight    11640.3; // normalizes pressure
}
transport
{
    mu           0;
    Pr           1.0;
}
equationOfState
{
    gamma        1.4;
}
thermo
{
    Cv           1.785714286;
    Hf           0.0;
}
}

```

The case can be initialized and run using the following commands:

```

blockMesh
setFields
blastFoam

```

The final results are shown in the following figure.

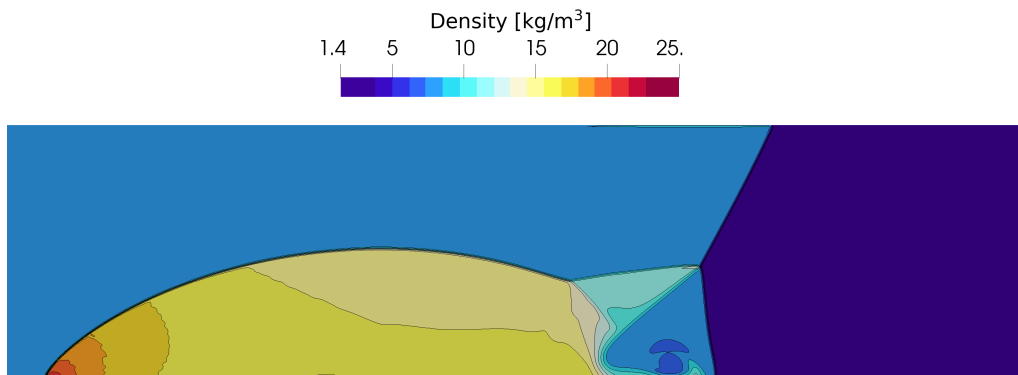


Figure 2: Density contours at $t = 200$ ms

21.2 Shock Tube - Two Fluid

This 1-D shock tube problem is taken from Zheng et al. (2011) and consists of gas and water initially separated at the center of the domain. The gas uses the van der Waals EOS and the water uses the Stiffened gas EOS. The shock tube is 1 m long, and is discretized into 300 computational cells.

The left state is defined as:

$$\rho = 1000 \text{ kg/m}^3 \quad \alpha = 1 \quad p = 1e9 \text{ Pa}$$

The right state is defined as:

$$\rho = 50 \text{ kg/m}^3 \quad \alpha = 0 \quad p = 1e5 \text{ Pa}$$

The *phaseProperties* dictionary is:

```
phases (fluid gas);
```

```
fluid
{
    type          basic;
    thermoType
    {
        transport  const;
        thermo      eConst;
        equationOfState stiffenedGas;
    }
    specie
    {
        molWeight  18.0;
    }
    transport
    {
        mu          0;
        Pr          1.0;
    }
    equationOfState
    {
        gamma       4.4;
    }
}
```

```

        a            6.0e8;
    }
    thermo
    {
        Cv            4186;
        Hf            0.0;
    }

    residualRho 1e-6;
    residualAlpha 1e-10;
}

gas
{
    type            basic;
    thermoType
    {
        transport    const;
        thermo        eConst;
        equationOfState vanderWaals;
    }
    specie
    {
        molWeight    28.97;
    }
    transport
    {
        mu            0;
        Pr            1.0;
    }
    equationOfState
    {
        gamma        1.4;
        a            5.0;
        b            1e-3;
    }
    thermo
    {

```

```

        Cv          718;
        Hf          0.0;
    }

    residualRho 1e-6;
    residualAlpha 1e-10;
}

```

and the *fvSchemes* dictionary is:

```

fluxScheme      HLLC;

ddtSchemes
{
    default      Euler;
    timeIntegrator RK2SSP;
}

gradSchemes
{
    default      leastSquares;
}

divSchemes
{
    default      none;
}

laplacianSchemes
{
    default      Gauss linear corrected;
}

interpolationSchemes
{
    default      linear;
    reconstruct(alpha) vanLeer;
    reconstruct(rho)   vanLeer;
}

```

```

        reconstruct(U)      vanLeerV;
        reconstruct(e)      vanLeer;
        reconstruct(p)      vanLeer;
        reconstruct(speedOfSound) vanLeer;
    }

    snGradSchemes
    {
        default              corrected;
    }

```

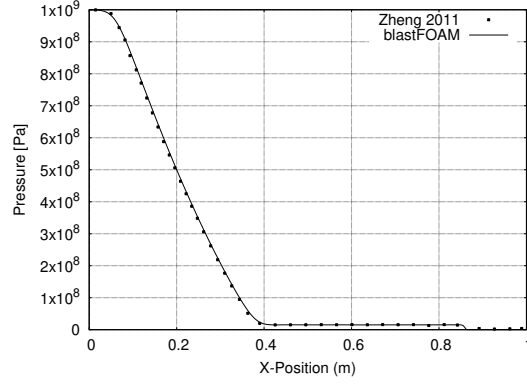
As seen above, the HLLC flux scheme is used with vanLeer limiters, and 2nd-order strong-stability-preserving time integration. The following commands were used to initialize and run the case in serial:

```

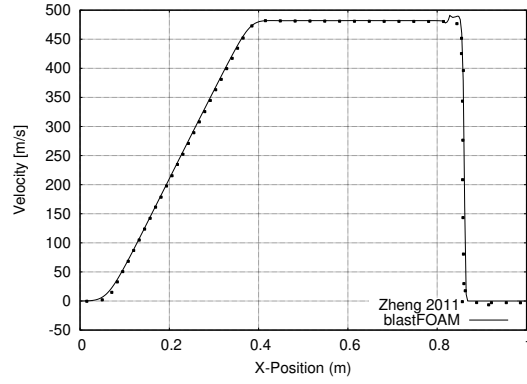
blockMesh
setFields
blastFoam

```

The results are shown in Fig. 3.



(a) Pressure



(b) X-velocity

Figure 3: Pressure and velocity fields at $t = 73$ ms

21.3 Two charge detonation

This case demonstrates the ability of **blastFoam** to solve complex problems involving multiple phases and detonation dynamics. Two charges, one composed of RDX and the other of TNT, are detonated in air. The RDX has a single detonation point at the center of its circular domain, and the TNT is detonated using the pressure based activation model. Fig. 4 shows the initial configuration, where the red line is the outline of the RDX charge, the red dot is the initiation point of the RDX charge, and the blue line is the outline of the TNT charge.

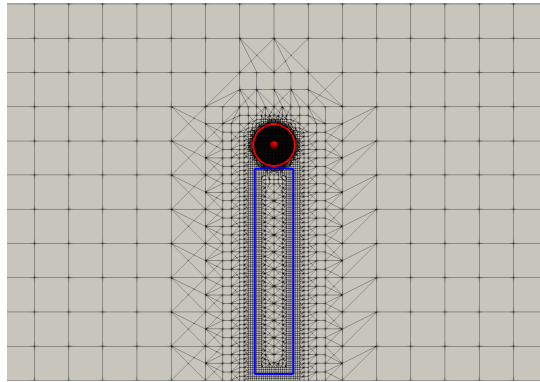


Figure 4: Initial charge configuration

The *phaseProperties* dictionary is defined as

```
phases (RDX tnt gas);

RDX
{
    type detonating;
    reactants
    {
        thermoType
        {
            transport    const;
            thermo        eConst;
            equationOfState Murnaghan;
        }
        equationOfState
        {
            rho0    1160;
            n        7.4;
            kappa    3.9e11;
            Gamma    0.35;
            pRef     101298;
        }
        specie
        {
```

```

        molWeight 222.12;
    }
    transport
    {
        mu      0;
        Pr      1;
    }
    thermodynamics
    {
        Cv      1130.78;
        Hf      0.0;
    }
}
products
{
    thermoType
    {
        transport  const;
        thermo     eConst;
        equationOfState JWL;
    }
    equationOfState
    {
        A      2.9867e11;
        B      4.11706e9;
        R1      4.95;
        R2      1.15;
        C      7.206147e8;
        omega   0.35;
        rho0    1160;
    }
    specie
    {
        molWeight 55.0;
    }
    transport
    {
        mu      0;

```

```

        Pr      1;
    }
    thermodynamics
    {
        Cv      1000;
        Hf      0.0;
    }
}

activationModel none;
initiation
{
    E0      4.0e9;
}

residualRho      1e-6;
residualAlpha    1e-10;
}

tnt
{
    type detonating;
    reactants
    {
        thermoType
        {
            transport  const;
            thermo      eConst;
            equationOfState BirchMurnaghan3;
        }
        equationOfState
        {
            rho0      1601; // Reference density [kg/m^3]
            K0        9.6e9; // Bulk modulus \units{Pa}
            K0Prime    6.6; // dK0/dp []
            Gamma      0.35; // Gruneisen coefficient
            pRef       101298; // Reference pressure \units{Pa}
        }
    }
}

```

```

specie
{
    molWeight 227.13;
}
transport
{
    mu      0;
    Pr      1;
}
thermodynamics
{
    Cv      1095;
    Hf      0.0;
}
}
products
{
    thermoType
    {
        transport  const;
        thermo     eConst;
        equationOfState JWL;
    }
    equationOfState
    {
        rho0      1601;
        A          371.21e9;
        B          3.23e9;
        R1         4.15;
        R2         0.95;
        omega      0.3;
    }
    specie
    {
        molWeight 55.0;
    }
    transport
    {

```

```

        mu      0;
        Pr      1;
    }
    thermodynamics
    {
        Cv      1000;
        Hf      0.0;
    }
}

activationModel pressureBased;
initiation
{
    E0      7.0e9; // Detonation energy \units{Pa}

    // Ignition coefficients
    I      0; // Ignition rate [1/s]

    // First stage coefficients
    G1      3.5083e-7; // Activation rate [Pa/s]
    c      1.0; // (1-lambda) exponent
    d      0.0; // lambda exponent
    y      1.3; // pressure exponent
    minLambda1 0.0; // 1st stage start value

    // Second stage coefficients
    G2      0.0; // Activation rate [Pa/s]

    pMin    1.1e5; // Cutoff pressure
}

residualRho    1e-6;
residualAlpha  1e-6;
}

gas
{
    type basic;

```

```

thermoType
{
    transport    const;
    thermo       eConst;
    equationOfState idealGas;
}
equationOfState
{
    gamma       1.4;
}
specie
{
    molWeight    28.97;
}
transport
{
    mu           0;
    Pr           1;
}
thermodynamics
{
    Cv           718;
    Hf           0;
}

residualRho     1e-6;
residualAlpha    1e-6;
}

```

The HLLC flux scheme is used with vanLeer limiters, and 2nd-order strong-stability-preserving time integration. The *fvSchemes* dictionary is defined as

```

fluxScheme       HLLC;

ddtSchemes
{
    default       Euler;
}

```

```

        timeIntegrator  RK2SSP;
    }

    gradSchemes
    {
        default  cellMDLimited leastSquares 1;
    }

    divSchemes
    {
        default          none;
        div(alphaRhoPhi.tnt,lambda.tnt) Riemann;
    }

    laplacianSchemes
    {
        default          Gauss linear corrected;
    }

    interpolationSchemes
    {
        default          linear;
        reconstruct(alpha)  vanLeer;
        reconstruct(rho)    vanLeer;
        reconstruct(U)      vanLeerV;
        reconstruct(e)      vanLeer;
        reconstruct(p)      vanLeer;
        reconstruct(speedOfSound) vanLeer;
        reconstruct(lambda.tnt) vanLeer;
    }

    snGradSchemes
    {
        default          corrected;
    }

```

Because the circular charge domain is not well represented by the hexahedral mesh generated by **blockMesh**, refinement around this region is used.

A maximum of four levels of refinement are used, with two cells between each level. This allows for a more accurate initial mass of the charges, and provides better initial resolution of the solution. The **setRefinedFields** utility is used where the *setFieldsDict* is defined as

```
fields (alpha.RDX alpha.tnt);
nBufferLayers 1;

defaultFieldValues
(
    volScalarFieldValue alpha.gas      1
    volScalarFieldValue alpha.RDX      0
    volScalarFieldValue alpha.tnt      0
);

regions
(
    cylinderToCell
    {
        refineInternal yes;
        level 6;

        p1 (0 0.55 -1);
        p2 (0 0.55 1);
        radius 0.05;

        fieldValues
        (
            volScalarFieldValue alpha.RDX  1
            volScalarFieldValue alpha.gas   0
        );
    }
    boxToCell
    {
        level 5;
        refineInternal no;
    }
);
```

```

    box (-0.05 0.01 -1) (0.05 0.5 1);

    fieldValues
    (
        volScalarFieldValue alpha.tnt    1
        volScalarFieldValue alpha.gas    0
    );
}
);

```

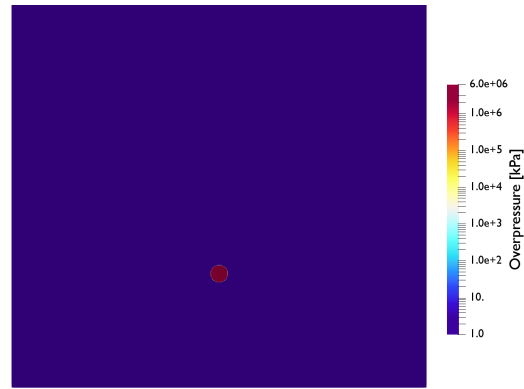
The following commands were used to run the case in parallel:

```

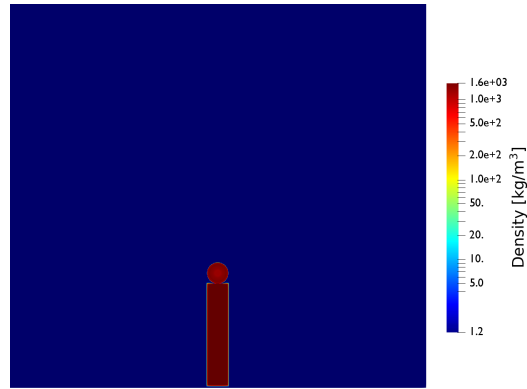
blockMesh
decomposePar
mpirun -np $nProcs setRefineFields -parallel
mpirun -np $nProcs blastFoam -parallel

```

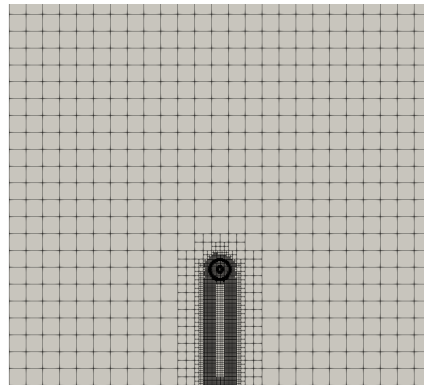
The overpressure, density, and updated mesh are shown in Fig. 5, Fig. 6, and Fig. 7 for time states at 0.005 ms, 0.05 ms, and 0.15 ms, respectively.



(a) Overpressure

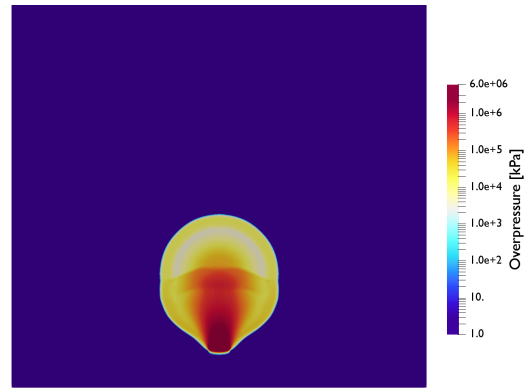


(b) Density

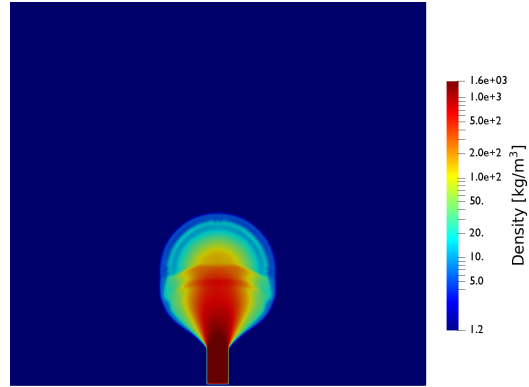


(c) Mesh

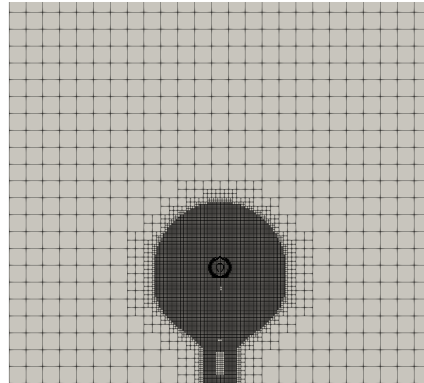
Figure 5: Instantaneous fields at $t = 0.005$ ms



(a) Overpressure

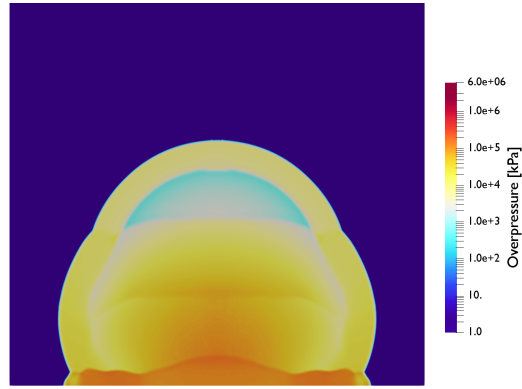


(b) Density

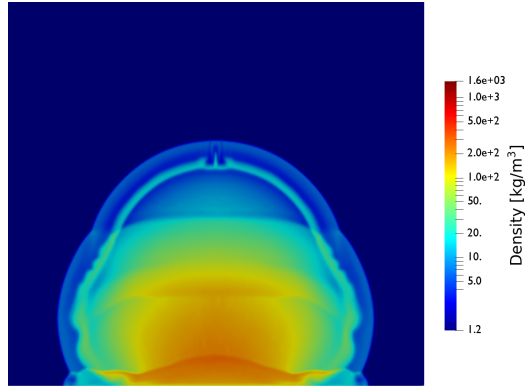


(c) Mesh

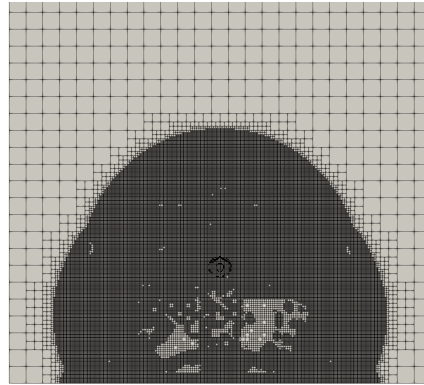
Figure 6: Instantaneous fields at $t = 0.05$ ms



(a) Overpressure



(b) Density



(c) Mesh

Figure 7: Instantaneous fields at $t = 0.15$ ms

21.4 Mine (buried charge)

This case demonstrates the simulation of an underground detonation using **blastEulerFoam**, i.e., a mine. A C-4 charge is buried in a volume of particles (sand) and detonated, leading to the ejection of explosive gases and particles into the air above. The multiphase phase model is used to represent the air and C-4 gases. The particle phase is represented by the granular phase model with a density of 1470 kg/m^3 and a diameter of $50 \text{ }\mu\text{m}$. The *phaseProperties* file is shown below:

```
phases (particles gas);

particles
{
    phaseType          granular;

    diameterModel constant;
    d                   50e-6;

    alphaMax            0.65;
    alphaMinFriction 0.5;

    type basic;
    thermoType
    {
        transport    constIso;
        thermo        eConst;
        equationOfState rhoConst;
    }
    specie
    {
        molWeight 100.0;
    }
    equationOfState
    {
        rho0       1470;
    }
    transport
```

```

{
    kappa      237;
}
thermodynamics
{
    Cv         987;           // Heat capacity
    Hf         0.0;
}

residualAlpha 1e-10;
residualRho   1e-6;

kineticTheoryCoeffs
{
    e          0.9;
}
}

gas
{
    phaseType      multiphase;
    diameterModel   constant;
    d              0;

    phases (c4 air);

    c4
    {
        type detonating;
        reactants
        {
            thermoType
            {
                transport  const;
                thermo      eConst;
                equationOfState idealGas;
            }
        }
    }
}

```

```

equationOfState
{
    rho0          1601;
    n              7.4;
    kappa          3.9e11;
    Gamma          0.35;
    gamma          1.35;
    pRef           101298;
}
specie
{
    molWeight      55.0;
}
transport
{
    mu             1.81e-5;           // Viscosity
    Pr             1;                 // Prandtl number
}
thermodynamics
{
    Cv             1400;               // Heat capacity
    Hf             0.0;
}
}
products
{
    thermoType
    {
        transport  const;
        thermo     eConst;
        equationOfState JWL;
    }

    equationOfState
    {
        rho0       1601;
        A           609.77e9;
        B           12.95e9;
    }
}

```

```

        R1      4.50;
        R2      1.4;
        omega   0.25;
    }
    specie
    {
        molWeight      55.0;
    }
    transport
    {
        mu      1.81e-5;           // Viscosity
        Pr      1;                 // Prandtl number
    }
    thermodynamics
    {
        Cv      1400;              // Heat capacity
        Hf      0.0;
    }
}
residualRho      1e-6;           // Minimum density of the phase
residualAlpha    1e-10;         // Minimum volume fraction used for division

activationModel linear;
initiation
{
    E0      9.0e9;
    points ((0 -1 0));
    vDet    7850;                 // Detonation velocity [m/s]
}
}

air
{
    type basic;
    thermoType
    {
        transport const;
        equationOfState idealGas;
    }
}

```

```

        thermo eConst;
    }
    specie
    {
        molWeight 28.97;
    }
    equationOfState
    {
        gamma      1.4;
    }
    transport
    {
        mu          1.81e-5;           // Viscosity
        Pr          1;                 // Prandtl number
    }
    thermodynamics
    {
        Cv          718;               // Heat capacity
        Hf          0.0;
    }
    residualRho    1e-6;
    residualAlpha   1e-10;
}

}

blending
{
    default
    {
        type none;
        continuousPhase gas;
    }
}

interfacialPressure
(
    (gas and particles)
    {

```

```

        type single;
        phase gas;
    }
);

interfacialVelocity
(
    (gas and particles)
    {
        type single;
        phase particles;
    }
);

aspectRatio
();

drag
(
    (particles in gas)
    {
        type GidaspowErgunWenYu;
        residualRe 1e-3;
        swarmCorrection
        {
            type none;
        }
    }
);

virtualMass
();

heatTransfer
(
    (particles in gas)
    {
        type RanzMarshall;
    }
);

```

```

    }
);

lift
();

wallLubrication
();

turbulentDispersion
();

kineticTheory
{
    residualAlpha      1e-6;

    radialModel SinclairJackson;
    viscosityModel Gidaspow;
    conductivityModel Gidaspow;
    granularPressureModel Lun;
    frictionalStressModel JohnsonJackson;

    JohnsonJacksonCoeffs
    {
        alphaMinFriction    0.5;
        Fr                  0.05;
        eta                  2;
        p                   5;
        phi                  28.5;
        alphaDeltaMin        1e-6;
    }
}

```

and the *fvSchemes* dictionary is:

```

fluxSchemes
{
    gas

```

```

    {
        fluxScheme    HLLC;
    }
    particles
    {
        fluxScheme      AUSM+Up;
        alphaMinFriction 0.5;
    }
}

ddtSchemes
{
    default      Euler;
    timeIntegrator RK2SSP 3;
}

gradSchemes
{
    default      cellMDLimited leastSquares 1;
}

divSchemes
{
    default      none;
    div(alphaRhoPhi.c4,lambda.c4) Gauss Riemann;
    div((((alpha.gas*rho.gas)*nuEff.gas)*dev2(T(grad(U.gas)))) Gauss linear;
    div(sigma.particles) Gauss linear;
}

laplacianSchemes
{
    default      Gauss linear corrected;
}

interpolationSchemes
{
    default      linear;
    reconstruct(alpha)      vanAlbada;
}

```

```

        reconstruct(rho)          vanAlbada;
        reconstruct(U)            vanAlbadaV;
        reconstruct(e)            vanAlbada;
        reconstruct(p)            vanAlbada;
        reconstruct(speedOfSound) vanAlbada;
        reconstruct(Theta)        vanAlbada;
        reconstruct(lambda)        vanAlbada;
    }

    snGradSchemes
    {
        default          corrected;
    }

```

Because the spherical charge is not well represented by the hexahedral mesh generated by **blockMesh**, refinement around this region is used. A maximum of four levels of refinement are used, with two cells between each level. This allows for a more accurate initial mass of the charges, and provides better initial resolution of the solution. The **setRefinedFields** utility is used, and the *setFieldsDict* is

```

fields (alpha.particles);
nBufferLayers 1;

defaultFieldValues
(
    volScalarFieldValue alpha.particles 0
    volScalarFieldValue alpha.c4        0
    volScalarFieldValue alpha.air       1
);

regions
(
    boxToCell
    {
        level 1;
        box (-25 -25 -0.1) (25 0 0.1);
    }

```

```

        fieldValues
        (
            volScalarFieldValue alpha.particles 0.55
        );
    }
    sphereToCell
    {
        centre (0 -1 0);
        radius 0.5;
        level 2;
        refineInternal yes;

        backup
        {
            centre (0 -1 0);
            radius 1;
        }

        fieldValues
        (
            volScalarFieldValue alpha.particles 0
            volScalarFieldValue alpha.c4        1
            volScalarFieldValue alpha.air        0
        );
    }
};

```

The following commands were used to run the case in parallel:

```

blockMesh
decomposePar
mpirun -np $nProcs setRefineFields -parallel
mpirun -np $nProcs blastEulerFoam -parallel

```

Contour plots of the pressure and particle volume fraction are shown in Fig. 8, Fig. 9, and Fig. 10 at time states of 0.0002 s, 0.002 s, and 0.005 s, respectively. Note that the results shown here were mirrored during post-processing; the simulation used an axisymmetric domain.

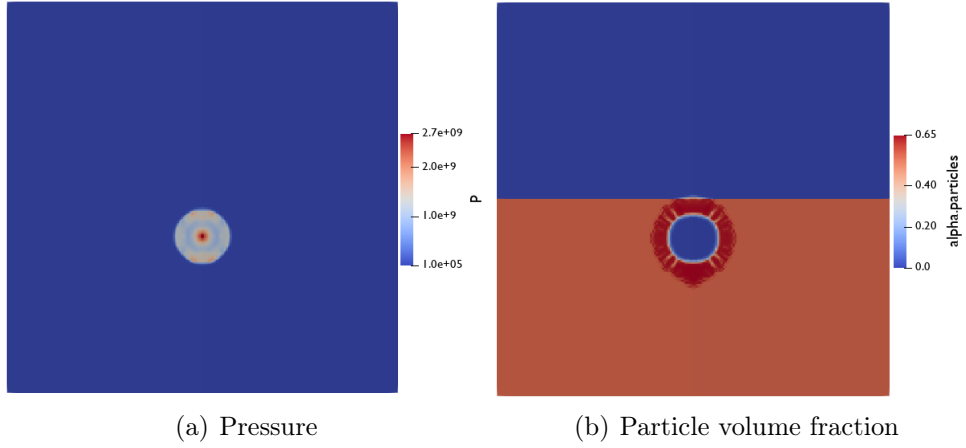


Figure 8: Instantaneous fields at $t = 0.0002$ s

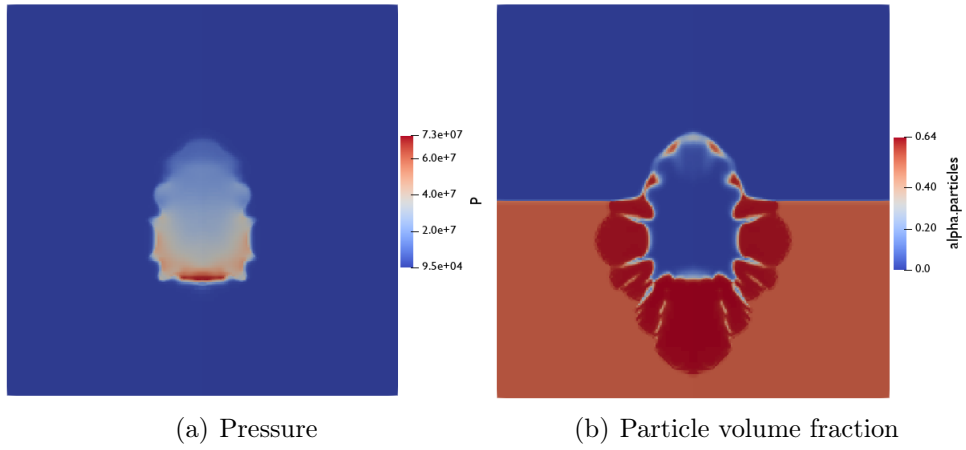


Figure 9: Instantaneous fields at $t = 0.002$ s

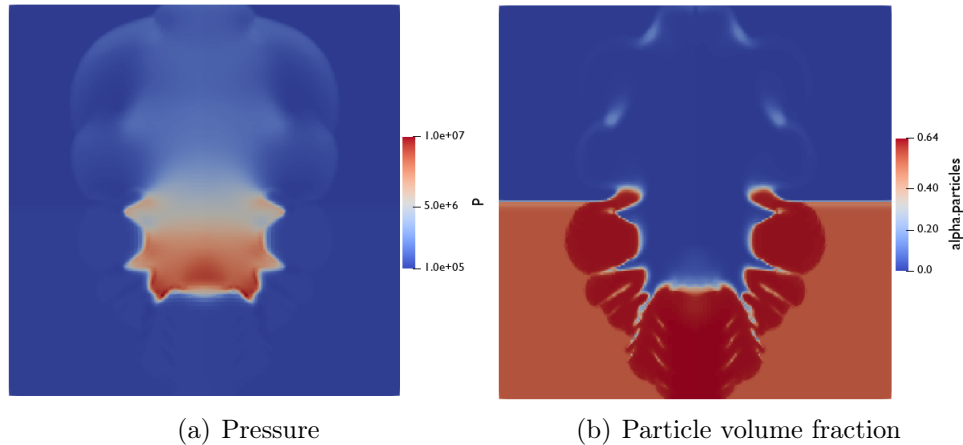


Figure 10: Instantaneous fields at $t = 0.005$ s

21.5 Bursting windows

This case will demonstrate how to set up a **blastFoam** case to use burst patches. Only the mesh generation and initial conditions will be described since the other parts of the case setup remain the same.

After constructing the initial mesh using block mesh with bounds $(-5, 0, 0)$ to $(10, 10, 10)$ with a symmetry condition for the x-z plane, **snappy-HexMesh** is used to cut cells from inside the building and refine the faces on the boundary of the "windows" searchableBox. Additionally a baffle will be created with names "window" and "windows_slave" which will be the intact window patches. The "geometry" and "castellatedMeshControls" sections are shown below

```
castellatedMesh on;
snap           on;
addLayers      off;

geometry
{
    building
    {
        type triSurfaceMesh;
        file "building.stl";
    }
}
```

```

        scale 0.001;
    }
    windows
    {
        type searchableBox;
        min (1.125 -2.375 -1);
        max (5.875 2.375 3.875);
    }
};

castellatedMeshControls
{
    // Refinement parameters
    // ~~~~~

    // If local number of cells is >= maxLocalCells on any processor
    // switches from from refinement followed by balancing
    // (current method) to (weighted) balancing before refinement.
    maxLocalCells 10000000;

    // Overall cell limit (approximately). Refinement will stop immediately
    // upon reaching this number so a refinement level might not complete.
    // Note that this is the number of cells before removing the part which
    // is not 'visible' from the keepPoint. The final number of cells might
    // actually be a lot less.
    maxGlobalCells 200000000;

    // The surface refinement loop might spend lots of iterations refining just a
    // few cells. This setting will cause refinement to stop if <= minimumRefine
    // are selected for refinement. Note: it will at least do one iteration
    // (unless the number of cells to refine is 0)
    minRefinementCells 2;

    maxLoadUnBalance 0.1;

    // Number of buffer layers between different levels.
    // 1 means normal 2:1 refinement restriction, larger means slower
    // refinement.

```

```

nCellsBetweenLevels 2;

resolveFeatureAngle 30;
allowFreeStandingZoneFaces false;

features
(
    {
        file "building.extendedFeatureEdgeMesh";
        level 0;
    }
);

refinementSurfaces
{
    building
    {
        level (2 2);
        patchInfo
        {
            type wall;
        }
    }

    windows
    {
        level (2 2);
        patchInfo
        {
            type wall;
        }

        faceZone windows;
        faceType baffle;
        cellZone building; // Important for adding baffles
    }
}

```

```

refinementRegions
{
    windows
    {
        mode distance;
        levels ((0.25 2));
    }
}

locationInMesh (0 1 1.5);
}

```

Once the mesh has been created with the new building, windows, and windows_slave patches and empty internal patch is created for load balancing and the **splitBaffles** utility is run to create unique points on either side of the baffle so mesh refinement can occur without limitations.

Next, to enable flow though the burst windows, two empty patches are added, "windowsBurst" and "windowsBurst_slave", and coupling is create using **createNonConformalCouples**. While the boundary conditions for these patches correspond to a wall, in reality overlapping faces are treated as internal so the boundary conditions are ignored. When window faces burst, faces belonging to "windows" and windows_slave" will be transferred to "windowsBurst" and "windowsBurst_slave" respectively. The initial conditions for U can be seen below. The same format will be used will all other initial fields.

```

dimensions      [0 0 0 0 0 0 0];

internalField    uniform 0;

boundaryField
{
    //- Set patchGroups for constraint patches
    #includeEtc "caseDicts/setConstraintTypes"

    building
    {
        type      zeroGradient;
    }
}

```

```

    }

    "windows.*"
    {
        type            slip;
    }

    outlet
    {
        type            zeroGradient;
    }

    ground
    {
        type            zeroGradient;
    }

    symmetry
    {
        type            symmetry;
    }
}

```

The *dynamicMeshDict* can be seen below:

```

topoChanger
{
    type list; // multiple topoChangers

    topoChangers
    {
        refinement
        {
            type            hexRefiner;
            errorEstimator    densityGradient;

            // How often to refine
            refineInterval    3;
        }
    }
}

```

```

// Refine field in between lower..upper
lowerRefineLevel 0.1;

// If value < unrefineLevel unrefine
unrefineLevel 0.1;

// Have slower than 2:1 refinement
nBufferLayers 2;

// Refine cells only up to maxRefinement levels
maxRefinement 1;

// Write the refinement level as a volScalarField
dumpLevel true;
}
burst
{
    type burst;

    patches
    (
        windows
        {
            // Coupled patches
            intactMasterPatch windows;
            intactSlavePatch windows_slave;
            burstMasterPatch windowsBurst;
            burstSlavePatch windowsBurst_slave;

            // Burst model definition
            burstModel pressure;
            partialBurst yes;
            pBurst 5e6;
            useDelta yes;

            // Interpolation method between master

```

```

        // and slave sides of the patch
        patchToPatch    nearest;
    }
};
}
}
}

```

The following commands were used to run the case in parallel:

```

surfaceFeatures
blockMesh

```

```

decomposePar -copyZero

```

```

mpirun -np 4 snappyHexMesh -overwrite -parallel
mpirun -np 4 addInternalPatch -parallel -noFields
mpirun -np 4 splitBaffles -overwrite -parallel

```

```

mpirun -np 4 addEmptyPatch windowsBurst wall -overwrite -noFields
mpirun -np 4 addEmptyPatch windowsBurst_slave wall -overwrite -noFields
runParallel createNonConformalCouples windowsBurst windowsBurst_slave -overwrite

```

```

mpirun -np 4 setRefinedFields

```

```

mpirun -np 4 blastFoam -parallel

```

Contour plots of the overpressure are on the building shown in Fig. 11 and Fig. 12 at time states of 0.0003 and 0.001 s respectively, and the threshold filter is used to remove burst faces. Note that the results shown here were mirrored during post-processing.

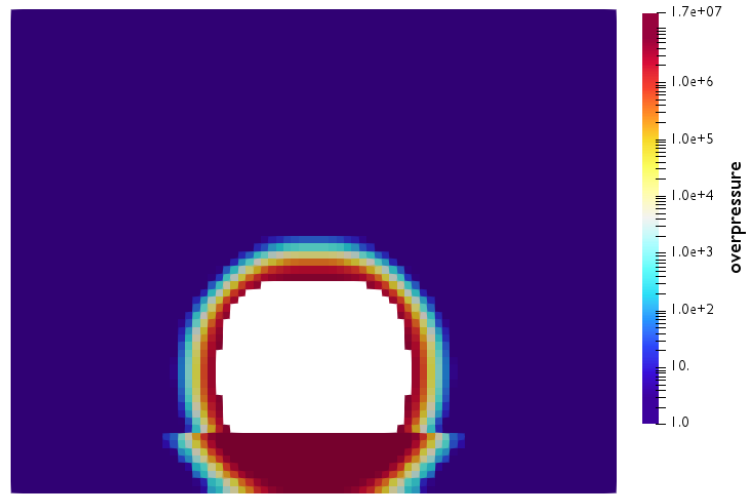


Figure 11: Overpressure field at $t = 0.0003$ s

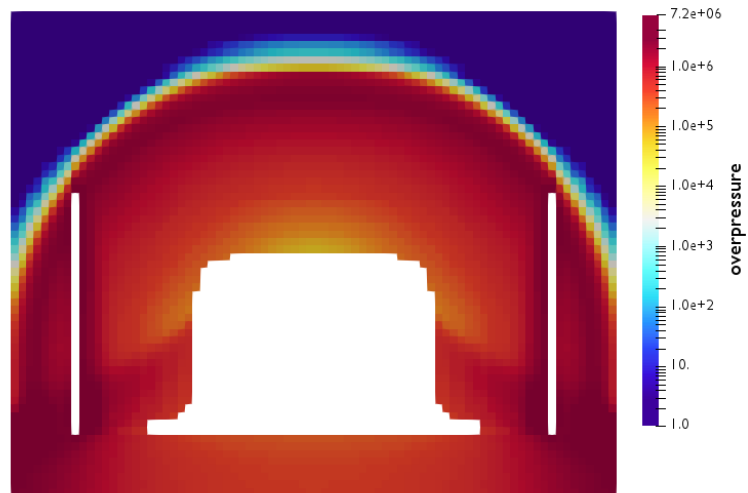


Figure 12: Overpressure field at $t = 0.0010$ s

References

- Agrawal, K., Loezos, P. N., Syamlal, M., and Sundaresan, S. (2001). The role of meso-scale structures in rapid gas–solid flows. *Journal of Fluid Mechanics*, 445:151 – 185.
- Benedict, M., Webb, G. B., and Rubin, L. C. (1942). An Empirical Equation for Thermodynamic Properties of Light Hydrocarbons and Their Mixtures II. Mixtures of Methane, Ethane, Propane, and n-Butane. *The Journal of Chemical Physics*, 10(12):747–758. Publisher: American Institute of Physics.
- Benyahia, S., Syamlal, M., and O’Brien, T. (2012). Summary of MFIx Equations 2012-1. Technical report.
- Broyden, C. G. (1965). A class of methods for solving nonlinear simultaneous equations. *Mathematics of Computation*, 19(92):577–593.
- Cardiff, P. and Demirdzic, I. (2021). Thirty years of the finite volume method for solid mechanics. *Archives of Computational Methods in Engineering*, 28(5):3721–3780. Publisher: Springer.
- Cardiff, P., Karac, A., De Jaeger, P., Jasak, H., Nagy, J., Ivankovic, A., and Tukovic, Z. (2018). An open-source finite volume toolbox for solid mechanics and fluid-solid interaction simulations. *arXiv preprint arXiv:1808.10736*.
- Chao, Z., Wang, Y., Jakobsen, J. P., Fernandino, M., and Jakobsen, H. A. (2011). Derivation and validation of a binary multi-fluid Eulerian model for fluidized beds. *Chemical Engineering Science*, 66(16):3605–3616.
- Chapman, S., Cowling, T. G., and Burnett, D. (1990). *The Mathematical Theory of Non-uniform Gases: An Account of the Kinetic Theory of Viscosity, Thermal Conduction and Diffusion in Gases*. Cambridge University Press. Google-Books-ID: Cbp5JP2OTrwC.
- Dittus, F. W. and Boelter, L. M. K. (1930). *Heat transfer in automobile radiators of the tubular type*. Number v. 2, no. 13 in University of California publications in engineering. University of California press, Berkeley, Calif.

- Doan, L. R. and Nickel, G. H. (1963). A subroutine for the equation of state of air. Technical Report RTD (WLR) TN63-2, Air Force Weapons Laboratory.
- Enwald, H., Peirano, E., and Almstedt, A.-E. (1996). Eulerian two-phase flow theory applied to fluidization. *International Journal of Multiphase Flow*, 22:21–66.
- Fedors, R. F. and Landel, R. F. (1979). An Empirical method of estimating the void fraction in mixtures of uniform particles of different size. *Powder Technology*, 23(2):225–231.
- Fox, R. O. (2019). A kinetic-based hyperbolic two-fluid model for binary hard-sphere mixtures. *Journal of Fluid Mechanics*, 877:282–329. Publisher: Cambridge University Press.
- Gao, J., Chang, J., Lu, C., and Xu, C. (2008). Experimental and computational studies on flow behavior of gas–solid fluidized bed with disparately sized binary particles. *Particuology*, 6(2):59–71.
- Gelperin, N. I. and Einstein, V. G. (1971). "Heat Transfer in Fluidized Beds. In *Fluidization*. Academic Press, New York.
- Gidaspow, D. (1994). *Multiphase Flow and Fluidization: Continuum and Kinetic Theory Descriptions*. Elsevier. Google-Books-ID: fHecceQyaYkC.
- Giroux, E. D. (1973). *HEMP User's Manual*. Lawrence Livermore National Laboratory, University of California. Google-Books-ID: 3XgpHQAA-CAAJ.
- Greenshields, C. J., Weller, H. G., Gasparini, L., and Reese, J. M. (2010). Implementation of semi-discrete, non-staggered central schemes in a collocated, polyhedral, finite volume framework, for high-speed viscous flows. *International journal for numerical methods in fluids*, 63(1):1–21.
- Gunn, D. J. (1978). Transfer of heat or mass to particles in fixed and fluidised beds. *International Journal of Heat and Mass Transfer*, 21:467–476.
- Henriques, F. C. and Moritz, A. R. (1947). Studies of Thermal Injury: I. The Conduction of Heat to and through Skin and the Temperatures Attained Therein. A Theoretical and an Experimental Investigation. *The American journal of pathology*, 23(4):530–549. Place: United States.

- Houim, R. W. and Oran, E. S. (2016). A multiphase model for compressible granular–gaseous flows: formulation and initial tests. *Journal of Fluid Mechanics*, 789:166–220.
- Hrenya, C. M. and Sinclair, J. L. (1997). Effects of particle-phase turbulence in gas-solid flows. *AIChE Journal*, 43(4):853–869. _eprint: <https://aiche.onlinelibrary.wiley.com/doi/pdf/10.1002/aic.690430402>.
- Huilin, L. and Gidaspow, D. (2003). Hydrodynamics of binary fluidization in a riser: CFD simulation using two granular temperatures. *Chemical Engineering Science*, 58(16):3777–3792.
- Johnston, I. A. (2005). The Noble-Abel Equation of State: Thermodynamic Derivations for Ballistics Modeling. Technical Report DSTO-TN-0670, Defense Science and Technology Organisation.
- Kochenderfer, M. J. and Wheeler, T. A. (2019). *Algorithms for Optimization*. MIT Press, Cambridge, MA, USA.
- Kurganov, A. and Tadmor, E. (2000). New high-resolution central schemes for nonlinear conservation laws and convection–diffusion equations. *Journal of Computational Physics*, 160(1):241–282.
- Kvaalen, E. (1991). A faster Broyden method. *BIT Numerical Mathematics*, 31(2):369–372.
- Lai, S., Houim, R. W., and Oran, E. S. (2018). Effects of particle size and density on dust dispersion behind a moving shock. *Physical Review Fluids*, 3(6):064306.
- Liou, M.-S. (2006). A sequel to AUSM, Part II: AUSM+–up for all speeds. *Journal of Computational Physics*, 214(1):137–170.
- Lun, C. K. K., Savage, S. B., Jeffrey, D. J., and Chepurniy, N. (1984). Kinetic theories for granular flow: inelastic particles in Couette flow and slightly inelastic particles in a general flowfield. *Journal of Fluid Mechanics*, 140:223–256.
- Luo, H., Baum, J. D., and Löhner, R. (2004). On the computation of multi-material flows using ALE formulation. *Journal of Computational Physics*, 194(1):304–328.

- Lutzky, M. (1964). The flow field behind a spherical detonation in TNT using the landau-stanyukovich equation of state for detonation products. Technical report, NAVAL ORDNANCE LAB WHITE OAK MD.
- Löhner, R. (1987). An adaptive finite element scheme for transient problems in CFD. *Computer Methods in Applied Mechanics and Engineering*, 61(3):323–338.
- Mader, C. (1963). Detonation properties of condensed explosives computed using the becker-kistiakowsky-wilson equation of state.
- Mehta, A. K. (1973). PREDICTION OF THERMAL INJURY TO SKIN AND ASSESSMENT OF BURN DAMAGE POTENTIAL OF SOME CLOTHING FABRICS. Technical report, Massachusetts Institute of Technology.
- Miller, P. J. (1995). A Reactive Flow Model with Coupled Reaction Kinetics for Detonation and Combustion in Non-Ideal Explosives. *MRS Online Proceedings Library Archive*, 418.
- Needham, C. E. (2018). *Blast Waves*. Shock Wave and High Pressure Phenomena. Springer International Publishing, Cham.
- OpenCFD Ltd. (2018a). *OpenFOAM - The Open Source CFD Toolbox - Programmer's Guide*. United Kingdom, 2 edition.
- OpenCFD Ltd. (2018b). *OpenFOAM - The Open Source CFD Toolbox - User's Guide*. United Kingdom, 2 edition.
- Rettenmaier, D., Deising, D., Ouedraogo, Y., Gjonaj, E., De Gersem, H., Bothe, D., Tropea, C., and Marschall, H. (2019). Load balanced 2D and 3D adaptive mesh refinement in OpenFOAM. *SoftwareX*, 10:100317.
- Sapareto, S. A. and Dewey, W. C. (1984). Thermal dose determination in cancer therapy. *International journal of radiation oncology, biology, physics*, 10(6):787–800. Place: United States.
- Sarkar, S., Erlebacher, G., Hussaini, M. Y., and Kreiss, H. O. (1991). The analysis and modelling of dilatational terms in compressible turbulence. *Journal of Fluid Mechanics*, 227:473–493.

- Schiller, L. and Naumann, A. (1933). Über die grundlegenden Berechnungen bei der Schwerkraftaufbereitung. *Ver. Deut. Ing.*, 77:318–320.
- Shyue, K.-M. (2001). A Fluid-Mixture Type Algorithm for Compressible Multicomponent Flow with Mie–Grüneisen Equation of State. *Journal of Computational Physics*, 171(2):678–707.
- Simo, J. C. and Hughes, T. J. R. (2000). *Computational Inelasticity*. Springer New York. Google-Books-ID: ftL2AJL8OPYC.
- Sinclair, J. L. and Jackson, R. (1989). Gas-particle flow in a vertical pipe with particle-particle interactions. *AIChE Journal*, 35(9):1473–1486.
- Souers, P. C., Anderson, S., Mercer, J., McGuire, E., and Vitello, P. (2000). JWL++: A Simple Reactive Flow Code Package for Detonation. page 5.
- Spiteri, R. J. and Ruuth, S. J. (2002). A New Class of Optimal High-Order Strong-Stability-Preserving Time Discretization Methods. *SIAM Journal on Numerical Analysis*, 40(2):469–491.
- Stoll, A. M. and Greene, L. C. (1959). Relationship between pain and tissue damage due to thermal radiation. *Journal of Applied Physiology*, 14(3):373–382. tex.eprint: <https://doi.org/10.1152/jappl.1959.14.3.373>.
- Syamral, M., Rogers, W., and O’Brien, T. (1993). MFIX documentation theory guide. Technical Report DOE/METC–94/1004, 10145548.
- Takata, A. N., Zaneveld, L., and Richter, W. (1977). Laser-induced thermal damage of skin. Final report, September 1976–April 1977. Technical report, IIT Research Inst., Chicago, IL (USA), United States.
- Tillotson, J. H. (1962). Metallic Equations of State For Hypervelocity Impact. *General Atomic Report GA-3216. 1962. Technical Report*.
- Toro, E. F., Spruce, M., and Speares, W. (1994). Restoration of the contact surface in the HLL-Riemann solver. *Shock waves*, 4(1):25–34.
- Wardlaw, A., Mckeown, R., and Luton, A. (1998). Coupled hydrocode prediction of underwater explosion damage. Technical report, NAVAL SURFACE WARFARE CENTER INDIAN HEAD DIV MD.

- Williams, M. L., Landel, R. F., and Ferry, J. D. (1955). The Temperature Dependence of Relaxation Mechanisms in Amorphous Polymers and Other Glass-forming Liquids. *Journal of the American Chemical Society*, 77(14):3701–3707. Publisher: American Chemical Society.
- Woodward, P. and Colella, P. (1984). The numerical simulation of two-dimensional fluid flow with strong shocks. *Journal of Computational Physics*, 54(1):115–173.
- Xie, W., Zhang, R., Lai, J., and Li, H. (2019). An accurate and robust HLLC-type Riemann solver for the compressible Euler system at various Mach numbers. *International Journal for Numerical Methods in Fluids*, 89(10):430–463.
- Xue, Q., Heindel, T. J., and Fox, R. O. (2011). A CFD model for biomass fast pyrolysis in fluidized-bed reactors. *Chemical Engineering Science*, 66(11):2440–2452.
- Yu, A. B. and Standish, N. (1987). Porosity calculations of multi-component mixtures of spherical particles. *Powder Technology*, 52(3):233–241.
- Zeman, O. (1990). Dilatation dissipation: The concept and application in modeling compressible mixing layers. *Physics of Fluids A: Fluid Dynamics*, 2(2):178–188. tex.eprint: https://pubs.aip.org/aip/pof/article-pdf/2/2/178/12260733/178_1_online.pdf.
- Zheng, H., Shu, C., and Chew, Y. (2008). An object-oriented and quadrilateral-mesh based solution adaptive algorithm for compressible multi-fluid flows. *Journal of Computational Physics*, 227(14):6895–6921.
- Zheng, H. W., Shu, C., Chew, Y. T., and Qin, N. (2011). A solution adaptive simulation of compressible multi-fluid flows with general equation of state. *International Journal for Numerical Methods in Fluids*, 67(5):616–637.
- Zheng, Z. and Zhao, J. (2016). Unreacted equation of states of typical energetic materials under static compression: A review. *Chinese Physics B*, 25(7):076202.